MLREML: modelos lineares mistos por REML de informação média para o melhoramento de plantas e animais

Guia do usuário, fundamentos teóricos e exemplos resolvidos com dados reais — versão 1.0.0

Osval Antonio Montesinos López, Abelardo Montesinos-López, José Crossa, J. Cricelio Montesinos-López

Outubro de 2026

Sumário

1 Introdução 5
1.1 O que é o MLREML 5
1.2 A quem se destina este guia 5
1.3 Instalação 6
1.4 Funcionalidades e sua correspondência com o ASReml 6
1.5 Um primeiro exemplo 8
2 Fundamentos teóricos 9
2.1 O modelo linear misto 9
2.2 Estruturas como produtos diretos 9
2.3 Verossimilhança restrita 10
2.4 O algoritmo de informação média 10
2.5 Estimação dos efeitos fixos e predição dos efeitos aleatórios 12
2.6 Médias preditas e erros padrão das diferenças 12
2.7 Testes de hipóteses 13
2.8 Funções dos parâmetros de variância 13
2.9 Matrizes de parentesco 13
2.10 Custo computacional 14
2.11 Efeitos correlacionados de vários fatores: modelos direto–materno 14
2.12 O motor esparso: REML com as equações do modelo misto 15
2.13 Graus de liberdade de Kenward–Roger 17
3 Sintaxe do modelo 18
3.1 A função `mlreml()` 18
3.2 Termos aleatórios 18
3.3 Estruturas residuais 20
3.4 Modelos multivariados 21
3.5 Nomes dos parâmetros, valores iniciais e restrições 21
3.6 O objeto ajustado 21

4 Exemplo 1: um ensaio de aveia em delineamento alfa-látice 22
4.1 Os dados 22
4.2 Variedades como efeitos fixos: médias ajustadas 23
4.3 Variedades como efeitos aleatórios: BLUP e herdabilidade 24
4.4 Comparação de médias ajustadas e BLUPs 25
4.5 Vale a pena o delineamento em blocos incompletos? 26
4.6 Diagnóstico 26
4.7 Verificação com lme4 26
5 Exemplo 2: análise espacial de um ensaio de trigo 27
5.1 Os dados 27
5.2 Modelo de blocos completos 28
5.3 Modelo espacial AR1 × AR1 28
5.4 Tendência global 29
5.5 Variograma amostral 30
5.6 Ganho em precisão 30
5.7 Tendência espacial estimada 31
5.8 Dados com espaçamento irregular 31
6 Exemplo 3: ensaios multiambientes e modelos fator-analíticos 32
6.1 Os dados 32
6.2 A sequência de modelos 33
6.3 Interpretação do modelo FA(1) 34
6.4 Predições por ambiente e desempenho geral 35
6.5 Recomendações para a análise MET 37
7 Exemplo 4: predição genômica (GBLUP) com 599 linhagens de trigo 37
7.1 Os dados 37
7.2 Controle de qualidade e matriz genômica 38
7.3 GBLUP em um ambiente 39
7.4 Predição de linhagens não fenotipadas 40
7.5 Validação cruzada: pedigree, marcadores e ambos 40
7.6 GBLUP multiambiente 41
7.7 Notas práticas 42
8 Exemplo 5: modelo animal com pedigree em gado leiteiro 43
8.1 Os dados 43
8.2 Preparação do pedigree 43
8.3 Modelo animal para a primeira lactação 44
8.4 Modelo de repetibilidade com todas as lactações 45
8.5 Efeito materno 46
8.6 Análise bivariada de leite e gordura 47

8.7 Efeitos genéticos diretos e maternos 48
8.8 Grandes conjuntos de dados: o motor esparso 51
9 Exemplo 6: GBLUP de passo único com indivíduos genotipados e não genotipados 53
9.1 Contexto 53
9.2 Alinhamento de G com o pedigree e construção de H 53
9.3 Comparação por validação 54
9.4 Passo único com o motor geral 55
9.5 Recomendações 55
10 Exemplo 7: medidas repetidas e regressão aleatória 55
10.1 Correlação residual: crescimento de crianças 55
10.2 Regressão aleatória linear: privação de sono 56
10.3 Regressão aleatória com polinômios de Legendre: curvas de crescimento 58
11 Exemplo 8: análise dialélica 60
11.1 Os dados 60
11.2 O modelo 60
11.3 Interpretação 61
11.4 Capacidades de combinação preditas 62
11.5 Comparação de modelos 62
12 Exemplo 9: ensaio de geração precoce sem repetições 63
12.1 Os dados 63
12.2 Modelos 64
12.3 Consequências para a seleção 65
12.4 Tendência espacial e testemunhas 66
13 Uso avançado e solução de problemas 67
13.1 Histórico de iterações 67
13.2 Valores iniciais e reinícios 68
13.3 Parâmetros fixos e verossimilhança perfilada 69
13.4 Componentes de variância conhecidos 70
13.5 Efeitos fixos aliased 70
13.6 Parâmetros na fronteira 71
13.7 Matrizes de parentesco mal condicionadas 71
13.8 Escala das variáveis 72
13.9 Memória e tempo 72
13.10 Problemas comuns 72
14 Estudo de simulação: propriedades dos estimadores 73
14.1 Delineamento em blocos completos com genótipos aleatórios 73
14.2 Tamanho do teste da razão de verossimilhanças na fronteira 75
14.3 Modelo espacial 76

15 Validação numérica, desempenho e limitações 77
15.1 Comparação com outros softwares 77
15.2 Tempo de computação e memória 80
15.3 Estratégias para grandes conjuntos de dados 82
15.4 Limitações da versão 1.0.0 83
16 Resumo das funções 84
16.1 Equivalências de sintaxe com o ASReml-R 85
16.2 Guia rápido de modelos 85
16.3 Funções pós-ajuste 86
Referências 87
Apêndice: manual de referência das funções 90
16.4 `MLREML-package`: MLREML: modelos lineares mistos por REML de Informação Média 90
16.5 `mlreml`: Ajusta um modelo linear misto por REML de Informação Média 91
16.6 `summary.mlreml`: Métodos print e summary para ajustes mlreml 94
16.7 `coef.mlreml`: Extrai efeitos fixos e aleatórios 94
16.8 `wald`: Testes de Wald para efeitos fixos 95
16.9 `predict.mlreml`: Médias preditas com erros padrão e erros padrão das diferenças 96
16.10 `vpredict`: Funções dos parâmetros de variância com erros padrão pelo método delta 97
16.11 `lrt`: Teste da razão de verossimilhanças entre modelos aninhados 97
16.12 `vc_matrix`: Matriz de covariância e de correlação estimada de um componente estruturado 98
16.13 `variogram`: Variograma amostral dos resíduos de um ensaio de campo 98
16.14 `plot.mlreml`: Gráficos de diagnóstico 99
16.15 `pedigree_prepare`: Prepara um pedigree 99
16.16 `inbreeding_coef`: Coeficientes de endogamia (Meuwissen e Luo, 1992) 99
16.17 `Amatrix`: Matriz de parentesco do numerador (aditiva) 100
16.18 `Ainverse`: Inversa esparsa da matriz de parentesco aditivo 100
16.19 `snp_qc`: Controle de qualidade de uma matriz de SNPs 101
16.20 `Gmatrix`: Matriz de parentesco genômico 101
16.21 `align_G`: Alinha uma matriz genômica com a matriz de pedigree 102
16.22 `Hmatrix`: Matriz de parentesco de passo único H e sua inversa 102
16.23 `bend`: Torna uma matriz simétrica positiva definida (bending) 103
16.24 `gblup`: GBLUP rápido por decomposição espectral 103
16.25 `cv_gblup`: Validação cruzada K-fold do GBLUP 104

1 Introdução

1.1 O que é o MLREML

O MLREML é um pacote do R que ajusta modelos lineares mistos por máxima verossimilhança restrita (REML) com o algoritmo de Informação Média (Average Information, AI). Ele foi projetado para os problemas típicos do melhoramento de plantas e animais: estimação de componentes de variância e covariância, herdabilidades e correlações genéticas, predição de valores genéticos (BLUP, EBV e GEBV), análise de ensaios multiambientes, análise espacial de experimentos de campo, modelos com informação de pedigree e genômica, modelos multivariados e modelos longitudinais.

O nome combina as duas ideias que organizam o pacote: modelos lineares mistos (mixed linear models) e sua estimação por REML. Sua sintaxe é inspirada na do ASReml (Gilmour et al., 2015; Butler et al., 2017), o programa de referência nesta área, de modo que um usuário do ASReml pode ler e escrever modelos no MLREML quase sem aprendizado adicional. O MLREML é, no entanto, um software de código aberto independente: não contém código do ASReml, não depende dele e não requer licença.

A ideia central do pacote é a mesma que torna o ASReml poderoso. Um modelo linear misto é definido não apenas pelos efeitos que contém, mas pela forma como esses efeitos e os resíduos se distribuem e se correlacionam. Por isso, cada termo aleatório no MLREML é escrito como um produto direto de estruturas de variância: por exemplo, `us(env):vm(gen, G)` declara efeitos genéticos de cada genótipo em cada ambiente, correlacionados entre genótipos segundo a matriz genômica **G** e entre ambientes segundo uma matriz de covariância não estruturada estimada a partir dos dados.

O MLREML também contém a covariância entre os efeitos genéticos diretos e maternos (e, em geral, entre efeitos aleatórios de vários fatores que compartilham uma matriz de parentesco) por meio do termo `str()`; um segundo motor de cálculo baseado nas equações do modelo misto com matrizes esparsas, para conjuntos de dados com centenas de milhares de registros; e graus de liberdade do denominador de Kenward–Roger para os testes F dos efeitos fixos.

1.2 A quem se destina este guia

Este guia foi escrito para pesquisadores, professores e estudantes de pós-graduação em genética quantitativa, melhoramento de plantas e animais, estatística experimental e bioestatística. Pressupõe conhecimentos básicos de R e de modelos lineares. A teoria dos modelos mistos é apresentada integralmente no Capítulo 2, com a notação usada ao longo de todo o documento, de modo que o guia também pode ser usado como material de curso.

O documento tem três partes. A primeira (Capítulos 1 a 3) descreve o modelo, o algoritmo de estimação e a sintaxe. A segunda (Capítulos 4 a 12) apresenta exemplos completos com dados reais de domínio público: um ensaio de aveia em alfa-látice, um ensaio de trigo com análise espacial, um ensaio multiambiente de milho, 599 linhagens de trigo do CIMMYT genotipadas com marcadores, registros de produção de leite de vacas Holandesas com seu pedigree, curvas de crescimento de frangos, tempos de reação de pessoas em um estudo de privação de sono, um dialelo completo de tabaco e um ensaio de trigo de geração precoce sem repetições. A terceira parte (Capítulos 13 a 16) trata do uso avançado, de um estudo de simulação, da validação numérica do pacote em relação a outros programas, de suas limitações e de um resumo de todas as funções; a referência completa das funções é dada no apêndice.

1.3 Instalação

O pacote é distribuído como um arquivo-fonte `MLREML_1.0.0.tar.gz` disponível em https://github.com/osval78/MLREML. Ele depende dos pacotes `Matrix` (versão 1.6-0 ou posterior, que acompanha o R) e `Rcpp`, e de `methods`, `stats`, `graphics` e `utils`. O pacote contém código C++ (a inversa esparsa usada pelo motor esparso e o cálculo dos coeficientes de endogamia), de modo que é instalado a partir do código-fonte com um compilador: Rtools no Windows, as ferramentas de linha de comando do Xcode no macOS (`xcode-select --install`) e as ferramentas de desenvolvimento habituais no Linux (por exemplo, o pacote `r-base-dev` do Debian e do Ubuntu):

```
install.packages("Rcpp")
install.packages("MLREML_1.0.0.tar.gz", repos = NULL, type = "source")
library(MLREML)
```

Os exemplos deste guia usam dados dos pacotes `agridat` (Wright, 2024), `BGLR` (Pérez e de los Campos, 2014), `pedigreemm` (Vazquez et al., 2010) e `lme4` (Bates et al., 2015), e comparam os resultados com `lme4`, `nlme` (Pinheiro e Bates, 2000) e `pbkrtest` (Halekoh e Højsgaard, 2014). Nenhum deles é necessário para usar o MLREML:

```
install.packages(c("agridat", "BGLR", "pedigreemm", "lme4", "nlme",
"pbkrtest"))
```

Para obter álgebra linear rápida, o R deve ser usado com uma biblioteca BLAS otimizada (OpenBLAS, Intel MKL ou Accelerate no macOS). O tempo de um ajuste com o motor denso é dominado por fatorações de Cholesky de matrizes $n \times n$, e uma BLAS otimizada as acelera de 10 a 30 vezes em relação à BLAS de referência. O motor esparso também se beneficia, porque os blocos densos de sua fatoração esparsa e de sua inversa esparsa são calculados com BLAS e LAPACK.

1.4 Funcionalidades e sua correspondência com o ASReml

A tabela abaixo resume as funcionalidades do ASReml normalmente mencionadas no melhoramento de plantas e animais e como obtê-las no MLREML. Ela indica claramente o que está totalmente implementado, o que é parcial e o que não está disponível nesta versão.

Funcionalidades do ASReml e seu equivalente no MLREML.

Funcionalidade (ASReml)	No MLREML	Situação
Modelos lineares mistos, dados desbalanceados	`mlreml()`	Completo
REML com informação média (AI)	Algoritmo AI com amortecimento	Completo
Componentes de variância com erros padrão	`summary()`, inversa da matriz AI	Completo
Herdabilidade, correlações, funções de componentes	`vpredict()` (método delta)	Completo
Modelo animal com pedigree, EBV	`vm(animal, A)`, `Amatrix()`, `Ainverse()`	Completo
Coeficientes de endogamia	`inbreeding_coef()` (Meuwissen e Luo)	Completo
GBLUP	`vm(gen, G)`, `Gmatrix()`, `gblup()`	Completo

Funcionalidade (ASReml)	No MLREML	Situação
GBLUP de passo único (matriz H)	`Hmatrix()`, `align_G()`, `bend()`	Completo
Controle de qualidade de SNPs	`snp_qc()`	Básico
Modelos multivariados, correlação genética	`cbind(y1, y2) ~ trait`, `us(trait)`	Completo
Ensaios multiambientes (MET), G×E	`diag()`, `us()`, `fa(k)`	Completo
Variâncias residuais heterogêneas	`residual = ~ diag(env):units`	Completo
Análise espacial linha × coluna	`residual = ~ ar1(row):ar1(col)`	Completo
Dados espaciais irregulares	`expc(x)`, `sexp(x, y)`	Completo
Medidas repetidas, ambiente permanente	`id(animal)` + `vm(animal, A)`, `ar1()`	Completo
Regressão aleatória	`us(leg(x, k)):id(animal)`, `pol()`	Completo
Efeitos maternos	termos adicionais `id(dam)`, `vm(dam, A)`	Completo
Dialelos (CGC, CEC, recíprocos)	`mm(p1, p2)`, `id(cross)`	Completo
BLUP, PEV, confiabilidade	`coef(fit, "random")`	Completo
Médias preditas e SED	`predict(fit, classify = ...)`	Completo
Testes de Wald incrementais e condicionais	`wald()`	Completo (os testes condicionais respeitam a marginalidade)
Comparação de modelos (LRT, AIC, BIC)	`lrt()`, `AIC`, `BIC`	Completo
Variograma amostral	`variogram()`	Completo
Covariância genética direta–materna	`str(animal, dam, K = A)`	Completo
Métodos de matrizes esparsas (milhões de registros)	`engine = "sparse"` (equações do modelo misto)	Completo para termos `id`, `vm`, `diag`, `us` e `str()`
Graus de liberdade de Kenward–Roger	`wald(fit, ddf = "KR")`	Completo (motor denso)
Modelos lineares mistos generalizados	—	Não disponível

O MLREML tem dois motores de cálculo. O motor denso usa o método baseado em **V**: é exato, simples e fácil de verificar, e permite estruturas muito gerais sem código especial, mas limita seu uso confortável a cerca de 5.000–6.000 registros em um computador pessoal com 8 GB de memória. O motor esparso resolve as equações do modelo misto com uma fatoração de Cholesky esparsa e a inversa esparsa da matriz de coeficientes, como fazem os programas usados nas avaliações genéticas nacionais; ele suporta as estruturas mais usadas com grandes

conjuntos de dados e ajusta modelos animais com centenas de milhares de registros no mesmo computador (Seções 2.12 e 15.2). Para GBLUP com um único efeito genético, a função `gblup()` oferece uma alternativa exata baseada na decomposição espectral de **G**. O Capítulo 15 documenta tempos de computação e uso de memória.

1.5 Um primeiro exemplo

Para mostrar a forma geral de uma análise, considere um experimento simulado com 30 genótipos avaliados em 4 blocos completos. O modelo é

$$y_{ij} = \mu + \beta_j + g_i + e_{ij}, \qquad g_i \sim N\left(0, \sigma_g^2\right), \qquad e_{ij} \sim N(0, \sigma_e^2),$$

com blocos fixos e genótipos aleatórios.

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)), gen = factor(rep(1:30,
4)))
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d, trace = FALSE)
summary(fit)

## MLREML fit (AI-REML)
## Observations: 120   Fixed-effect columns: 4   Iterations: 6 (converged)
## REML log-likelihood: -192.9292   AIC: 389.86   BIC: 395.37
##
## Variance parameters:
##                 estimate std.error z.ratio bound
## id(gen)!sigma2   1.9621    0.5686    3.45     P
## R!units!sigma2   0.7992    0.1212    6.60     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

A saída mostra o número de observações, as colunas de efeitos fixos, as iterações, a log-verossimilhança restrita e os critérios de informação. Em seguida, apresenta cada parâmetro de variância com seu erro padrão aproximado (obtido da inversa da matriz de informação média), a razão z e um código de fronteira. A herdabilidade no sentido amplo em base de média de genótipos e seu erro padrão são obtidos com `vpredict()`:

```
vpredict(fit, H2 ~ V1 / (V1 + V2 / 4))

##    estimate std.error
## H2   0.9076   0.02802

head(coef(fit, "random", "id(gen)"))

##   level     blup    pev     se reliability
## 1     1  0.14700 0.2407 0.4906      0.8773
## 2     2 -1.03768 0.2407 0.4906      0.8773
## 3     3 -0.29958 0.2407 0.4906      0.8773
## 4     4  1.02779 0.2407 0.4906      0.8773
## 5     5  0.47789 0.2407 0.4906      0.8773
## 6     6 -0.08843 0.2407 0.4906      0.8773
```

A segunda instrução retorna os BLUPs dos genótipos com sua variância do erro de predição (PEV), erro padrão e confiabilidade $1 - \text{PEV}/\sigma_g^2$.

2 Fundamentos teóricos

Este capítulo apresenta o modelo ajustado pelo MLREML, a verossimilhança restrita, o algoritmo de informação média tal como implementado, e as quantidades que o pacote calcula após o ajuste: estimadores dos efeitos fixos, preditores dos efeitos aleatórios, variâncias do erro de predição, médias preditas, testes de hipóteses e funções dos parâmetros de variância.

2.1 O modelo linear misto

Seja $\mathbf{y}$ o vetor de n observações. O modelo linear misto geral é

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{u}_k + \mathbf{e}, \qquad (2.1)$$

em que $\boldsymbol{\beta}$ é o vetor de p efeitos fixos com matriz de delineamento $\mathbf{X}$ ($n \times p$, de posto completo após a remoção de colunas redundantes), $\mathbf{u}_k$ é o vetor de q_k efeitos aleatórios do termo k com matriz de delineamento $\mathbf{Z}_k$, e $\mathbf{e}$ é o vetor de resíduos. Assume-se que

$$\mathbf{u}_k \sim N\big(\mathbf{0}, \mathbf{G}_k(\boldsymbol{\theta})\big), \qquad \mathbf{e} \sim N\big(\mathbf{0}, \mathbf{R}(\boldsymbol{\theta})\big), \qquad \operatorname{Cov}(\mathbf{u}_k, \mathbf{u}_l) = \mathbf{0}, \ \operatorname{Cov}(\mathbf{u}_k, \mathbf{e}) = \mathbf{0},$$

e que as matrizes $\mathbf{G}_k$ e $\mathbf{R}$ dependem de um vetor de parâmetros de variância $\boldsymbol{\theta}$ (variâncias, covariâncias, correlações, cargas fatoriais). A distribuição marginal das observações é

$$\mathbf{y} \sim N(\mathbf{X}\boldsymbol{\beta}, \mathbf{V}), \qquad \mathbf{V}(\boldsymbol{\theta}) = \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{G}_k \mathbf{Z}_k{}' + \mathbf{R}. \qquad (2.2)$$

Toda a flexibilidade do modelo está na forma de $\mathbf{G}_k$ e $\mathbf{R}$. Em uma análise de variância tradicional, cada $\mathbf{G}_k$ é $\sigma_k^2 \mathbf{I}$ e $\mathbf{R} = \sigma_e^2 \mathbf{I}$. Em genética quantitativa, ao contrário, $\mathbf{G}$ pode ser $\sigma_a^2 \mathbf{A}$ (pedigree), $\sigma_g^2 \mathbf{G}_{\text{genomic}}$, ou $\boldsymbol{\Sigma}_E \otimes \mathbf{K}$ em um ensaio multiambiente; e $\mathbf{R}$ pode ser diagonal com variâncias diferentes por ambiente, ou ter a estrutura autorregressiva separável de um ensaio de campo.

2.2 Estruturas como produtos diretos

O MLREML constrói cada $\mathbf{G}_k$ como um produto direto (de Kronecker) de matrizes pequenas, uma para cada fator que define o termo:

$$\mathbf{G}_k = \sigma_k^2 \, \boldsymbol{\Sigma}_{k1} \otimes \boldsymbol{\Sigma}_{k2} \otimes \cdots \otimes \boldsymbol{\Sigma}_{kC}. \qquad (2.3)$$

Por exemplo, para efeitos de genótipo dentro de ambiente com E ambientes e N genótipos, $\mathbf{u}$ tem EN elementos ordenados com o genótipo variando mais rapidamente, e $\mathbf{G} = \boldsymbol{\Sigma}_E \otimes \mathbf{K}$. A matriz de delineamento do termo é o produto face-splitting (produto de Khatri–Rao por linhas) das matrizes de delineamento de cada fator: se $\mathbf{z}_{c,i}{}'$ é a linha i da matriz de delineamento $\mathbf{Z}_c$ do fator c, a linha i de $\mathbf{Z}_k$ é $\mathbf{z}_{1,i}{}' \otimes \mathbf{z}_{2,i}{}' \otimes \cdots$.

Uma propriedade algébrica simples permite calcular a contribuição de cada termo para $\mathbf{V}$ sem nunca formar a $\mathbf{Z}_k$ ou a $\mathbf{G}_k$ completas. Para o produto face-splitting,

$$\mathbf{Z}_k(\boldsymbol{\Sigma}_1 \otimes \boldsymbol{\Sigma}_2)\mathbf{Z}_k{}' = (\mathbf{Z}_1 \boldsymbol{\Sigma}_1 \mathbf{Z}_1{}') \circ (\mathbf{Z}_2 \boldsymbol{\Sigma}_2 \mathbf{Z}_2{}'), \qquad (2.4)$$

em que $\circ$ é o produto de Hadamard (elemento a elemento). Quando o fator c é uma variável de classificação, $\mathbf{Z}_c$ tem um único um por linha e $\mathbf{Z}_c \boldsymbol{\Sigma}_c \mathbf{Z}_c{}'$ é simplesmente a matriz $\boldsymbol{\Sigma}_c$ indexada pelos níveis de cada observação: seu elemento (i, j) é $\Sigma_c[\ell_c(i), \ell_c(j)]$, em que $\ell_c(i)$ é o nível do fator c na observação i. Assim, uma matriz genômica de 6.547 animais é usada apenas nas

linhas e colunas dos animais com registros, e o termo `us(env):vm(gen, G)` contribui com os elementos $\Sigma_E[e_i, e_j]\, K[g_i, g_j]$ para $\mathbf{V}$. A identidade (2.4) é o que permite ao MLREML combinar livremente qualquer estrutura com qualquer outra.

Os resíduos são tratados da mesma forma. A estrutura padrão, `units`, é a identidade sobre as observações. A estrutura `diag(env):units` fornece $R_{ij} = \sigma^2_{e(i)}\delta_{ij}$, variâncias residuais diferentes por ambiente. A estrutura `ar1(row):ar1(col)` fornece

$$R_{ij} = \sigma^2\, \rho_r^{|r_i - r_j|}\, \rho_c^{|c_i - c_j|}, \qquad (2.5)$$

o modelo espacial separável de Gilmour et al. (1997), em que r_i e c_i são a linha e a coluna da parcela i. Como a matriz é construída a partir das coordenadas de cada observação, o ensaio não precisa ser uma grade completa: as parcelas perdidas simplesmente não aparecem.

2.3 Verossimilhança restrita

Os efeitos fixos são removidos da verossimilhança por contrastes de erro $\mathbf{K}'\mathbf{y}$, com $\mathbf{K}'\mathbf{X} = \mathbf{0}$, cuja distribuição não depende de $\boldsymbol{\beta}$ (Patterson e Thompson, 1971). A menos de uma constante, a log-verossimilhança restrita é

$$\ell_R(\boldsymbol{\theta}) = -\frac{1}{2}\left[\log|\mathbf{V}| + \log|\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}| + \mathbf{y}'\mathbf{P}\mathbf{y} + (n-p)\log(2\pi)\right], \qquad (2.6)$$

com a matriz de projeção

$$\mathbf{P} = \mathbf{V}^{-1} - \mathbf{V}^{-1}\mathbf{X}(\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}. \qquad (2.7)$$

O REML corrige o viés da máxima verossimilhança devido à estimação dos efeitos fixos; em um delineamento balanceado, seus estimadores coincidem com os da análise de variância quando estes são positivos. O valor relatado pelo MLREML inclui a constante $(n-p)\log(2\pi)$ e é igual ao de `lme4` e `nlme`, o que facilita a comparação entre programas (Capítulo 15). Duas verossimilhanças REML só são comparáveis se os modelos tiverem exatamente os mesmos efeitos fixos.

2.4 O algoritmo de informação média

Seja $\mathbf{V}_i = \partial\mathbf{V}/\,\partial\theta_i$. As primeiras derivadas de (2.6), que formam o vetor escore, são

$$s_i = \frac{\partial \ell_R}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(\mathbf{P}\mathbf{V}_i) - \mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{y}\right]. \qquad (2.8)$$

A matriz de informação observada contém traços de produtos de quatro matrizes $n \times n$, que são muito custosos, e a informação de Fisher contém $\operatorname{tr}(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j)$, também custosa. Gilmour, Thompson e Cullis (1995) observaram que a média das duas, quando $\mathbf{V}$ é linear nos parâmetros, é

$$\mathcal{I}_{A,ij} = \frac{1}{2}\,\mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\mathbf{P}\mathbf{y}, \qquad (2.9)$$

que requer apenas produtos matriz–vetor: se $\mathbf{w}_i = \mathbf{V}_i\mathbf{P}\mathbf{y}$ e $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_m]$, então $\mathcal{I}_A = 1/2\;\mathbf{W}'\mathbf{P}\mathbf{W}$. A matriz de informação média é semidefinida positiva e é uma aproximação do tipo Gauss–Newton para a informação também quando $\mathbf{V}$ não é linear em $\boldsymbol{\theta}$ (correlações autorregressivas, cargas fatoriais). A atualização é

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \left[\mathcal{I}_A^{(t)}\right]^{-1}\mathbf{s}^{(t)}. \qquad (2.10)$$

2.4.1 Implementação no MLREML

Em cada iteração, o motor denso do MLREML (i) constrói $\mathbf{V}$ com a identidade (2.4); (ii) calcula seu fator de Cholesky $\mathbf{V} = \mathbf{U}'\mathbf{U}$, $\log|\mathbf{V}| = 2\sum \log u_{ii}$, $\mathbf{V}^{-1}$ e $\mathbf{P}$; (iii) para cada parâmetro livre forma $\mathbf{V}_i$ — novamente por meio de produtos de Hadamard, substituindo a matriz do componente por sua derivada — e calcula $tr(\mathbf{PV}_i) = \sum_{jk} P_{jk} V_{i,jk}$ e $\mathbf{w}_i$; e (iv) obtém $\mathbf{s}$ e $\mathcal{I}_A$. O motor esparso obtém as mesmas quantidades a partir das equações do modelo misto (Seção 2.12). As derivadas de todas as estruturas são analíticas; por exemplo, para uma estrutura autorregressiva $\partial\rho^d / \partial\rho = d\rho^{d-1}$ para toda distância $d > 0$, incluindo distâncias menores que um, e zero para $d = 0$, para uma matriz não estruturada a derivada em relação a σ_{ab} é $\mathbf{E}_{ab} + \mathbf{E}_{ba}$, e para um modelo fator-analítico $\mathbf{\Lambda\Lambda}' + \mathbf{\Psi}$ a derivada em relação à carga λ_{lm} é $\mathbf{e}_l \boldsymbol{\lambda}_m{}' + \boldsymbol{\lambda}_m \mathbf{e}_l{}'$.

O passo de Newton puro (2.10) pode sair do espaço paramétrico (variâncias negativas, correlações maiores que um, matrizes que não são positivas definidas) ou diminuir a verossimilhança longe do ótimo. O MLREML inclui três salvaguardas:

1. **Amortecimento de Levenberg–Marquardt.** O passo é calculado como $\left(\mathcal{I}_A + \lambda\,\mathrm{diag}(\mathcal{I}_A)\right)^{-1}\mathbf{s}$. Começa com $\lambda = 0$ (passo AI puro); se o candidato não for válido ou reduzir ℓ_R, λ é multiplicado por 10. Quando λ é grande, o passo se aproxima de um passo curto de gradiente, que sempre aumenta a verossimilhança. Após um passo aceito, λ é reduzido.
2. **Restrições de fronteira.** As variâncias são mantidas acima de um limite inferior positivo muito pequeno; a correlação de `cs` dentro de $(-1/(L-1) + 10^{-4}, 0.999)$, porque a matriz de simetria composta $L \times L$ é positiva definida apenas para $\rho > -1/(L-1)$; e as correlações autorregressivas dentro de $(-0.99, 0.999)$ quando todas as distâncias são inteiras e dentro de $(10^{-6}, 0.999)$ caso contrário, porque ρ^d não está definido para $\rho < 0$ e d não inteiro. Um parâmetro que termina em seu limite é marcado com o código `B` na saída, como no ASReml. Uma variância que está em seu limite e cujo escore, ou cujo passo de Newton, aponta para fora do espaço paramétrico é mantida no limite e removida do sistema AI, e o passo dos demais parâmetros é calculado condicionalmente (um método de conjunto ativo). Recortar, em vez disso, o passo conjunto poderia deixar todos os parâmetros praticamente parados em um ponto que não é um máximo.
3. **Projeção semidefinida.** Se uma matriz não estruturada (`us`) deixa de ser positiva definida, ela é projetada sobre o cone das matrizes semidefinidas positivas, substituindo seus autovalores negativos por um valor positivo pequeno. Isso permite que o algoritmo se aproxime de soluções na fronteira (por exemplo, correlações genéticas próximas de um), que são frequentes em ensaios multiambientes.

A convergência é declarada quando a mudança em ℓ_R é menor que `tol` (por padrão 10^{-6}) e o decremento de Newton $\mathbf{s}'\mathcal{I}_A^{-1}\mathbf{s}$, calculado sem os parâmetros mantidos em seu limite, é menor que 10^{-3}, ou quando a mudança relativa nos parâmetros é desprezível. Neste último caso, se o decremento de Newton ainda for grande — o que acontece quando uma matriz não estruturada converge para uma matriz singular —, um aviso recomenda verificar as estimativas e considerar uma estrutura reduzida.

Valores iniciais. Calcula-se a variância v_p dos resíduos de mínimos quadrados ordinários do modelo de efeitos fixos, que é dividida igualmente entre os termos aleatórios e o resíduo. As variâncias específicas de cada nível (estruturas `diag`, `us`, `fa`) começam na variância residual observada dentro de cada nível multiplicada pela mesma proporção, o que fornece valores na escala correta em modelos multivariados com caracteres medidos em unidades diferentes. As

correlações autorregressivas começam no valor que dá uma correlação de 0,5 na distância mínima entre níveis. Os usuários podem fornecer seus próprios valores iniciais com o argumento `start` e fixar parâmetros com `fix`.

Erros padrão. A inversa de $\mathcal{I}_A$ no ótimo aproxima a matriz de covariância assintótica de $\widehat{\boldsymbol{\theta}}$ e fornece os erros padrão na tabela de componentes. Para parâmetros na fronteira, essa aproximação não é válida e o erro padrão deve ser ignorado.

2.5 Estimação dos efeitos fixos e predição dos efeitos aleatórios

Dado $\widehat{\boldsymbol{\theta}}$, o estimador de mínimos quadrados generalizados (BLUE empírico) dos efeitos fixos e sua variância são

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}\mathbf{y}, \qquad \operatorname{Var}(\widehat{\boldsymbol{\beta}}) = \mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \tag{2.11}$$

e o melhor preditor linear não viesado (BLUP empírico) de cada termo aleatório é

$$\widehat{\mathbf{u}}_k = \mathbf{G}_k\mathbf{Z}_k'\mathbf{V}^{-1}\left(\mathbf{y} - \mathbf{X}\widehat{\boldsymbol{\beta}}\right) = \mathbf{G}_k\mathbf{Z}_k'\mathbf{P}\mathbf{y}. \tag{2.12}$$

Essas expressões são a solução das equações do modelo misto de Henderson (1975),

$$\begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{X}'\mathbf{R}^{-1}\mathbf{Z} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{Z}'\mathbf{R}^{-1}\mathbf{Z} + \mathbf{G}^{-1} \end{bmatrix} \begin{bmatrix} \widehat{\boldsymbol{\beta}} \\ \widehat{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{y} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{y} \end{bmatrix},$$

mas o motor denso do MLREML as avalia diretamente na forma (2.12), que não requer a inversão de $\mathbf{G}$. Portanto, matrizes de parentesco singulares (por exemplo, uma matriz genômica com menos marcadores do que indivíduos, ou linhagens geneticamente idênticas) não causam problemas, desde que $\mathbf{V}$ seja positiva definida. O motor esparso (Seção 2.12) resolve as próprias equações e precisa de $\mathbf{G}^{-1}$.

Quando um termo usa uma matriz de parentesco com mais níveis do que os observados — por exemplo, a matriz $\mathbf{A}$ de um pedigree com ancestrais sem registros, ou a matriz genômica de candidatos à seleção não fenotipados —, (2.12) também prediz esses níveis, porque $\mathbf{G}_k\mathbf{Z}_k'$ inclui suas covariâncias com os indivíduos observados.

Variância do erro de predição. A inversa da matriz de coeficientes das equações de Henderson, $\mathbf{C}$, fornece a covariância conjunta dos erros de estimação e de predição. Seus blocos, em termos de $\mathbf{V}$, são

$$\mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \quad \mathbf{C}_{21} = -\mathbf{G}\mathbf{Z}'\mathbf{V}^{-1}\mathbf{X}\mathbf{C}_{11}, \quad \mathbf{C}_{22} = \operatorname{Var}(\widehat{\mathbf{u}} - \mathbf{u}) = \mathbf{G} - \mathbf{G}\mathbf{Z}'\mathbf{P}\mathbf{Z}\mathbf{G}. \tag{2.13}$$

A diagonal de $\mathbf{C}_{22}$ é a PEV de cada BLUP. A confiabilidade, amplamente usada no melhoramento animal, é $r_i^2 = 1 - PEV_i/G_{ii}$, a correlação ao quadrado entre o valor genético verdadeiro e o predito. O motor denso do MLREML calcula a PEV por blocos de linhas de $\mathbf{G}\mathbf{Z}'$, sem formar $\mathbf{G}$, de modo que a memória necessária é proporcional a n vezes o tamanho do bloco; o motor esparso as obtém da diagonal da inversa esparsa de $\mathbf{C}$ (Seção 2.12).

2.6 Médias preditas e erros padrão das diferenças

Uma predição é uma combinação linear $\boldsymbol{\tau} = \mathbf{L}\boldsymbol{\beta} + \mathbf{M}\mathbf{u}$ (Welham et al., 2004). A função `predict()` constrói $\mathbf{L}$ calculando a média das linhas da matriz de delineamento sobre os níveis dos fatores fixos que não estão na classificação (média simples) e avaliando as covariáveis em sua média; $\mathbf{M}$ seleciona os BLUPs dos termos aleatórios cujos fatores estão todos na classificação, e os demais termos aleatórios são ignorados porque sua esperança é zero. A variância do erro de predição é

$$\operatorname{Var}(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = \mathbf{L}\mathbf{C}_{11}\mathbf{L}' + \mathbf{L}\mathbf{C}_{12}\mathbf{M}' + \mathbf{M}\mathbf{C}_{21}\mathbf{L}' + \mathbf{M}\mathbf{C}_{22}\mathbf{M}', \qquad (2.14)$$

e o erro padrão da diferença entre duas predições é $\mathrm{SED}_{ab} = \sqrt{v_{aa} + v_{bb} - 2v_{ab}}$. O SED médio é a base da herdabilidade generalizada de Cullis, Smith e Coombes (2006), $H^2_C = 1 - \overline{\mathrm{SED}}^2 / \left(2\sigma^2_g\right)$, útil em ensaios desbalanceados.

2.7 Testes de hipóteses

Efeitos fixos. A estatística de Wald de uma hipótese $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$, com $q = rank(\mathbf{L})$, é $W = \left(\mathbf{L}\widehat{\boldsymbol{\beta}}\right)'\left(\mathbf{L}\mathbf{C}_{11}\mathbf{L}'\right)^{-1}\mathbf{L}\widehat{\boldsymbol{\beta}}$, assintoticamente χ^2_q. A função `wald()` fornece duas versões para cada termo T da fórmula fixa. A estatística *incremental* ajusta T para os termos que o precedem na fórmula (testes sequenciais ou do "tipo I"): se $\mathbf{U}'\mathbf{U} = \mathbf{C}_{11}^{-1} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{X}$ é a fatoração de Cholesky, $\mathbf{L}$ é formada pelas linhas de $\mathbf{U}$ correspondentes às colunas de T, e $W_T = \sum_{j\in T}\left(\mathbf{U}\widehat{\boldsymbol{\beta}}\right)^2_j$. A estatística *condicional* ajusta T para todos os termos que não o contêm, respeitando o princípio da marginalidade como faz o ASReml (testes do "tipo II"): em `y ~ A * B` o efeito principal `A` é ajustado para `B`, mas não para `A:B`, e a interação é ajustada para ambos os efeitos principais. Ela é calculada como a estatística incremental após reordenar as colunas de modo que T venha depois dos termos que não o contêm e antes dos que o contêm. Ajustar cada termo para todos os demais, incluindo as interações que o contêm, testaria, com contrastes de tratamento, o efeito de `A` no nível de referência de `B`. Cada estatística é acompanhada de um teste F. Por padrão (`ddf = "auto"`), os ajustes do motor denso usam o método de Kenward e Roger (1997), que ajusta a matriz de covariância de $\widehat{\boldsymbol{\beta}}$ e a estatística F pela estimação dos parâmetros de variância e fornece os graus de liberdade do denominador de cada teste (Seção 2.13). Com `ddf = "residual"`, e sempre para os ajustes do motor esparso, $F = W/q$ com $n - p$ graus de liberdade no denominador; essa aproximação é adequada com muitos dados e liberal em experimentos pequenos.

Parâmetros de variância. Dois modelos aninhados com os mesmos efeitos fixos são comparados com a razão de verossimilhanças $LR = 2\left(\ell_{R,1} - \ell_{R,0}\right)$. Ao testar se uma única variância é zero, o parâmetro está na fronteira e a distribuição nula é uma mistura 50:50 de χ^2_0 e χ^2_1 (Self e Liang, 1987; Stram e Lee, 1994), de modo que o valor p é a metade do de uma χ^2_1. A função `lrt()` aplica essa correção. Modelos não aninhados são comparados com o AIC $= -2\ell_R + 2m$ e o BIC $= -2\ell_R + m\log(n - p)$, em que m é o número de parâmetros de variância.

2.8 Funções dos parâmetros de variância

Herdabilidades, repetibilidades e correlações genéticas são funções não lineares $f(\boldsymbol{\theta})$. Seu erro padrão aproximado é obtido pelo método delta,

$$\operatorname{Var}\left[f\left(\widehat{\boldsymbol{\theta}}\right)\right] \approx \nabla f\left(\widehat{\boldsymbol{\theta}}\right)' \mathcal{I}_A^{-1} \nabla f\left(\widehat{\boldsymbol{\theta}}\right), \qquad (2.15)$$

que é o que `vpredict()` faz, com o gradiente calculado por diferenças centrais. Os parâmetros são denominados `V1`, `V2`, … na ordem da tabela de componentes.

2.9 Matrizes de parentesco

Pedigree. O parentesco aditivo entre dois indivíduos é o dobro de seu coeficiente de coancestria. A matriz $\mathbf{A}$ é construída com o método tabular: se s e d são os pais de i, $A_{ji} = 1/2\left(A_{js} + A_{jd}\right)$ para $j < i$ e $A_{ii} = 1 + 1/2\,A_{sd}$, em que $F_i = 1/2\,A_{sd}$ é o coeficiente de endogamia. A inversa é construída diretamente com as regras de Henderson (1976) com endogamia (Quaas, 1976): com $b_i = [1/2 - 1/4\,(F_s + F_d)]^{-1}$ (usando $F = -1$ para pais desconhecidos), b_i é somado ao elemento (i,i), $-b_i/2$ a (i,s), (s,i), (i,d), (d,i), e $b_i/4$ a (s,s),

(d,d), (s,d), (d,s). Os coeficientes F são calculados com o algoritmo de Meuwissen e Luo (1992).

Genômica. Com a matriz $\mathbf{M}$ de genótipos codificados como $0/1/2$ e as frequências alélicas p_j, VanRaden (2008) define $\mathbf{W} = \mathbf{M} - 2\mathbf{p}'$ e $\mathbf{G} = \mathbf{W}\mathbf{W}'/\left[2\sum_j p_j\left(1-p_j\right)\right]$. A alternativa padronizada divide cada coluna de $\mathbf{W}$ por $\sqrt{2p_j(1-p_j)}$ e o produto pelo número de marcadores. O GBLUP com $\sigma_g^2\mathbf{G}$ é equivalente à regressão ridge sobre os marcadores (Meuwissen et al., 2001; VanRaden, 2008).

Passo único. Quando apenas alguns indivíduos são genotipados, Legarra et al. (2009) combinam pedigree e genômica em

$$\mathbf{H} = \mathbf{A} + \begin{bmatrix}\mathbf{A}_{12}\mathbf{A}_{22}^{-1}\\ \mathbf{I}\end{bmatrix}(\mathbf{G}-\mathbf{A}_{22})[\mathbf{A}_{22}^{-1}\mathbf{A}_{21} \quad \mathbf{I}], \qquad \mathbf{H}^{-1} = \mathbf{A}^{-1} + \begin{bmatrix}\mathbf{0} & \mathbf{0}\\ \mathbf{0} & \tau\mathbf{G}^{-1} - \omega\mathbf{A}_{22}^{-1}\end{bmatrix}, \qquad (2.16)$$

em que o índice 2 corresponde aos indivíduos genotipados. Para colocar $\mathbf{G}$ e $\mathbf{A}_{22}$ na mesma escala, $\mathbf{G}$ é alinhada como $\alpha + \beta\mathbf{G}$ de modo que as médias de sua diagonal e de todos os seus elementos coincidam com as de $\mathbf{A}_{22}$ (Christensen et al., 2012), e é combinada com uma pequena proporção de $\mathbf{A}_{22}$ para garantir que seja inversível.

2.10 Custo computacional

A etapa dominante de cada iteração do motor denso é a fatoração e a inversão de $\mathbf{V}$, com custo $O(n^3)$ em tempo e $O(n^2)$ em memória. Cada parâmetro acrescenta $O(n^2)$ operações para seu traço e seu vetor $\mathbf{w}_i$. A predição dos BLUPs e de suas PEV custa $O(qn^2)$ com memória $O(n \times \text{block})$. O custo do motor esparso é o da fatoração de Cholesky esparsa da matriz de coeficientes das equações do modelo misto e o de sua inversa esparsa, ambos proporcionais a $\sum_i |S_i|^2$, em que $|S_i|$ é o número de elementos não nulos abaixo da diagonal na coluna i do fator de Cholesky, mais uma resolução com o fator por parâmetro; sua memória é proporcional ao número de elementos não nulos do fator (Seção 2.12). O Capítulo 15 apresenta os tempos medidos.

2.11 Efeitos correlacionados de vários fatores: modelos direto–materno

Em muitas espécies o fenótipo de um indivíduo depende de seus próprios genes (efeito genético direto) e dos genes de sua mãe, expressos por meio do ambiente materno que ela proporciona: produção de leite, habilidade materna, capacidade uterina (efeito genético materno). Os dois efeitos de um mesmo animal costumam ser correlacionados, e o modelo de Willham (1963) os trata conjuntamente:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_d\mathbf{a}_d + \mathbf{Z}_m\mathbf{a}_m + \cdots + \mathbf{e}, \qquad Var\begin{pmatrix}\mathbf{a}_d\\ \mathbf{a}_m\end{pmatrix} = \begin{pmatrix}\sigma_d^2 & \sigma_{dm}\\ \sigma_{dm} & \sigma_m^2\end{pmatrix} \otimes \mathbf{A} = \boldsymbol{\Sigma} \otimes \mathbf{A}, \qquad (2.17)$$

em que $\mathbf{a}_d$ e $\mathbf{a}_m$ contêm os valores genéticos diretos e maternos de todos os animais do pedigree, $\mathbf{Z}_d$ liga cada registro ao animal que o produziu e $\mathbf{Z}_m$ o liga à mãe desse animal. Ambos os vetores são indexados pelos mesmos animais, mas um registro recebe o valor direto de um animal e o valor materno de outro, de modo que o termo não pode ser escrito como um produto direto de componentes e a identidade (2.4) não se aplica. Sua contribuição para $\mathbf{V}$ é uma soma de três matrizes,

$$\mathbf{Z}(\boldsymbol{\Sigma}\otimes\mathbf{A})\mathbf{Z}' = \sigma_d^2\,\mathbf{Z}_d\mathbf{A}\mathbf{Z}_d' + \sigma_m^2\,\mathbf{Z}_m\mathbf{A}\mathbf{Z}_m' + \sigma_{dm}(\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m' + \mathbf{Z}_m\mathbf{A}\mathbf{Z}_d'), \qquad \mathbf{Z} = [\mathbf{Z}_d, \mathbf{Z}_m], \qquad (2.18)$$

que é linear nos três parâmetros; suas derivadas $\mathbf{V}_i$ são as três matrizes de (2.18). O elemento (i, j) de $\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m{}'$ é o parentesco entre o animal do registro i e a mãe do registro j. Os registros cuja mãe é desconhecida têm uma linha de zeros em $\mathbf{Z}_m$: contribuem para a estimação dos efeitos diretos e são mantidos na análise.

O MLREML ajusta esse modelo com o termo `str(f1, f2, ..., K = K, cov = "us")`, que generaliza (2.17) para $m \geq 2$ fatores cujos níveis são as linhas da mesma matriz $\mathbf{K}$: $\mathbf{u} = (\mathbf{u}_1{}', \ldots, \mathbf{u}_m{}')'$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$ e $Var(\mathbf{u}) = \boldsymbol{\Sigma} \otimes \mathbf{K}$, com $\boldsymbol{\Sigma}$ não estruturada (`cov = "us"`, $m(m+1)/2$ parâmetros) ou diagonal (`cov = "diag"`). O BLUP do efeito a e sua variância do erro de predição decorrem de (2.12) e (2.13):

$$\hat{\mathbf{u}}_a = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{K}\mathbf{Z}_b{}'\mathbf{P}\mathbf{y}, \qquad PEV_{a,l} = \Sigma_{aa}K_{ll} - \mathbf{g}_{a,l}{}'\mathbf{P}\mathbf{g}_{a,l}, \qquad \mathbf{g}_{a,l}{}' = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{k}_l{}'\mathbf{Z}_b{}', \tag{2.19}$$

em que $\mathbf{k}_l{}'$ é a linha l de $\mathbf{K}$. Além da herdabilidade direta $h^2_d = \sigma^2_d/\sigma^2_P$ e da correlação genética direta–materna $r_{dm} = \sigma_{dm}/(\sigma_d\sigma_m)$, a herdabilidade total de Willham (1972), $h^2_T = (\sigma^2_d + 1.5\sigma_{dm} + 0.5\sigma^2_m)/\sigma^2_P$, mede a resposta à seleção sobre o fenótipo. A variância genética materna só é identificável se o pedigree ligar mães com registros próprios ou de parentes, e as estimativas de σ_{dm} são frequentemente negativas e imprecisas (Meyer, 1992); se necessário, um efeito materno de ambiente permanente é adicionado como um termo separado `id(dam)`.

2.12 O motor esparso: REML com as equações do modelo misto

O método das seções anteriores trabalha com a matriz $n \times n$ $\mathbf{V}$ e é exato e geral, mas seu custo cresce com n^3. Os programas usados nas avaliações genéticas nacionais (ASReml, DMU, WOMBAT, BLUPF90) trabalham, em vez disso, com as equações do modelo misto de Henderson, cuja matriz de coeficientes é esparsa. O MLREML inclui esse segundo **motor esparso**. Sejam $\mathbf{T} = [\mathbf{X}, \mathbf{Z}]$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_K]$ e $\mathbf{G} = \text{blockdiag}(\mathbf{G}_1, \ldots, \mathbf{G}_K)$. A matriz de coeficientes das equações é

$$\mathbf{C} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{T} + \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}^{-1} \end{bmatrix}, \qquad \mathbf{C}\begin{bmatrix} \hat{\boldsymbol{\beta}} \\ \hat{\mathbf{u}} \end{bmatrix} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \tag{2.20}$$

Como $|\mathbf{V}||\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}| = |\mathbf{R}||\mathbf{G}||\mathbf{C}|$, a log-verossimilhança restrita (2.6) pode ser escrita sem $\mathbf{V}$:

$$\ell_R = -\frac{1}{2}[\log|\mathbf{R}| + \log|\mathbf{G}| + \log|\mathbf{C}| + \mathbf{y}'\mathbf{P}\mathbf{y} + (n-p)\log(2\pi)], \qquad \mathbf{y}'\mathbf{P}\mathbf{y} = \mathbf{y}'\mathbf{R}^{-1}\mathbf{y} - \left[\hat{\boldsymbol{\beta}}', \hat{\mathbf{u}}'\right]\mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \tag{2.21}$$

O valor de ℓ_R é o mesmo que o do motor denso. Com $\hat{\mathbf{e}} = \mathbf{y} - \mathbf{T}\left[\hat{\boldsymbol{\beta}}', \hat{\mathbf{u}}'\right]'$, o escore (2.8) torna-se

$$s_i = -\frac{1}{2}\left[tr(\mathbf{R}^{-1}\mathbf{R}_i) + tr(\mathbf{G}^{-1}\mathbf{G}_i) + tr(\mathbf{C}^{-1}\mathbf{C}_i) + \hat{\mathbf{e}}'\frac{\partial\mathbf{R}^{-1}}{\partial\theta_i}\hat{\mathbf{e}} + \hat{\mathbf{u}}'\frac{\partial\mathbf{G}^{-1}}{\partial\theta_i}\hat{\mathbf{u}}\right], \tag{2.22}$$

com $\mathbf{R}_i = \partial\mathbf{R}/\,\partial\theta_i$, $\mathbf{G}_i = \partial\mathbf{G}/\,\partial\theta_i$ e $\mathbf{C}_i = \partial\mathbf{C}/\,\partial\theta_i$, que é blockdiag$(\mathbf{0}, \partial\mathbf{G}^{-1}/\,\partial\theta_i)$ para um parâmetro de $\mathbf{G}$ e $\mathbf{T}'(\partial\mathbf{R}^{-1}/\,\partial\theta_i)\mathbf{T}$ para um parâmetro de $\mathbf{R}$.

Inversa esparsa. O traço $tr(\mathbf{C}^{-1}\mathbf{C}_i)$ precisa apenas dos elementos de $\mathbf{C}^{-1}$ nas posições em que $\mathbf{C}_i$ não é zero, todas as quais estão no padrão do fator de Cholesky de $\mathbf{C}$. Seja $\mathbf{C} = \mathbf{L}\mathbf{L}'$ após uma permutação de linhas e colunas que reduz o preenchimento (fill-in), e seja S_i o conjunto de linhas dos elementos não nulos da coluna i de $\mathbf{L}$ abaixo da diagonal. Os elementos Z_{ji} de $\mathbf{C}^{-1}$ no padrão de $\mathbf{L}$ são obtidos, coluna por coluna da última para a primeira, com as recursões de Takahashi et al. (1973) e Erisman e Tinney (1975), usadas no melhoramento animal por Misztal e Perez-Enciso (1993):

$$Z_{ji} = -\sum_{k \in S_i} \frac{L_{ki}}{L_{ii}} Z_{kj} \quad (j \in S_i), \qquad Z_{ii} = \frac{1}{L_{ii}^2} - \sum_{k \in S_i} \frac{L_{ki}}{L_{ii}} Z_{ki}. \qquad (2.23)$$

Do lado direito são necessários apenas elementos do padrão de $\mathbf{L}$, porque esse padrão é fechado: se k e j pertencem a S_i, o elemento (k, j) pertence ao padrão. O custo é proporcional a $\sum_i |S_i|^2$, a mesma ordem que o da própria fatoração, e a memória é igual à de $\mathbf{L}$. A diagonal do bloco dos efeitos aleatórios fornece as variâncias do erro de predição, e o bloco dos efeitos fixos fornece $Var(\widehat{\boldsymbol{\beta}})$, sem custo adicional.

Informação média. As variáveis de trabalho $\mathbf{w}_i = \mathbf{V}_i\mathbf{P}\mathbf{y}$ de (2.9) são calculadas sem $\mathbf{V}$, já que $\mathbf{P}\mathbf{y} = \mathbf{R}^{-1}\hat{\mathbf{e}}$ e $\mathbf{Z}'\mathbf{P}\mathbf{y} = \mathbf{G}^{-1}\hat{\mathbf{u}}$: $\mathbf{w}_i = \mathbf{Z}\mathbf{G}_i\mathbf{G}^{-1}\hat{\mathbf{u}}$ para um parâmetro de $\mathbf{G}$ e $\mathbf{w}_i = \mathbf{R}_i\mathbf{R}^{-1}\hat{\mathbf{e}}$ para um parâmetro de $\mathbf{R}$. Como $\mathbf{P} = \mathbf{R}^{-1} - \mathbf{R}^{-1}\mathbf{T}\mathbf{C}^{-1}\mathbf{T}'\mathbf{R}^{-1}$,

$$\mathcal{I}_{A,ij} = \frac{1}{2}\mathbf{w}_i'\mathbf{P}\mathbf{w}_j = \frac{1}{2}\left(\mathbf{w}_i'\mathbf{R}^{-1}\mathbf{w}_j - \mathbf{t}_i'\mathbf{C}^{-1}\mathbf{t}_j\right), \qquad \mathbf{t}_i = \mathbf{T}'\mathbf{R}^{-1}\mathbf{w}_i, \qquad (2.24)$$

o que requer uma resolução com o fator de $\mathbf{C}$ por parâmetro (Johnson e Thompson, 1995; Gilmour et al., 1995).

Estruturas. Para um termo $\mathbf{G}_k = \mathbf{K}_1 \otimes \boldsymbol{\Sigma} \otimes \mathbf{K}_2$, em que $\boldsymbol{\Sigma}$ é a única matriz com parâmetros (ou σ^2 para um termo sem estrutura de variância) e $\mathbf{K}_1, \mathbf{K}_2$ são identidades ou matrizes de parentesco, $\mathbf{G}_k^{-1} = \mathbf{K}_1^{-1} \otimes \boldsymbol{\Sigma}^{-1} \otimes \mathbf{K}_2^{-1}$; $\log|\mathbf{G}_k|$ decorre de $\log|\mathbf{A} \otimes \mathbf{B}| = b\log|\mathbf{A}| + a\log|\mathbf{B}|$ para matrizes de ordens a e b; e $\mathbf{G}_i\mathbf{G}^{-1}\hat{\mathbf{u}} = (\mathbf{I} \otimes \boldsymbol{\Sigma}_i\boldsymbol{\Sigma}^{-1} \otimes \mathbf{I})\hat{\mathbf{u}}$ envolve apenas a matriz pequena $\boldsymbol{\Sigma}$. As matrizes de parentesco são necessárias apenas por meio de suas inversas, de modo que $\mathbf{A}^{-1}$ construída com as regras de Henderson (Seção 2.9) pode ser fornecida diretamente. Um termo `str()` é $\boldsymbol{\Sigma} \otimes \mathbf{K}$ com $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$. A matriz residual é bloco-diagonal, com um bloco por registro: $\sigma_e^2\mathbf{I}$ (`units`), $diag\left(\sigma_{e(i)}^2\right)$ (`diag(f):units`) ou, em modelos multivariados, a submatriz de $\boldsymbol{\Sigma}_e$ dos caracteres observados em cada registro (`us(trait):units`).

Implementação. $\mathbf{C}$ é montada com um padrão fixo (a união de todas as posições que podem ser não nulas), de modo que o ordenamento que reduz o preenchimento e a fatoração simbólica do CHOLMOD (Chen et al., 2008), disponível no R por meio do pacote `Matrix`, são calculados uma única vez e apenas a fatoração numérica é repetida em cada iteração. As recursões (2.23) são programadas em C++ e usam uma intercalação ordenada de cada coluna com as colunas de suas linhas, sem buscas. Os pontos candidatos dos passos amortecidos precisam apenas da fatoração, porque $\log|\mathbf{C}| = 2\sum_i \log L_{ii}$. As médias preditas usam $Var(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = [\mathbf{L}, \mathbf{M}]\mathbf{C}^{-1}[\mathbf{L}, \mathbf{M}]'$, equivalente a (2.14), obtida resolvendo com o fator de $\mathbf{C}$.

O motor esparso suporta termos aleatórios construídos com componentes `id`, `vm`, `diag` e `us` (com fatores ou bases `pol()`, `leg()` ou `mm()`), termos `str()` e as estruturas residuais `units`, `diag(f):units` e `us(trait):units`. As estruturas com parâmetros de correlação (`cs`, `ar1`, `expc`, `sexp`) e `fa` são ajustadas com o motor denso. Com `engine = "auto"`, o padrão, o MLREML usa o motor esparso quando os dados têm mais de 5.000 registros e o modelo é suportado, e o motor denso caso contrário. Ambos os motores fornecem a mesma verossimilhança, estimativas, BLUPs e variâncias do erro de predição, a menos de erros de arredondamento (Seção 15.1). Como o motor esparso precisa de $\mathbf{G}^{-1}$, as matrizes de parentesco devem ser positivas definidas; uma matriz genômica singular deve ser corrigida (bending) ou combinada (Seção 13.7), ou ajustada com o motor denso.

2.13 Graus de liberdade de Kenward–Roger

A matriz de covariância $\boldsymbol{\Phi} = \left(\mathbf{X}'\widehat{\mathbf{V}}^{-1}\mathbf{X}\right)^{-1}$ de (2.11) trata os parâmetros de variância como conhecidos. Em experimentos pequenos ela subestima a variância de $\widehat{\boldsymbol{\beta}}$, e os testes F com $n - p$ graus de liberdade no denominador rejeitam uma hipótese verdadeira com frequência excessiva. Kenward e Roger (1997) propuseram uma matriz de covariância ajustada,

$$\boldsymbol{\Phi}_A = \boldsymbol{\Phi} + 2\boldsymbol{\Phi}\left\{\sum_{i,j} W_{ij}\left(\mathbf{Q}_{ij} - \mathbf{P}_i\boldsymbol{\Phi}\mathbf{P}_j - \frac{1}{4}\mathbf{R}_{ij}\right)\right\}\boldsymbol{\Phi}, \qquad (2.25)$$

com $\mathbf{P}_i = -\mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{Q}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{V}_j\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{R}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_{ij}\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{V}_{ij} = \partial^2\mathbf{V}/\,\partial\theta_i\,\partial\theta_j$, e $\mathbf{W}$ a inversa da matriz de informação esperada, cujos elementos são $\frac{1}{2}tr\left(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\right)$. Para uma hipótese $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$ com $q = rank(\mathbf{L})$ eles usam a estatística

$$F = \frac{\lambda}{q}\left(\mathbf{L}\widehat{\boldsymbol{\beta}}\right)'\left(\mathbf{L}\boldsymbol{\Phi}_A\mathbf{L}'\right)^{-1}\mathbf{L}\widehat{\boldsymbol{\beta}}, \qquad (2.26)$$

e escolhem a escala λ e os graus de liberdade do denominador m de modo que a média e a variância de F coincidam aproximadamente com as de uma distribuição $F(q, m)$. Com $\boldsymbol{\Theta} = \mathbf{L}'\left(\mathbf{L}\boldsymbol{\Phi}\mathbf{L}'\right)^{-1}\mathbf{L}$,

$$A_1 = \sum_{i,j} W_{ij}\; tr(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_i\boldsymbol{\Phi})tr\left(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_j\boldsymbol{\Phi}\right), \qquad A_2 = \sum_{i,j} W_{ij}\; tr\left(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_i\boldsymbol{\Phi}\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_j\boldsymbol{\Phi}\right),$$

$$B = \frac{A_1 + 6A_2}{2q}, \quad g = \frac{(q+1)A_1 - (q+4)A_2}{(q+2)A_2}, \quad c_1 = \frac{g}{3q + 2(1-g)}, \quad c_2 = \frac{q - g}{3q + 2(1-g)}, \quad c_3 = \frac{q + 2 - g}{3q + 2(1-g)},$$

$$E^* = \left(1 - \frac{A_2}{q}\right)^{-1}, \quad V^* = \frac{2}{q}\,\frac{1 + c_1 B}{(1 - c_2 B)^2(1 - c_3 B)}, \quad \rho = \frac{V^*}{2E^{*2}}, \quad m = 4 + \frac{q+2}{q\rho - 1}, \quad \lambda = \frac{m}{E^*(m-2)}. \qquad (2.27)$$

Em delineamentos balanceados nos quais o teste F é exato, o método o reproduz: $\lambda = 1$ e m é igual aos graus de liberdade clássicos (por exemplo, os do erro da parcela principal em um delineamento de parcelas subdivididas).

O MLREML calcula (2.25)–(2.27) para cada teste incremental e condicional de `wald()` (Seção 2.7); a matriz $\mathbf{L}$ de cada teste é dada pelas linhas do fator de Cholesky usado para calcular a estatística de Wald. Todos os parâmetros de variância livres são incluídos, exceto os que estão na fronteira do espaço paramétrico, que são tratados como conhecidos. As segundas derivadas $\mathbf{V}_{ij}$ são zero para as estruturas lineares em seus parâmetros (`id`, `vm`, `diag`, `us`, `str()`) e são incluídas analiticamente para as demais (`cs`, `ar1`, `expc`, `sexp`, cargas de `fa` e produtos de dois componentes com parâmetros, como `ar1(row):ar1(col)` com sua escala). O ajuste usa as matrizes de derivadas $n \times n$ e está disponível para os ajustes do motor denso armazenados com `store = TRUE`; quando a informação esperada exigiria mais de 2 GB de memória, a matriz de informação média é usada em seu lugar. Com o motor esparso, destinado a grandes conjuntos de dados nos quais o ajuste é desprezível, `wald()` usa $n - p$ graus de liberdade. Os resultados coincidem com os do pacote `pbkrtest` (Halekoh e Højsgaard, 2014) em cinco ou mais dígitos significativos (Seção 15.1).

3 Sintaxe do modelo

3.1 A função `mlreml()`

A chamada geral é

```
fit <- mlreml(fixed, random = NULL, residual = NULL, data,
              start = NULL, fix = NULL, maxit = 50, tol = 1e-6,
              pev = TRUE, trace = TRUE, store = TRUE,
              engine = c("auto", "dense", "sparse"))
```

Argumentos de `mlreml()`.

Argumento	Significado
`fixed`	Fórmula bilateral para a resposta e os efeitos fixos, com a sintaxe usual do R (`y ~ rep + gen`).
`random`	Fórmula unilateral com os termos aleatórios (`~ gen + rep:block`).
`residual`	Fórmula unilateral com um único termo para a estrutura residual. Padrão `~ units`.
`data`	Data frame. Linhas com resposta ou preditor ausente são removidas, exceto os valores ausentes dos fatores de um termo `str()` (Seção 2.11).
`start`	Vetor nomeado de valores iniciais dos parâmetros de variância.
`fix`	Nomes dos parâmetros mantidos fixos em seus valores iniciais.
`maxit`, `tol`	Número máximo de iterações e tolerância para a mudança em ℓ_R.
`pev`	Calcular as variâncias do erro de predição dos BLUPs.
`trace`	Mostrar o histórico de iterações.
`store`	Manter as matrizes necessárias para `predict()` e para o ajuste de Kenward–Roger de `wald()`; `FALSE` economiza memória.
`engine`	Motor de cálculo: `"dense"` (matriz de variância **V**), `"sparse"` (equações do modelo misto) ou `"auto"` (esparso com mais de 5.000 registros quando o modelo é suportado, denso caso contrário). Ver a Seção 2.12.

Os efeitos fixos são tratados como em `lm()`: fatores geram contrastes de tratamento, interações são escritas com `:` ou `*`, e colunas linearmente dependentes são removidas automaticamente (os efeitos “aliased” do ASReml).

3.2 Termos aleatórios

Cada termo aleatório é um produto de **componentes** separados por dois-pontos. Cada componente consiste em uma **estrutura** aplicada a uma **base**. A base define o nível ao qual cada observação pertence e a estrutura define a matriz de covariância (ou de correlação) dos efeitos dessa base.

3.2.1 Bases

Bases de um componente.

Base	Descrição
`f`	Um fator ou variável de classificação nos dados. Se for numérica, seus valores distintos, ordenados, são os níveis.
`pol(x, k)`	Polinômio bruto de ordem k com intercepto: colunas $1, x, \ldots, x^k$.

Base	Descrição
`leg(x, k)`	Polinômios de Legendre normalizados de ordem k de x reescalonado para $[-1,1]$, como nos modelos de regressão aleatória no melhoramento animal.
`mm(f1, f2, ...)`	Fator de múltipla pertinência: cada observação pertence aos níveis de `f1` e de `f2`. Usado para a capacidade geral de combinação dos dois pais.
`units`	As unidades de observação (um nível por registro; em modelos multivariados, um nível por registro original).

3.2.2 Estruturas

A tabela abaixo lista as estruturas disponíveis, a matriz que elas geram para um componente com L níveis e seu número de parâmetros. As estruturas de **correlação** não têm escala própria; as estruturas de **variância** têm.

Estruturas de variância.

Estrutura	Matriz $L \times L$	Parâmetros	Tipo
`id(f)` ou `f`	$\mathbf{I}_L$	0	correlação
`vm(f, K)`	matriz conhecida $\mathbf{K}$ (A, G, H, parentesco)	0	correlação
`cs(f)`	$(1-\rho)\mathbf{I} + \rho\mathbf{J}$	1	correlação
`ar1(f)`	$\rho^{\lvert x_a - x_b \rvert}$ (distâncias reais se os níveis forem números)	1	correlação
`expc(x)`	$\rho^{\lvert x_a - x_b \rvert}$ com x contínuo irregular	1	correlação
`sexp(x, y)`	$\rho^{d_{ab}}$ com d a distância euclidiana no plano	1	correlação
`diag(f)`	$\mathrm{diag}(\sigma_1^2, \ldots, \sigma_L^2)$	L	variância
`us(f)`	matriz não estruturada	$L(L+1)/2$	variância
`fa(f, k)`	$\mathbf{\Lambda\Lambda}' + \mathbf{\Psi}$, $\mathbf{\Lambda}$ de tamanho $L \times k$	$Lk + L$ (menos $k(k-1)/2$ zeros)	variância

Um termo formado apenas por componentes de correlação recebe um parâmetro de escala σ^2. Um termo pode ter no máximo um componente de variância, que então carrega a escala; por exemplo, em `us(env):vm(gen, G)` a matriz `us` é a covariância genética entre ambientes na escala dos dados e `vm` fornece as correlações entre genótipos. Dois componentes de variância no mesmo termo não são identificáveis (multiplicar um por c e dividir o outro por c não altera o modelo), e o MLREML os rejeita com uma mensagem de erro.

Em `fa(f, k)` com $k > 1$, as cargas acima da diagonal de $\mathbf{\Lambda}$ são fixadas em zero para eliminar a indeterminação rotacional. As cargas estimadas podem ser rotacionadas posteriormente para interpretação; `vc_matrix()` também retorna a solução rotacionada por componentes principais.

Termos correlacionados. O termo `str(f1, f2, ..., K = K, cov = "us")` não é um produto, mas uma soma de componentes (Seção 2.11): define um efeito aleatório para cada um dos fatores `f1`, `f2`, …, cujos níveis são as linhas da matriz `K`, com $Var(\mathbf{u}) = \mathbf{\Sigma} \otimes \mathbf{K}$, em que $\mathbf{\Sigma}$ é não estruturada (`cov = "us"`, o padrão) ou diagonal (`cov = "diag"`). Os fatores devem assumir valores em `rownames(K)`; um valor ausente (`NA`, `""` ou `"0"`) significa que o registro não recebe efeito desse fator, e o registro é mantido. Se `K` for omitido, usa-se a matriz identidade.

Os parâmetros são nomeados como os de `us()`: por exemplo, `str(animal, dam, K = A)!dam:animal` é a covariância direta–materna.

3.2.3 Exemplos de termos

Exemplos de termos aleatórios.

Termo	Modelo
`~ gen`	Genótipos independentes, uma variância.
`~ rep:block`	Blocos incompletos dentro de repetições.
`~ vm(animal, A)`	Modelo animal: $\mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A})$.
`~ vm(animal, Ai), Ai <- Ainverse(ped)`	O mesmo modelo com $\mathbf{A}$ dada por sua inversa esparsa.
`~ vm(animal, A) + animal`	Modelo de repetibilidade: valor genético mais ambiente permanente.
`~ str(animal, dam, K = A)`	Efeitos genéticos diretos e maternos com sua covariância, $\mathbf{\Sigma} \otimes \mathbf{A}$.
`~ str(animal, dam, K = A) + id(dam)`	O mesmo com um efeito materno de ambiente permanente.
`~ vm(gen, G)`	GBLUP.
`~ env:gen`	Interação G×E homogênea (simetria composta).
`~ diag(env):id(gen)`	Uma variância genética diferente em cada ambiente, sem covariância.
`~ us(env):vm(gen, G)`	MET genômico com covariância genética não estruturada entre ambientes.
`~ fa(env, 2):id(gen)`	MET com um modelo fator-analítico de dois fatores.
`~ us(trait):vm(animal, A)`	Matriz de covariância genética aditiva entre caracteres.
`~ us(leg(dim, 2)):vm(animal, A)`	Regressão aleatória genética de ordem 2.
`~ mm(p1, p2) + cross`	Dialelo: CGC dos pais e CEC do cruzamento.
`~ units`	Efeito *pepita* (nugget; erro independente adicional) em modelos espaciais.

As matrizes fornecidas a `vm()` e `str()` são procuradas no ambiente onde a fórmula fixa foi criada, geralmente o espaço de trabalho. Elas devem ter nomes de linhas contendo todos os níveis do fator; podem incluir níveis adicionais. Uma matriz de parentesco também pode ser fornecida por meio de sua inversa se tiver o atributo `INVERSE = TRUE`, como o resultado de `Ainverse()`: o motor esparso a usa diretamente e o motor denso a inverte.

3.3 Estruturas residuais

O argumento `residual` aceita um único termo com a mesma sintaxe. As estruturas mais comuns são:

Estruturas residuais.

Resíduo	$\mathbf{R}$
`~ units`	$\sigma_e^2 \mathbf{I}$ (padrão)
`~ diag(env):units`	Uma variância residual diferente em cada ambiente
`~ ar1(row):ar1(col)`	Modelo espacial separável (2.5)

Resíduo	R
`~ diag(env):ar1(row):ar1(col)`	Espacial por ambiente com variâncias próprias e correlações comuns
`~ ar1(time):id(animal)`	Medidas repetidas autocorrelacionadas dentro de cada indivíduo
`~ expc(time):id(animal)`	Igual, com tempos irregulares
`~ sexp(x, y)`	Modelo espacial isotrópico com coordenadas irregulares
`~ us(trait):units`	Covariância residual entre caracteres do mesmo registro (padrão multivariado)

Quando o resíduo é espacial, geralmente se adiciona um termo aleatório `units` para representar o erro de medição independente.

3.4 Modelos multivariados

Se o lado esquerdo de `fixed` for `cbind(y1, y2, ...)`, o MLREML empilha os dados em formato longo, cria o fator `trait` com um nível por caráter e a base `units` que identifica cada registro original. A fórmula dos efeitos fixos é escrita em termos de `trait`:

```
mlreml(cbind(weight, height) ~ trait + trait:sex,
       random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = mydata)
```

Registros com algum caráter ausente são mantidos para os caracteres observados, o que é uma das vantagens da análise REML multivariada.

3.5 Nomes dos parâmetros, valores iniciais e restrições

Cada parâmetro é denominado `term!component_level`; os parâmetros residuais começam com `R!`. Por exemplo, `us(env):vm(gen, G)!env_E2:E1` é a covariância genética entre os ambientes E2 e E1, e `R!diag(env):units!env_E1` é a variância residual de E1. Esses nomes são usados em `start` e `fix`. Para fixar a variância residual em 1 e estimar o restante:

```
fit0 <- mlreml(y ~ 1, random = ~ gen, data = d, trace = FALSE)
fit1 <- mlreml(y ~ 1, random = ~ gen, data = d,
               start = c("R!units!sigma2" = 1), fix = "R!units!sigma2")
```

Fixar todos os parâmetros permite avaliar a verossimilhança e os BLUPs com componentes conhecidos, por exemplo, componentes publicados ou componentes estimados em outro conjunto de dados.

3.6 O objeto ajustado

`mlreml()` retorna uma lista da classe `"mlreml"`. Seus principais elementos são:

Elementos do objeto `mlreml`.

Elemento	Conteúdo
`varcomp`	Tabela de parâmetros de variância: estimativa, erro padrão, razão z, código de fronteira.
`theta`, `vcov_theta`	Vetor de parâmetros e sua covariância aproximada ($\mathcal{I}_A^{-1}$).
`beta`,	Efeitos fixos e sua matriz de covariância (com o motor esparso e mais de

Elemento	Conteúdo
`vcov_beta`	1.000 efeitos fixos, uma matriz diagonal com as variâncias; `wald()` calcula a matriz completa quando precisa dela).
`blups`	Lista com um data frame por termo aleatório: nível, BLUP, PEV, erro padrão e confiabilidade (para um termo `str()`, também a coluna `effect`).
`fitted`, `residuals`	Valores ajustados ($\mathbf{X}\hat{\boldsymbol{\beta}} + \mathbf{Z}\hat{\mathbf{u}}$) e resíduos condicionais.
`loglik`, `AIC`, `BIC`	Verossimilhança restrita e critérios de informação.
`history`, `converged`	Histórico de iterações e indicador de convergência.
`data`	Dados usados (em formato longo se o modelo for multivariado).
`engine`	Motor usado, `"dense"` ou `"sparse"`.
`sparse`	Apenas motor esparso: número de equações (`N`), elementos não nulos do fator de Cholesky (`nnzL`) e, com `store = TRUE`, o próprio fator, usado por `predict()` e `wald()`.

Os métodos disponíveis são `summary()`, `print()`, `coef()` (fixos ou aleatórios), `fitted()`, `residuals()`, `vcov()`, `logLik()`, `AIC()`, `BIC()`, `plot()` e `predict()`. As funções `wald()`, `vpredict()`, `lrt()`, `vc_matrix()` e `variogram()` completam a inferência.

4 Exemplo 1: um ensaio de aveia em delineamento alfa-látice

4.1 Os dados

O conjunto de dados `john.alpha` do pacote `agridat` provém de John e Williams (1995). Contém 24 variedades de aveia avaliadas em um delineamento alfa com 3 repetições, cada uma dividida em 6 blocos incompletos de 4 parcelas. A resposta é a produtividade (t/ha). É um exemplo clássico de recuperação da informação interblocos: como cada bloco contém apenas 4 das 24 variedades, as comparações entre variedades que não compartilham um bloco dependem da variância de blocos.

```
library(agridat)
data(john.alpha)
d <- john.alpha
str(d)

## 'data.frame':    72 obs. of  7 variables:
##  $ plot : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ block: Factor w/ 6 levels "B1","B2","B3",..: 1 1 1 1 2 2 2 2 3 3 ...
##  $ gen  : Factor w/ 24 levels "G01","G02","G03",..: 11 4 5 22 21 10 20 2 23
14 ...
##  $ yield: num  4.12 4.45 5.88 4.58 4.65 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...

with(d, table(rep, block))[, 1:6]

##     block
## rep  B1 B2 B3 B4 B5 B6
##   R1  4  4  4  4  4  4
```

```
##   R2  4  4  4  4  4  4
##   R3  4  4  4  4  4  4
```

4.2 Variedades como efeitos fixos: médias ajustadas

Quando o objetivo é comparar variedades, elas são tratadas como fixas e os blocos como aleatórios. O modelo é

$$y_{ijk} = \mu + r_j + \tau_i + b_{k(j)} + e_{ijk}, \qquad b_{k(j)} \sim N(0, \sigma_b^2), \qquad e_{ijk} \sim N(0, \sigma_e^2).$$

```
fit_f <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = d, trace =
FALSE)
summary(fit_f)

## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 26   Iterations: 5 (converged)
## REML log-likelihood: -32.4492   AIC: 68.90   BIC: 72.56
##
## Variance parameters:
##                   estimate std.error z.ratio bound
## rep:block!sigma2  0.06195   0.03656    1.69     P
## R!units!sigma2    0.08522   0.02185    3.90     P
##
## Bound codes: P positive/free, B at boundary, F fixed.

wald(fit_f)

##          term df wald.inc    F.inc ddf.inc     p.inc wald.con    F.con
ddf.con
## 1 (Intercept)  1  4338.49 4338.495   11.52 3.307e-16  4338.49 4338.495
11.52
## 2         rep  2    18.42    9.212   11.52 4.078e-03    18.42    9.212
11.52
## 3         gen 23   125.30    5.363   35.50 4.495e-06   125.30    5.363
35.50
##       p.con   p.chisq
## 1 3.307e-16 0.000e+00
## 2 4.078e-03 9.981e-05
## 3 4.495e-06 4.588e-16
```

O teste de Wald condicional para gen testa a hipótese de que todas as variedades são iguais, ajustada para repetições. Com 72 parcelas e 26 colunas de efeitos fixos, os graus de liberdade residuais, 46, superestimam a informação sobre as variedades, parte da qual provém da comparação de blocos incompletos. O método de Kenward–Roger fornece 35,5 graus de liberdade no denominador para gen e 11,5 para rep, cuja informação provém principalmente dos 18 blocos incompletos, e reduz ligeiramente a estatística F de gen (5,363 em vez de 5,448). Os valores p do teste χ^2 (`p.chisq`) ignoram essa incerteza e são pequenos demais. A estatística condicional do intercepto é igual à sua estatística incremental, já que todos os termos contêm o intercepto.

As médias ajustadas das variedades e seus erros padrão das diferenças são obtidos com `predict()`:

```
pf <- predict(fit_f, classify = "gen")
head(pf$pvals[order(-pf$pvals$predicted.value), ], 8)
```

```
##    gen predicted.value std.error
## 1  G01           5.108    0.1955
## 5  G05           5.037    0.1955
## 15 G15           4.969    0.1955
## 19 G19           4.840    0.1955
## 21 G21           4.795    0.1955
## 14 G14           4.776    0.1955
## 13 G13           4.758    0.1955
## 12 G12           4.755    0.1955
```

```
pf$avsed
```

```
## [1] 0.2647
```

```
range(pf$sed[upper.tri(pf$sed)])
```

```
## [1] 0.2574 0.2699
```

O SED não é constante: duas variedades que compartilham blocos são comparadas com mais precisão do que duas que nunca ocorrem juntas. Essa é a informação que uma análise em blocos completos ignoraria. Uma diferença mínima significativa aproximada a 5 % é $t_{0.975,\mathrm{df}} \times \overline{\mathrm{SED}}$:

```
df_error <- nrow(d) - fit_f$p
qt(0.975, df_error) * pf$avsed
```

```
## [1] 0.5329
```

4.3 Variedades como efeitos aleatórios: BLUP e herdabilidade

Em um programa de melhoramento, as variedades são uma amostra de uma população e o objetivo é predizer seu valor genético. Com variedades aleatórias obtêm-se BLUPs, que encolhem as médias em direção à média geral em proporção à sua incerteza.

```
fit_r <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace =
FALSE)
summary(fit_r)
```

```
## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 3   Iterations: 4 (converged)
## REML log-likelihood: -46.5969   AIC: 99.19   BIC: 105.90
##
## Variance parameters:
##                  estimate std.error z.ratio bound
## gen!sigma2        0.14290   0.05208    2.74     P
## rep:block!sigma2  0.07022   0.03734    1.88     P
## R!units!sigma2    0.08162   0.02029    4.02     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
pr <- predict(fit_r, classify = "gen")
head(pr$pvals, 4)
```

```
##   gen predicted.value std.error
## 1 G01           4.981    0.1766
## 2 G02           4.484    0.1766
## 3 G03           3.695    0.1766
## 4 G04           4.486    0.1766
```

A herdabilidade no sentido amplo em base de média de variedades pode ser calculada com a fórmula clássica para $r = 3$ repetições ou com a herdabilidade generalizada de Cullis et al. (2006), que usa o SED médio dos BLUPs e é apropriada para delineamentos desbalanceados:

```
vpredict(fit_r, H2_classical ~ V1 / (V1 + V3 / 3))

##              estimate std.error
## H2_classical   0.8401   0.06254

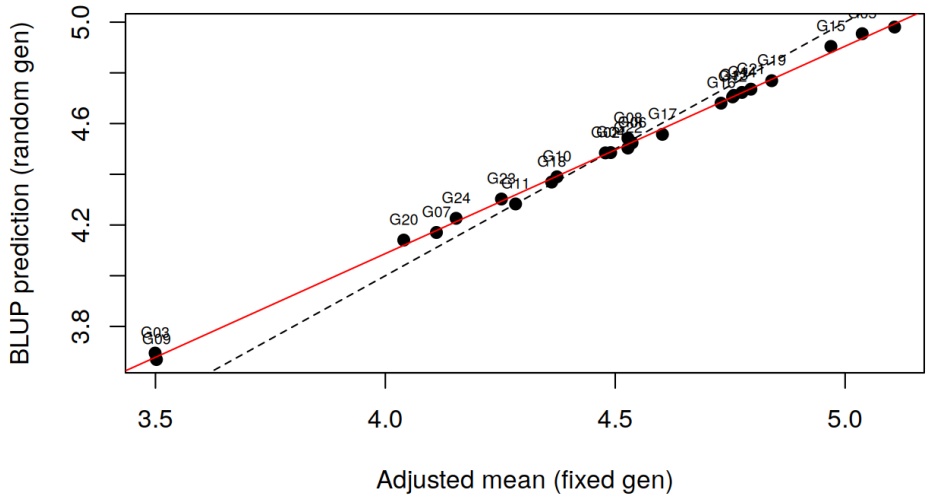
sg2 <- fit_r$theta[1]
H2_cullis <- 1 - pr$avsed^2 / (2 * sg2)
H2_cullis

## gen!sigma2
##     0.8092
```

4.4 Comparação de médias ajustadas e BLUPs

A figura seguinte compara as médias ajustadas (variedades fixas) com os BLUPs (variedades aleatórias). A inclinação menor que um é o encolhimento do BLUP.

```
plot(pf$pvals$predicted.value, pr$pvals$predicted.value, pch = 19,
     xlab = "Adjusted mean (fixed gen)", ylab = "BLUP prediction (random gen)")
abline(0, 1, lty = 2)
abline(lm(pr$pvals$predicted.value ~ pf$pvals$predicted.value), col = "red")
text(pf$pvals$predicted.value, pr$pvals$predicted.value, pf$pvals$gen, pos = 3,
cex = 0.6)
```



Médias ajustadas (variedades fixas) versus predições BLUP (variedades aleatórias) no ensaio de aveia.

A classificação das variedades é quase idêntica, mas os BLUPs estão mais próximos da média. Em delineamentos desbalanceados, o BLUP também pode alterar a classificação, porque variedades com menos informação são mais encolhidas.

4.5 Vale a pena o delineamento em blocos incompletos?

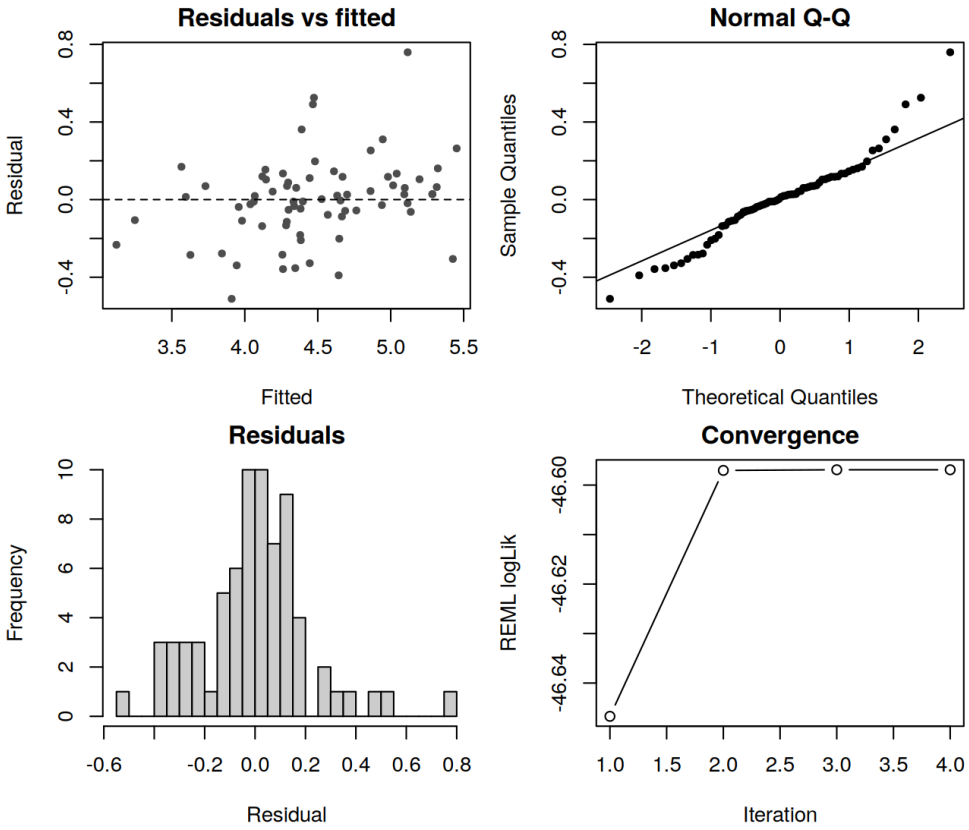
Para saber se os blocos incompletos capturam variação, o modelo com e sem `rep:block` é comparado por um teste da razão de verossimilhanças. Ambos os modelos têm os mesmos efeitos fixos, de modo que suas verossimilhanças REML são comparáveis, e como se está testando uma variância na fronteira, usa-se a mistura 50:50.

```
fit_r0 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
lrt(fit_r0, fit_r)

##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        8.606 0.001675 105.8 99.19 110.3 105.9
```

4.6 Diagnóstico

```
plot(fit_r)
```



Gráficos de diagnóstico do modelo com variedades aleatórias.

Os resíduos condicionais não mostram tendência com os valores ajustados nem desvios importantes da normalidade. O painel inferior direito mostra a convergência do algoritmo AI, que precisa de poucas iterações neste modelo simples.

4.7 Verificação com lme4

```
library(lme4)
l <- lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)
data.frame(MLREML = unname(fit_r$theta),
           lme4 = as.data.frame(VarCorr(l))$vcov,
           row.names = c("gen", "rep:block", "residual"))

##             MLREML     lme4
## gen        0.14290 0.14290
```

```
## rep:block 0.07022 0.07022
## residual  0.08162 0.08162
```

```
c(MLREML = fit_r$loglik, lme4 = as.numeric(logLik(l)))
```

```
## MLREML   lme4
##  -46.6  -46.6
```

Os dois programas concordam até a sexta casa decimal na verossimilhança.

5 Exemplo 2: análise espacial de um ensaio de trigo

5.1 Os dados

Os dados `gilmour.serpentine` provêm do artigo de Gilmour, Cullis e Verbyla (1997), que estabeleceu a análise espacial com estruturas autorregressivas separáveis como prática padrão nos ensaios agrícolas australianos. São 108 variedades de trigo em um delineamento de blocos completos casualizados com 3 repetições, em uma grade de 22 linhas por 15 colunas (330 parcelas). A resposta é a produtividade.

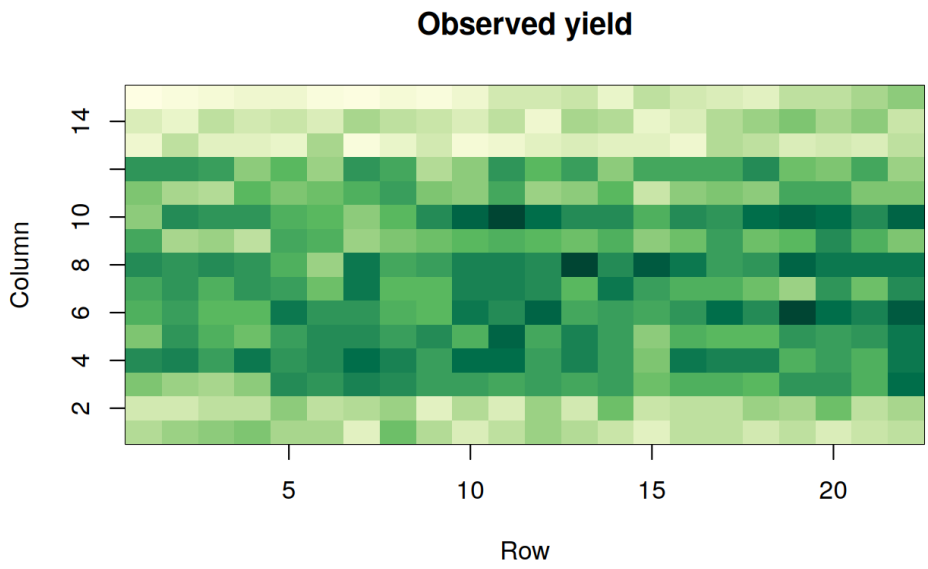
```
data(gilmour.serpentine)
d <- gilmour.serpentine
str(d)
```

```
## 'data.frame':    330 obs. of  5 variables:
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ gen  : Factor w/ 107 levels "(WWH*MM)*WR*",..: 4 10 15 17 21 32 33 34 72
74 ...
##  $ yield: int  483 526 557 564 498 510 344 600 466 370 ...
```

```
c(rows = length(unique(d$row)), columns = length(unique(d$col)))
```

```
##    rows columns
##      22      15
```

```
Ym <- tapply(d$yield, list(d$row, d$col), mean)
image(1:22, 1:15, Ym, col = hcl.colors(30, "YlGn", rev = TRUE),
      xlab = "Row", ylab = "Column", main = "Observed yield")
```



Produtividade observada na grade do ensaio (linhas × colunas). Cores mais escuras indicam maior produtividade.

A figura acima já sugere um gradiente de fertilidade ao longo das linhas e uma variação local que as repetições completas não conseguem absorver.

5.2 Modelo de blocos completos

```
m1 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
summary(m1)$varcomp
```

```
##                 estimate std.error z.ratio bound
## gen!sigma2          1934     947.3   2.042     P
## R!units!sigma2     13329    1263.6  10.548     P
```

5.3 Modelo espacial AR1 × AR1

Seguindo Gilmour et al. (1997), o resíduo é modelado como um processo autorregressivo separável em linhas e colunas mais um erro independente (*nugget*), que no MLREML é o termo aleatório `units`:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_g\mathbf{g} + \boldsymbol{\eta} + \boldsymbol{\xi}, \qquad \boldsymbol{\xi} \sim N\left(\mathbf{0}, \sigma^2_{\xi}\, \boldsymbol{\Sigma}_r(\rho_r) \otimes \boldsymbol{\Sigma}_c(\rho_c)\right), \qquad \boldsymbol{\eta} \sim N\left(\mathbf{0}, \sigma^2_{\eta}\mathbf{I}\right).$$

```
m2 <- mlreml(yield ~ rep, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m2)
```

```
## MLREML fit (AI-REML)
## Observations: 330   Fixed-effect columns: 3   Iterations: 16 (converged)
## REML log-likelihood: -1853.6433   AIC: 3717.29   BIC: 3736.24
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                  2.583e+03 4.764e+02    5.42     P
## units!sigma2                1.702e+03 2.356e+02    7.22     P
## R!ar1(row):ar1(col)!sigma2  3.954e+04 2.093e+04    1.89     P
## R!ar1(row):ar1(col)!row_cor 9.903e-01 5.309e-03  186.55     P
## R!ar1(row):ar1(col)!col_cor 8.202e-01 6.620e-02   12.39     P
```

```
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

Ambos os modelos têm os mesmos efeitos fixos, de modo que podem ser comparados com AIC e BIC:

```
data.frame(model = c("complete blocks", "AR1 x AR1 + nugget"),
           logLik = c(m1$loglik, m2$loglik), AIC = c(m1$AIC, m2$AIC), BIC =
c(m1$BIC, m2$BIC))
```

```
##                model logLik  AIC  BIC
## 1    complete blocks  -2043 4091 4099
## 2 AR1 x AR1 + nugget  -1854 3717 3736
```

A correlação entre linhas vizinhas é muito alta, indicando uma tendência suave ao longo das linhas, enquanto a correlação entre colunas é menor. A melhoria no AIC é grande.

5.4 Tendência global

Gilmour et al. (1997) recomendam adicionar uma tendência linear como efeito fixo quando o variograma mostra uma semivariância que continua aumentando. A covariável numérica `row` é incluída na parte fixa e testada com Wald:

```
m3 <- mlreml(yield ~ rep + row, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m3)$varcomp
```

```
##                               estimate std.error z.ratio bound
## gen!sigma2                   2.581e+03 4.761e+02   5.421     P
## units!sigma2                 1.697e+03 2.351e+02   7.218     P
## R!ar1(row):ar1(col)!sigma2  4.194e+04 2.267e+04   1.850     P
## R!ar1(row):ar1(col)!row_cor 9.905e-01 5.225e-03 189.573     P
## R!ar1(row):ar1(col)!col_cor 8.303e-01 6.455e-02  12.863     P
```

```
wald(m3)
```

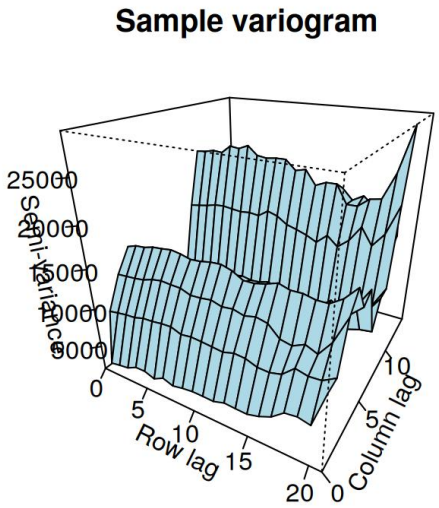
```
##          term df wald.inc   F.inc ddf.inc   p.inc wald.con   F.con ddf.con
p.con
## 1 (Intercept)  1  16.7395 20.0048   3.443 0.01547  16.7395 20.0048   3.443
0.01547
## 2         rep  2   1.8132  0.7871  11.472 0.47829   1.8166  0.7886  11.472
0.47766
## 3         row  1   0.6921  0.5961   9.759 0.45838   0.6921  0.5961   9.759
0.45838
##     p.chisq
## 1 4.288e-05
## 2 4.032e-01
## 3 4.055e-01
```

Neste ensaio, a tendência linear não é significativa (p = 0,46 com o método de Kenward–Roger, que fornece apenas 9,8 graus de liberdade no denominador para este teste): a autorregressão entre linhas, com uma correlação próxima de um, já representa o gradiente. Os modelos `m2` e `m3` têm efeitos fixos diferentes, de modo que suas verossimilhanças REML não são comparáveis e a decisão se baseia no teste de Wald. Mantemos o termo linear a seguir apenas para ilustrar seu uso; os resultados com `m2` são praticamente os mesmos.

5.5 Variograma amostral

O variograma dos resíduos espaciais é a ferramenta de diagnóstico mais útil. Para o modelo de blocos completos, ele é calculado a partir dos resíduos condicionais; um variograma que aumenta com a distância indica correlação espacial não modelada.

```
v1 <- variogram(m1, "row", "col")
```



Variograma amostral dos resíduos do modelo de blocos completos.

```
head(v1[order(v1$row.lag, v1$col.lag), ], 6)
```

```
##     row.lag col.lag gamma npairs
## 22        0       1  8657    308
## 44        0       2 12548    286
## 66        0       3 15256    264
## 88        0       4 10720    242
## 110       0       5 11402    220
## 132       0       6  9477    198
```

A semivariância do modelo de blocos completos aumenta com as defasagens de linha e de coluna e não se estabiliza dentro do ensaio; essa é a correlação espacial que o modelo AR1 × AR1 representa.

5.6 Ganho em precisão

A consequência prática de um modelo melhor é uma comparação mais precisa das variedades. Para comparar as precisões, ajusta-se o modelo com variedades fixas e calcula-se o SED médio:

```
m1f <- mlreml(yield ~ rep + gen, data = d, trace = FALSE)
m3f <- mlreml(yield ~ rep + row + gen, random = ~ units,
              residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
c(SED_blocks = predict(m1f, "gen")$avsed, SED_spatial = predict(m3f,
"gen")$avsed)
```

```
##  SED_blocks SED_spatial
##       93.66       38.81
```

A análise espacial reduz o erro das comparações, o que equivale a avaliar as variedades com mais repetições sem aumentar o custo do ensaio. A herdabilidade generalizada aumenta de forma correspondente:

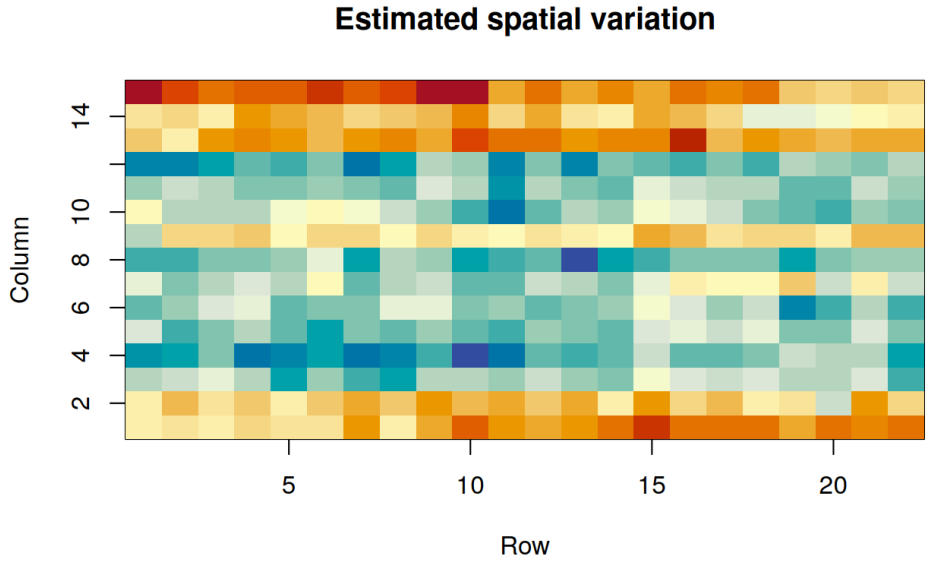
```
H2 <- function(fit) 1 - predict(fit, "gen")$avsed^2 / (2 * fit$theta[1])
c(blocks = H2(m1), spatial = H2(m3))

##  blocks.gen!sigma2 spatial.gen!sigma2
##             0.3080             0.7698
```

5.7 Tendência espacial estimada

O componente espacial predito pelo modelo é a diferença entre os dados e a parte fixa mais os efeitos genéticos. A figura seguinte o mostra na grade.

```
gb <- coef(m3, "random", "gen")
geff <- gb$blup[match(as.character(m3$data$gen), gb$level)]
spatial <- m3$y - m3$fixed_fitted - geff
Em <- tapply(spatial, list(m3$data$row, m3$data$col), mean)
image(1:22, 1:15, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Row", ylab =
"Column",
      main = "Estimated spatial variation")
```



Resíduos do modelo apenas com efeitos genéticos e fixos (estimativa do componente espacial).

5.8 Dados com espaçamento irregular

Quando as unidades não estão em uma grade regular (árvores em um povoamento, parcelas georreferenciadas por GPS, plantas faltantes), a estrutura `sexp(x, y)` define uma correlação exponencial isotrópica $\rho^{d_{ij}}$ com a distância euclidiana real. Como exemplo, usamos os dados de volume de 1.600 pinheiros-de-douglas (Johnson, 1998, em `agridat`), dos quais se toma uma amostra aleatória de 450 árvores para obter um padrão espacial irregular.

```
data(johnson.douglasfir)
set.seed(11)
```

```
df <- johnson.douglasfir[sample(nrow(johnson.douglasfir), 450), ]
df$lv <- log(df$volume + 1)
f_iid <- mlreml(lv ~ 1, data = df, trace = FALSE)
f_exp <- mlreml(lv ~ 1, random = ~ units, residual = ~ sexp(row, col), data =
df, trace = FALSE)
summary(f_exp)$varcomp
```

```
##                                     estimate std.error z.ratio bound
## units!sigma2                          5.8841    2.4044  2.4473     P
## R!sexp(row, col)!sigma2               0.8822    2.3803  0.3706     P
## R!sexp(row, col)!sexp(row,col)_cor    0.3581    0.6771  0.5289     P
```

```
lrt(f_iid, f_exp, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        1.031  0.5971 2141 2144 2145 2156
```

A correlação por unidade de distância e a melhoria na verossimilhança indicam que árvores próximas têm volumes semelhantes, devido à competição e ao microssítio. A correlação entre duas árvores a uma distância d é $\hat{\rho}^d$; por exemplo, a distâncias de 1, 3 e 5 posições:

```
rho <- f_exp$theta[grep("_cor$", names(f_exp$theta))]
unname(rho^c(1, 3, 5))
```

```
## [1] 0.358088 0.045917 0.005888
```

6 Exemplo 3: ensaios multiambientes e modelos fator-analíticos

6.1 Os dados

O conjunto de dados `besag.met` (Besag e Higdon, 1999; em `agridat`) contém a produtividade de 64 variedades de milho avaliadas em 6 locais (*condados*), cada um com 3 repetições divididas em 8 blocos incompletos de 8 parcelas: 1.152 parcelas no total. É um ensaio multiambiente (MET) típico de um programa de melhoramento.

```
data(besag.met)
d <- besag.met
str(d)
```

```
## 'data.frame':    1188 obs. of  7 variables:
##  $ county: Factor w/ 6 levels "C1","C2","C3",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ row   : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ col   : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep   : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ block : Factor w/ 8 levels "B1","B2","B3",..: 2 2 2 2 2 2 2 2 5 5 ...
##  $ yield : num  170 130 129 131 132 ...
##  $ gen   : Factor w/ 64 levels "G01","G02","G03",..: 42 58 26 10 34 50 2 18
45 61 ...
```

```
round(tapply(d$yield, d$county, mean), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

```
round(tapply(d$yield, d$county, sd), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

As médias e os desvios padrão diferem muito entre locais, antecipando variâncias genéticas e residuais heterogêneas.

6.2 A sequência de modelos

A análise MET de referência (Smith, Cullis e Thompson, 2001; 2005) modela os efeitos genéticos em cada ambiente, $\mathbf{u} = (\mathbf{u}_1', \ldots, \mathbf{u}_E')'$, com

$$\mathrm{Var}(\mathbf{u}) = \boldsymbol{\Sigma}_E \otimes \mathbf{I}_N,$$

e compara estruturas para a matriz $E \times E$ de covariâncias genéticas entre ambientes $\boldsymbol{\Sigma}_E$. Todas as estruturas compartilham os efeitos fixos (médias dos locais), o delineamento (repetições e blocos dentro de local) e uma variância residual específica de cada local.

Estruturas para a covariância genética entre ambientes.

Modelo	$\boldsymbol{\Sigma}_E$	Parâmetros genéticos
Diagonal	$\mathrm{diag}(\sigma^2_{g1}, \ldots, \sigma^2_{gE})$	E
FA(1)	$\boldsymbol{\lambda}\boldsymbol{\lambda}' + \boldsymbol{\Psi}$	$2E$
FA(2)	$\boldsymbol{\Lambda}\boldsymbol{\Lambda}' + \boldsymbol{\Psi}$	$3E - 1$
Não estruturada	completa	$E(E+1)/2$

O modelo diagonal assume ausência de correlação genética entre ambientes, o que na prática nunca é verdade. O modelo não estruturado é o mais geral, mas com muitos ambientes tem parâmetros demais e muitas vezes é difícil de estimar. Os modelos fator-analíticos são o meio-termo: com poucos fatores, descrevem a maior parte da covariância genética e tornam interpretável a interação genótipo × ambiente.

```
f_diag <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
                 residual = ~ diag(county):units, data = d, trace = FALSE)
f_fa1 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
f_fa2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 2):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 200)
f_us <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
us(county):id(gen),
               residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
cmp <- data.frame(model = c("diag", "FA(1)", "FA(2)", "US"),
                  parameters = sapply(list(f_diag, f_fa1, f_fa2, f_us),
function(f) sum(f$free)),
                  logLik = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
"loglik"),
                  AIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "AIC"),
                  BIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "BIC"),
                  converged = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
```

```
"converged"))
cmp
```

```
##   model parameters logLik  AIC  BIC converged
## 1  diag         14  -4849 9726 9796      TRUE
## 2 FA(1)         20  -4814 9668 9769      TRUE
## 3 FA(2)         25  -4812 9675 9801      TRUE
## 4    US         29  -4812 9681 9827      TRUE
```

O modelo diagonal é claramente o pior: ignorar a correlação genética entre locais custa 34,9 unidades de log-verossimilhança em relação ao FA(1), com apenas 6 parâmetros adicionais. Segundo o AIC, o melhor modelo é o FA(1). A verossimilhança aumenta do FA(1) para o FA(2) e para o modelo não estruturado, como deve acontecer porque os modelos são aninhados, mas os aumentos são pequenos para o número de parâmetros adicionados. A verossimilhança é muito plana na direção do segundo fator, situação comum na análise MET que também ocorre no ASReml. Várias variâncias específicas do modelo FA(2) estão na fronteira; graças ao tratamento da fronteira por conjunto ativo (Seção 2.4.1), elas não travam o passo AI conjunto e o modelo converge. A log-verossimilhança do FA(2) está entre as do FA(1) e do modelo não estruturado e não altera a escolha do modelo.

O LRT entre o FA(1) e o modelo não estruturado (ambos com os mesmos efeitos fixos) é:

```
lrt(f_fa1, f_us, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  9        4.712  0.8587 9668 9681 9769 9827
```

6.3 Interpretação do modelo FA(1)

`vc_matrix()` retorna a matriz de covariância genética estimada, sua matriz de correlação, as cargas, as variâncias específicas e a porcentagem da variância genética de cada ambiente explicada pelos fatores.

```
vfa <- vc_matrix(f_fa1, "fa(county, 1):id(gen)")
round(vfa$cor, 2)
```

```
##      C1   C2   C3   C4   C5   C6
## C1 1.00 0.86 0.68 0.86 0.86 0.40
## C2 0.86 1.00 0.79 1.00 1.00 0.46
## C3 0.68 0.79 1.00 0.79 0.79 0.36
## C4 0.86 1.00 0.79 1.00 1.00 0.46
## C5 0.86 1.00 0.79 1.00 1.00 0.46
## C6 0.40 0.46 0.36 0.46 0.46 1.00
```

```
data.frame(loading = round(vfa$loadings[, 1], 2), psi = signif(vfa$psi, 3),
           pct_explained = round(vfa$pct_variance, 1))
```

```
##    loading      psi pct_explained
## C1    6.49 1.46e+01          74.3
## C2    5.15 8.44e-07         100.0
## C3    7.57 3.47e+01          62.3
## C4    5.34 7.30e-07         100.0
## C5    8.90 6.70e-07         100.0
## C6    2.35 2.05e+01          21.3
```

```
summary(f_fa1)$varcomp[grepl("fa1|var_", rownames(summary(f_fa1)$varcomp)), ]
```

```
##                                          estimate std.error   z.ratio bound
## fa(county, 1):id(gen)!county_fa1_C1 6.491e+00     1.366 4.752e+00     P
## fa(county, 1):id(gen)!county_fa1_C2 5.149e+00     1.319 3.904e+00     P
## fa(county, 1):id(gen)!county_fa1_C3 7.569e+00     1.567 4.830e+00     P
## fa(county, 1):id(gen)!county_fa1_C4 5.345e+00     1.419 3.765e+00     P
## fa(county, 1):id(gen)!county_fa1_C5 8.898e+00     1.392 6.391e+00     P
## fa(county, 1):id(gen)!county_fa1_C6 2.355e+00     1.710 1.377e+00     P
## fa(county, 1):id(gen)!county_var_C1 1.458e+01    16.469 8.855e-01     P
## fa(county, 1):id(gen)!county_var_C2 8.438e-07    16.874 5.001e-08     B
## fa(county, 1):id(gen)!county_var_C3 3.472e+01    21.181 1.639e+00     P
## fa(county, 1):id(gen)!county_var_C4 7.300e-07    19.599 3.725e-08     B
## fa(county, 1):id(gen)!county_var_C5 6.700e-07    16.851 3.976e-08     B
## fa(county, 1):id(gen)!county_var_C6 2.053e+01    29.724 6.907e-01     P
```

Várias variâncias específicas ψ estão na fronteira (código `B`). Nesses locais, o fator comum explica toda a variância genética, e a correlação genética estimada entre eles é um. Esse resultado é frequente em ensaios com poucas repetições e não indica erro de ajuste; indica que os dados não conseguem distinguir a variância específica de zero. Os erros padrão de parâmetros na fronteira não são válidos.

```
vus <- vc_matrix(f_us, "us(county):id(gen)")
op <- par(mfrow = c(1, 2), mar = c(3, 3, 2, 1))
for (M in list(US = vus$cor, FA1 = vfa$cor)) {
  image(1:6, 1:6, M[, 6:1], zlim = c(-1, 1), col = hcl.colors(21, "RdBu"),
        axes = FALSE, xlab = "", ylab = "")
  axis(1, 1:6, rownames(M), cex.axis = 0.7); axis(2, 1:6, rev(rownames(M)),
cex.axis = 0.7, las = 1)
  text(rep(1:6, 6), rep(6:1, each = 6), sprintf("%.2f", M), cex = 0.6)
}
```



Correlações genéticas entre locais estimadas com o modelo não estruturado (esquerda) e com o FA(1) (direita).

```
par(op)
```

6.4 Predições por ambiente e desempenho geral

As predições de cada variedade em cada local combinam a média fixa do local com o BLUP da variedade nesse local. A classificação `county:gen` inclui o termo fator-analítico porque ambos os seus fatores estão na classificação:

```
pr <- predict(f_fa1, classify = "county:gen", sed = FALSE)
P <- with(pr$pvals, tapply(predicted.value, list(gen, county), mean))
round(head(P[order(-rowMeans(P)), ], 10), 1)
```

```
##        C1   C2    C3    C4    C5   C6
## G05 160.6 64.8 100.0 153.7 130.6 92.7
## G54 159.5 64.8 101.6 153.7 130.5 91.3
## G45 157.4 63.1 100.1 151.9 127.5 89.8
## G55 158.7 62.6 101.1 151.4 126.8 88.8
## G61 155.7 61.4 101.0 150.1 124.6 91.6
## G32 157.1 61.8 100.2 150.6 125.4 88.5
## G51 157.3 62.0  96.1 150.8 125.7 91.2
## G30 155.4 60.9  98.0 149.7 123.8 89.1
## G22 155.0 61.2  94.0 149.9 124.2 91.0
## G52 155.8 60.6  96.1 149.4 123.3 89.8
```

Uma medida natural do desempenho geral em um modelo FA(1) com cargas positivas é o escore fatorial de cada variedade multiplicado pela carga média (Smith e Cullis, 2018). Como $\mathbf{u}_i = \boldsymbol{\lambda} f_i + \boldsymbol{\delta}_i$, o escore pode ser estimado por mínimos quadrados a partir dos BLUPs de cada variedade:

```
U <- matrix(coef(f_fa1, "random", "fa(county, 1):id(gen)")$blup, ncol = 6,
            dimnames = list(levels(d$gen), levels(d$county)))
lam <- vfa$loadings[, 1]
score <- drop(U %*% lam / sum(lam^2))
OP <- mean(lam) * score
head(sort(OP, decreasing = TRUE), 10)
```

```
##    G05    G54    G55    G45    G32    G61    G51    G30    G09    G52
## 10.149 10.131  8.381  8.330  7.407  7.220  6.954  6.159  5.860  5.709
```

```
top <- names(sort(OP, decreasing = TRUE))[c(1, 2, 63, 64)]
plot(range(lam), range(U[top, ]), type = "n", xlab = "Loading of the location",
     ylab = "BLUP in the location")
for (k in seq_along(top)) {
  points(lam, U[top[k], ], pch = 19, col = k)
  abline(0, score[top[k]], col = k)
}
legend("topleft", top, col = seq_along(top), pch = 19, bty = "n", cex = 0.8)
```

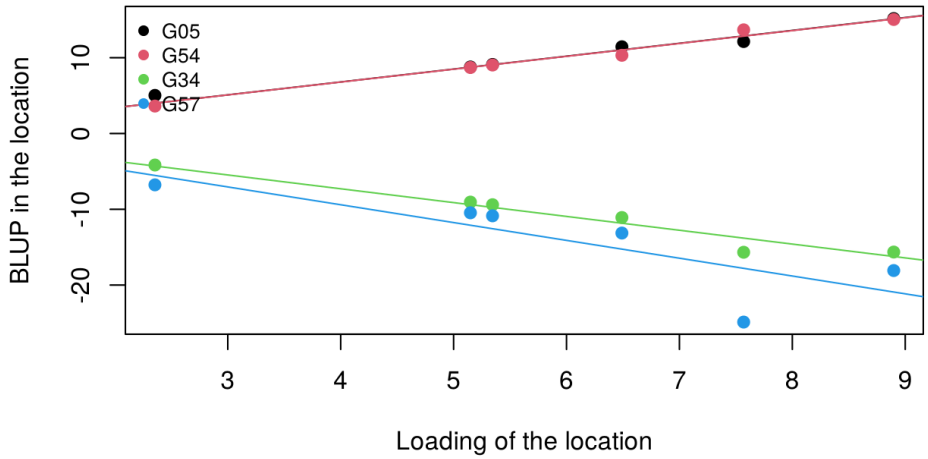


Gráfico de regressão latente: BLUPs de quatro variedades em cada local versus a carga do local. As retas têm inclinação igual ao escore de cada variedade.

Variedades com grande inclinação positiva são superiores em locais com cargas altas; a dispersão dos pontos em torno da reta é a parte da interação não explicada pelo fator comum.

6.5 Recomendações para a análise MET

1. Comece com variâncias residuais e genéticas heterogêneas (`diag`) para verificar se o modelo base se ajusta.
2. Ajuste o FA(1) e aumente o número de fatores enquanto o AIC ou o LRT o justificarem e a porcentagem de variância genética explicada for baixa em alguns ambientes.
3. Use `us` apenas com poucos ambientes; com mais de 8–10 ambientes, ele frequentemente não converge ou converge para matrizes singulares.
4. Verifique os parâmetros na fronteira e aumente `maxit` quando o algoritmo não convergir; um modelo FA que não convergiu na verossimilhança ainda pode ter predições estáveis, o que pode ser verificado comparando os BLUPs entre iterações.

7 Exemplo 4: predição genômica (GBLUP) com 599 linhagens de trigo

7.1 Os dados

O conjunto de dados `wheat` do pacote `BGLR` (Crossa et al., 2010; Pérez e de los Campos, 2014) contém 599 linhagens de trigo do CIMMYT genotipadas com 1.279 marcadores DArT (codificados como 0/1), sua matriz de parentesco por pedigree e a produtividade de grãos avaliada em quatro ambientes (E1 a E4). É um dos conjuntos de dados mais usados para comparar métodos de seleção genômica.

```
library(BGLR)
data(wheat)
ids <- paste0("L", 1:599)
M <- wheat.X; rownames(M) <- ids
A <- wheat.A; dimnames(A) <- list(ids, ids)
Y <- wheat.Y; rownames(Y) <- ids
dim(M); table(M)
```

```
## [1]  599 1279

## M
##      0      1
## 336588 429533

round(cor(Y), 2)

##       1     2     4     5
## 1  1.00 -0.02 -0.19 -0.12
## 2 -0.02  1.00  0.66  0.41
## 4 -0.19  0.66  1.00  0.39
## 5 -0.12  0.41  0.39  1.00
```

7.2 Controle de qualidade e matriz genômica

Os marcadores DArT são dominantes e codificados como 0/1. Para construir **G**, nós os recodificamos como 0/2 (as linhagens são endogâmicas, de modo que todo loco é homozigoto) e aplicamos o controle de qualidade:

```
qc <- snp_qc(2 * M, maf = 0.05, callrate = 0.9)
qc$report

##   markers_in removed_callrate removed_maf markers_out individuals_in
individuals_out
## 1       1279                0          96        1183            599
599

G <- Gmatrix(qc$M, method = "VanRaden")
summary(diag(G))

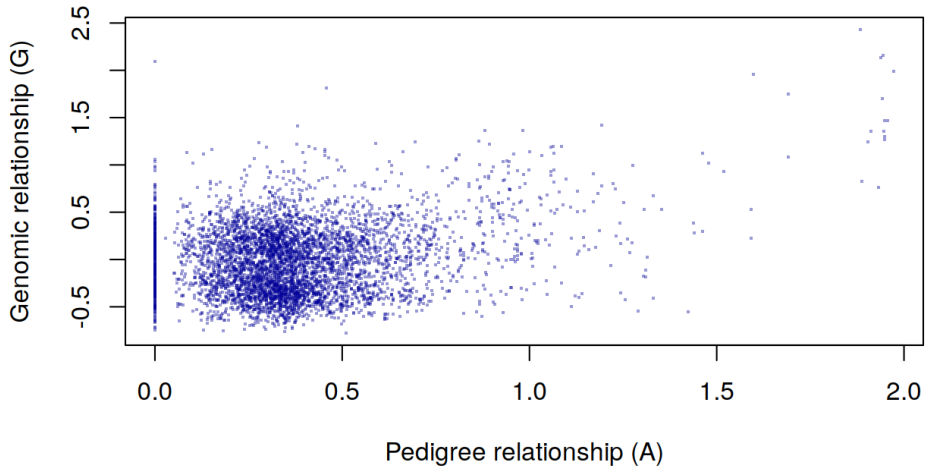
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.34    1.77    2.00    2.00    2.23    2.88

summary(diag(A))

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.56    1.98    1.99    1.98    2.00    2.00
```

A diagonal de **A** é próxima de 2, como esperado para linhagens endogâmicas ($1 + F$ com $F \approx 1$). A diagonal de **G** também tem média 2, mas varia muito mais entre as linhagens (de 1,3 a 2,9): ela mede a endogamia realizada em relação à população genotipada, enquanto o pedigree fornece apenas seu valor esperado. As diferenças de escala entre as matrizes não afetam as predições, porque são absorvidas pela variância genética, mas afetam a interpretação de σ^2_g. A relação entre as duas matrizes é mostrada abaixo:

```
up <- upper.tri(A)
set.seed(1); s <- sample(which(up), 5000)
plot(A[s], G[s], pch = ".", cex = 2, col = rgb(0, 0, 0.6, 0.4),
     xlab = "Pedigree relationship (A)", ylab = "Genomic relationship (G)")
```



Parentescos por pedigree versus parentescos genômicos (elementos fora da diagonal).

```
cor(A[up], G[up])
```

```
## [1] 0.2469
```

A correlação moderada reflete que o pedigree prediz o parentesco esperado, enquanto os marcadores medem o parentesco realizado, que varia em torno do esperado por causa da segregação mendeliana.

7.3 GBLUP em um ambiente

O modelo GBLUP para o ambiente E1 é $\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{e}$, $\mathbf{g} \sim N\left(\mathbf{0}, \sigma_g^2 \mathbf{G}\right)$. Ele é ajustado com o motor geral e com a função `gblup()`, que usa a decomposição espectral de $\mathbf{G}$ e reduz o REML a uma otimização unidimensional.

```
G <- G + diag(1e-4, nrow(G))      # small numerical regularisation
d1 <- data.frame(gen = ids, y = Y[, 1])
t_general <- system.time(f1 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1,
trace = FALSE))
y1 <- setNames(Y[, 1], ids)
t_spectral <- system.time(g1 <- gblup(y1, G))
summary(f1)$varcomp
```

```
##                    estimate std.error z.ratio bound
## vm(gen, G)!sigma2   0.2942   0.05462   5.387     P
## R!units!sigma2      0.5551   0.04607  12.049     P
```

```
g1$varcomp
```

```
## sigma2_g sigma2_e       h2
##   0.2942   0.5551   0.3464
```

```
c(general = t_general[3], spectral = t_spectral[3])
```

```
##  general.elapsed spectral.elapsed
##            0.209            0.061
```

```
c(logLik_general = f1$loglik, logLik_spectral = g1$loglik)
```

```
##  logLik_general logLik_spectral
##          -794.5          -794.5

cor(f1$blups[[1]]$blup, g1$gebv[f1$blups[[1]]$level])

## [1] 1
```

Ambas as implementações fornecem a mesma verossimilhança e os mesmos GEBVs. A herdabilidade genômica estimada é 0,35. A confiabilidade dos GEBVs das linhagens fenotipadas é obtida diretamente:

```
summary(f1$blups[[1]]$reliability)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.603   0.717   0.754   0.763   0.809   0.964
```

7.4 Predição de linhagens não fenotipadas

O objetivo da seleção genômica é predizer candidatos sem fenótipos. Se os fenótipos de 120 linhagens forem removidos, `vm()` ainda as prediz porque elas estão em **G**; basta retirar seus registros dos dados:

```
set.seed(2024)
test <- sample(ids, 120)
f_tr <- mlreml(y ~ 1, random = ~ vm(gen, G), data = subset(d1, !gen %in% test),
trace = FALSE)
b <- f_tr$blups[[1]]
pred <- b$blup[match(test, b$level)]
c(correlation = cor(pred, Y[test, 1]),
  mean_reliability_test = mean(b$reliability[match(test, b$level)]))

##           correlation mean_reliability_test
##                0.5457                0.6374
```

7.5 Validação cruzada: pedigree, marcadores e ambos

A função `cv_gblup()` repete partições aleatórias de *k* grupos (folds). Comparamos a capacidade preditiva do pedigree (**A**), dos marcadores (**G**) e de um modelo com ambas as matrizes, escrito no MLREML como dois termos aleatórios sobre o mesmo fator.

```
Ar <- A + diag(1e-4, nrow(A))
cvA <- cv_gblup(y1, Ar, folds = 5, reps = 4, seed = 3)
cvG <- cv_gblup(y1, G, folds = 5, reps = 4, seed = 3)
data.frame(matrix = c("A (pedigree)", "G (markers)"),
           mean_r = c(mean(cvA$cor), mean(cvG$cor)),
           sd_r = c(sd(cvA$cor), sd(cvG$cor)))

##         matrix mean_r   sd_r
## 1 A (pedigree) 0.4157 0.0615
## 2  G (markers) 0.4909 0.0551

f_AG <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = d1, trace =
FALSE)
summary(f_AG)$varcomp

##                     estimate std.error z.ratio bound
## vm(gen, Ar)!sigma2   0.1127   0.04147   2.717     P
```

```
## vm(gen, G)!sigma2    0.2351   0.05213   4.510     P
## R!units!sigma2       0.4519   0.05102   8.856     P
```

Com a mesma partição para o modelo de dois kernels:

```
set.seed(3); fold <- sample(rep(1:5, length.out = 599))
r_AG <- sapply(1:5, function(k) {
  tr <- d1[fold != k, ]
  f <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = tr, trace =
FALSE, pev = FALSE)
  te <- ids[fold == k]
  p <- f$blups[[1]]$blup[match(te, f$blups[[1]]$level)] +
f$blups[[2]]$blup[match(te, f$blups[[2]]$level)]
  cor(p, Y[te, 1])
})
mean(r_AG)
```

```
## [1] 0.5161
```

7.6 GBLUP multiambiente

Com os quatro ambientes, ajustamos um modelo MET genômico no qual o efeito genético de cada linhagem em cada ambiente tem covariância $\boldsymbol{\Sigma}_E \otimes \mathbf{G}$ (Burgueño et al., 2012), com variâncias residuais específicas de cada ambiente:

```
dl <- data.frame(gen = rep(ids, 4), env = factor(rep(paste0("E", 1:4), each =
599)), y = as.vector(Y))
t_met <- system.time(
  fm <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
               data = dl, trace = FALSE, maxit = 100))
t_met[3]
```

```
## elapsed
##   65.96
```

```
summary(fm)
```

```
## MLREML fit (AI-REML)
## Observations: 2396   Fixed-effect columns: 4   Iterations: 15 (converged)
## REML log-likelihood: -3054.2139   AIC: 6136.43   BIC: 6217.35
##
## Variance parameters:
##                                estimate std.error z.ratio bound
## us(env):vm(gen, G)!env_E1:E1  0.29972   0.05433    5.52     P
## us(env):vm(gen, G)!env_E2:E1 -0.07022   0.04270   -1.64     P
## us(env):vm(gen, G)!env_E2:E2  0.57991   0.07460    7.77     P
## us(env):vm(gen, G)!env_E3:E1 -0.10855   0.04109   -2.64     P
## us(env):vm(gen, G)!env_E3:E2  0.52893   0.06206    8.52     P
## us(env):vm(gen, G)!env_E3:E3  0.49306   0.06739    7.32     P
## us(env):vm(gen, G)!env_E4:E1 -0.12528   0.03693   -3.39     P
## us(env):vm(gen, G)!env_E4:E2  0.34978   0.04934    7.09     P
## us(env):vm(gen, G)!env_E4:E3  0.35240   0.04828    7.30     P
## us(env):vm(gen, G)!env_E4:E4  0.34639   0.05632    6.15     P
## R!diag(env):units!env_E1      0.55463   0.04568   12.14     P
## R!diag(env):units!env_E2      0.41498   0.03492   11.88     P
## R!diag(env):units!env_E3      0.50242   0.03863   13.01     P
## R!diag(env):units!env_E4      0.56067   0.04425   12.67     P
```

```
##
## Bound codes: P positive/free, B at boundary, F fixed.

round(vc_matrix(fm, "us(env):vm(gen, G)")$cor, 2)

##       E1    E2    E3    E4
## E1  1.00 -0.17 -0.28 -0.39
## E2 -0.17  1.00  0.99  0.78
## E3 -0.28  0.99  1.00  0.85
## E4 -0.39  0.78  0.85  1.00
```

As correlações genéticas entre ambientes são mais extremas do que as fenotípicas: E2, E3 e E4 estão fortemente correlacionados entre si, enquanto E1 tem correlações genéticas negativas com os demais. Essa é a estrutura que o modelo MET explora para predizer uma linhagem em um ambiente a partir de seus dados nos outros ambientes (o esquema CV2 de Burgueño et al., 2012).

```
set.seed(5)
hidden <- sample(ids, 120)
dtr <- dl[!(dl$env == "E1" & dl$gen %in% hidden), ]
fm_tr <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
                data = dtr, trace = FALSE, maxit = 100, pev = FALSE)
b <- fm_tr$blups[[1]]
p_met <- b$blup[match(paste0("E1:", hidden), b$level)]
y_e1 <- y1; y_e1[hidden] <- NA
p_uni <- gblup(y_e1, G)$gebv[hidden]
c(single_environment = cor(p_uni, Y[hidden, 1]), multi_environment = cor(p_met,
Y[hidden, 1]))

## single_environment  multi_environment
##             0.3651             0.3438
```

Neste caso, o modelo multiambiente não supera o modelo de um ambiente (0,344 versus 0,365). No esquema CV2, o modelo MET usa os registros das mesmas linhagens nos outros ambientes, mas E1 tem correlações genéticas fracas e negativas com E2–E4, de modo que esses registros trazem pouca informação sobre E1 e o custo de estimar mais parâmetros não é compensado. Quando os ambientes estão positivamente correlacionados, o ganho do modelo MET no CV2 costuma ser grande (Burgueño et al., 2012); o resultado ilustra que o benefício depende da estrutura de $\boldsymbol{\Sigma}_E$.

7.7 Notas práticas

- `gblup()` é conveniente para um único efeito genético com resíduo homogêneo, e é a base de `cv_gblup()`; com 599 linhagens, leva uma fração de segundo. Com vários milhares de indivíduos, seu custo é dominado pela decomposição espectral (Capítulo 15).
- `mlreml()` com `vm()` é necessário para modelos com mais termos, resíduos heterogêneos, vários ambientes ou vários caracteres.
- Uma matriz **G** com mais indivíduos do que marcadores é singular; o MLREML não a inverte, mas é aconselhável adicionar uma pequena constante à diagonal para estabilidade numérica, como feito acima.

8 Exemplo 5: modelo animal com pedigree em gado leiteiro

8.1 Os dados

O pacote `pedigreemm` (Vazquez et al., 2010) inclui `milk`, com 3.397 registros de lactação de 1.359 vacas Holandesas em 57 rebanhos, e `pedCowsR`, o pedigree dessas vacas e de seus ancestrais (6.547 animais). Cada registro contém a produção de leite, gordura e proteína em 305 dias e o número da lactação (1 a 5).

```
library(pedigreemm)
data(milk)
milk$animal <- as.character(milk$id)
milk$milk_t <- milk$milk / 1000    # tonnes
milk$fat_h <- milk$fat / 100       # hundreds of kg
milk$lact <- factor(milk$lact)
table(table(milk$animal))
```

```
##
##   1   2   3   4   5
## 337 405 314 207  96
```

8.2 Preparação do pedigree

`pedigree_prepare()` recodifica pais desconhecidos, adiciona como fundadores os pais que não aparecem como indivíduos, detecta erros e ordena o pedigree de modo que os pais precedam sua progênie.

```
p <- pedCowsR
ped <- data.frame(id = p@label, sire = p@label[p@sire], dam = p@label[p@dam])
pp <- pedigree_prepare(ped)
head(pp)
```

```
##   id sire  dam sire_i dam_i gen
## 1  1 <NA> <NA>      0     0   0
## 2  2 <NA> <NA>      0     0   0
## 3  3 <NA> <NA>      0     0   0
## 4  4 <NA> <NA>      0     0   0
## 5  5 <NA> <NA>      0     0   0
## 6  6 <NA> <NA>      0     0   0
```

```
table(generation = pp$gen)
```

```
## generation
##    0    1    2    3    4    5    6    7
## 2420 1208  777  612  757  562  173   38
```

```
c(animals = nrow(pp), with_sire = sum(pp$sire_i > 0), with_dam = sum(pp$dam_i >
0))
```

```
##   animals with_sire  with_dam
##      6547      2645      3454
```

```
t_A <- system.time(A <- Amatrix(pp))
Fi <- inbreeding_coef(pp)
t_A[3]
```

```
## elapsed
##   0.622
```

```r
summary(Fi[Fi > 0])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00049 0.00781 0.01562 0.03744 0.03125 0.25000
```

```r
sum(Fi > 0)
```

```
## [1] 31
```

```r
all.equal(unname(diag(A) - 1), unname(Fi))
```

```
## [1] TRUE
```

```r
Ai <- Ainverse(pp)
c(dim = nrow(Ai), non_zeros = length(Ai@x), density = length(Ai@x) /
nrow(Ai)^2)
```

```
##       dim non_zeros   density
## 6.547e+03 1.460e+04 3.405e-04
```

A matriz densa **A** de 6.547 animais ocupa cerca de 340 MB. Sua inversa, construída diretamente com as regras de Henderson, é muito esparsa. Os coeficientes de endogamia calculados com o algoritmo de Meuwissen e Luo concordam com a diagonal de **A**.

8.3 Modelo animal para a primeira lactação

O modelo animal (Henderson, 1984) para a produção de leite na primeira lactação é

$$y_{ij} = \mu + h_j + a_i + e_{ij}, \qquad \mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A}), \qquad \mathbf{h} \sim N(\mathbf{0}, \sigma_h^2 \mathbf{I}),$$

em que h_j é o efeito de rebanho, aqui aleatório porque vários rebanhos têm poucas vacas.

```r
m1 <- subset(milk, lact == 1)
t_m1 <- system.time(
  a1 <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A), data = m1,
trace = FALSE))
t_m1[3]
```

```
## elapsed
##   2.171
```

```r
summary(a1)
```

```
## MLREML fit (AI-REML)
## Observations: 1314   Fixed-effect columns: 1   Iterations: 6 (converged)
## REML log-likelihood: -3605.4877   AIC: 7216.98   BIC: 7232.52
##
## Variance parameters:
##                       estimate std.error z.ratio bound
## id(herd)!sigma2         5.6462    1.3650    4.14     P
## vm(animal, A)!sigma2    0.6750    0.8654    0.78     P
## R!units!sigma2         12.3830    0.9616   12.88     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```r
vpredict(a1, h2 ~ V2 / (V1 + V2 + V3))
```

```
##    estimate std.error
## h2  0.03609   0.04629
```

A herdabilidade estimada é baixa e seu erro padrão é grande, porque cada vaca tem um único registro e o pedigree relaciona poucas vacas fenotipadas entre si. Os valores genéticos (EBVs) são preditos para todos os 6.547 animais do pedigree, incluindo touros e ancestrais sem registros:

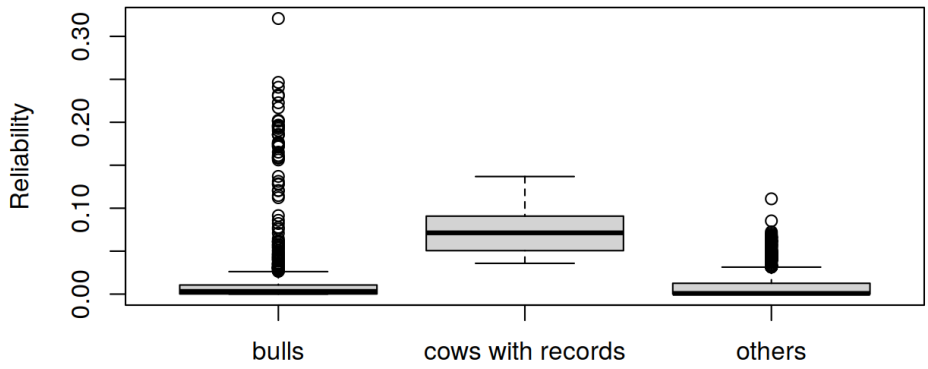
```
ebv <- coef(a1, "random", "vm(animal, A)")
nrow(ebv)

## [1] 6547

bulls <- unique(pp$sire[pp$sire_i > 0])
ebv_bulls <- ebv[ebv$level %in% bulls, ]
head(ebv_bulls[order(-ebv_bulls$blup), ], 8)

##      level   blup    pev     se reliability
## 4742  3749 0.7434 0.5434 0.7372     0.19497
## 5020  1630 0.5761 0.5125 0.7159     0.24076
## 4603  2896 0.5031 0.5393 0.7343     0.20114
## 4743  3754 0.4531 0.5426 0.7366     0.19616
## 5069  2833 0.4063 0.5496 0.7413     0.18586
## 3953  2863 0.2970 0.5449 0.7382     0.19282
## 3655  1424 0.2847 0.6334 0.7959     0.06166
## 4432  1679 0.2505 0.5194 0.7207     0.23050

group <- ifelse(ebv$level %in% m1$animal, "cows with records",
                ifelse(ebv$level %in% bulls, "bulls", "others"))
boxplot(reliability ~ group, data = cbind(ebv, group), ylab = "Reliability",
xlab = "")
```



Confiabilidade dos EBVs das vacas com registros, dos touros e dos demais animais do pedigree.

Embora os touros não tenham registros próprios, suas confiabilidades são maiores que as das vacas com um único registro, porque seu EBV acumula a informação de todas as suas filhas. Com uma herdabilidade tão baixa, no entanto, todas as confiabilidades são modestas.

8.4 Modelo de repetibilidade com todas as lactações

Com várias lactações por vaca, o valor genético a_i é separado do ambiente permanente p_i, comum a todos os registros de uma vaca, mas não herdável:

$$y_{ijk} = \mu + \ell_k + h_j + a_i + p_i + e_{ijk}, \qquad \mathbf{p} \sim N(\mathbf{0}, \sigma_p^2 \mathbf{I}).$$

No MLREML, o ambiente permanente é simplesmente um segundo termo com a mesma variável de animal e uma estrutura identidade:

```
t_rep <- system.time(
  a2 <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, A) + id(animal),
               data = milk, trace = FALSE))
t_rep[3]
```

```
## elapsed
##   24.67
```

```
summary(a2)$varcomp
```

```
##                       estimate std.error z.ratio bound
## id(herd)!sigma2         4.4460    1.0408   4.272     P
## vm(animal, A)!sigma2    0.7982    0.7329   1.089     P
## id(animal)!sigma2       4.7206    0.7970   5.923     P
## R!units!sigma2         10.3955    0.3236  32.121     P
```

```
vpredict(a2, h2 ~ V2 / (V1 + V2 + V3 + V4))
```

```
##    estimate std.error
## h2   0.0392   0.03597
```

```
vpredict(a2, repeatability ~ (V2 + V3) / (V1 + V2 + V3 + V4))
```

```
##                estimate std.error
## repeatability    0.2711   0.02212
```

```
wald(a2)
```

```
##          term df wald.inc   F.inc ddf.inc     p.inc wald.con   F.con ddf.con
## 1 (Intercept)  1   6258.1 6201.56   52.88 1.702e-56   6258.1 6201.56   52.88
## 2        lact  4    157.8   39.39 2425.07 5.312e-32    157.8   39.39 2425.07
##       p.con   p.chisq
## 1 1.702e-56 0.000e+00
## 2 5.312e-32 4.409e-33
```

A repetibilidade, a correlação entre duas lactações da mesma vaca, é muito maior que a herdabilidade: grande parte da semelhança entre lactações se deve ao ambiente permanente. Com mais registros por animal, a herdabilidade é estimada com mais precisão do que na análise de uma única lactação.

8.5 Efeito materno

O pedigree identifica a mãe de cada vaca. Um efeito de ambiente permanente materno (o ambiente que a mãe proporciona, compartilhado pelas irmãs maternas) é adicionado como um termo aleatório com o fator mãe:

```
m1$dam <- pp$dam[match(m1$animal, pp$id)]
m1m <- subset(m1, !is.na(dam))
a3 <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A) + id(dam), data =
m1m, trace = FALSE)
a3b <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A), data = m1m,
trace = FALSE)
summary(a3)$varcomp
```

```
##                         estimate std.error   z.ratio bound
## id(herd)!sigma2      6.464e+00     1.669 3.872e+00     P
## vm(animal, A)!sigma2 1.457e+00     1.284 1.135e+00     P
## id(dam)!sigma2       1.723e-07     2.568 6.709e-08     B
## R!units!sigma2       1.100e+01     2.676 4.110e+00     P
```

```
lrt(a3b, a3)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.341e-07     0.5 4957 4959 4972 4978
```

A variância materna está na fronteira (código `B`): não há evidência de um efeito ambiental materno sobre a produção de leite dessas vacas. A estatística da razão de verossimilhanças é zero dentro da precisão numérica.

Um efeito genético materno correlacionado com o efeito genético direto é ajustado com o termo `str()`, apresentado na Seção 8.7.

8.6 Análise bivariada de leite e gordura

Para estimar a correlação genética entre a produção de leite e de gordura na primeira lactação, ajustamos um modelo animal bivariado com matrizes de covariância não estruturadas para os efeitos de rebanho, genético e residual:

$$\text{Var}\begin{pmatrix}\mathbf{a}_M \\ \mathbf{a}_F\end{pmatrix} = \begin{pmatrix}\sigma^2_{a_M} & \sigma_{a_{MF}} \\ \sigma_{a_{MF}} & \sigma^2_{a_F}\end{pmatrix} \otimes \mathbf{A}.$$

```
rm(a2, a3, a3b, Ai, ebv); invisible(gc())
t_biv <- system.time(
  bv <- mlreml(cbind(milk_t, fat_h) ~ trait,
               random = ~ us(trait):id(herd) + us(trait):vm(animal, A),
               residual = ~ us(trait):units, data = m1, trace = FALSE))
t_biv[3]
```

```
## elapsed
##   37.63
```

```
summary(bv)$varcomp
```

```
##                                              estimate std.error z.ratio bound
## us(trait):id(herd)!trait_milk_t:milk_t         5.5995    1.3545  4.1339     P
## us(trait):id(herd)!trait_fat_h:milk_t          1.6135    0.4267  3.7812     P
## us(trait):id(herd)!trait_fat_h:fat_h           0.6589    0.1601  4.1150     P
## us(trait):vm(animal, A)!trait_milk_t:milk_t    0.6726    0.8661  0.7765     P
## us(trait):vm(animal, A)!trait_fat_h:milk_t     0.1022    0.2785  0.3669     P
## us(trait):vm(animal, A)!trait_fat_h:fat_h      0.2399    0.1358  1.7667     P
## R!us(trait):units!trait_milk_t:milk_t         12.3885    0.9624 12.8725     P
## R!us(trait):units!trait_fat_h:milk_t           3.1642    0.3041 10.4068     P
## R!us(trait):units!trait_fat_h:fat_h            1.4939    0.1396 10.7042     P
```

```
vpredict(bv, rg ~ V5 / sqrt(V4 * V6))
```

```
##    estimate std.error
## rg   0.2544     0.538
```

```
vpredict(bv, re ~ V8 / sqrt(V7 * V9))
```

```
##    estimate std.error
## re   0.7355   0.03364

vpredict(bv, h2_milk ~ V4 / (V1 + V4 + V7))

##          estimate std.error
## h2_milk  0.03604   0.04644

vpredict(bv, h2_fat ~ V6 / (V3 + V6 + V9))

##         estimate std.error
## h2_fat   0.1003   0.05661

round(vc_matrix(bv, "us(trait):vm(animal, A)")$cor, 3)

##         milk_t fat_h
## milk_t  1.000 0.254
## fat_h   0.254 1.000

round(vc_matrix(bv, "residual")$cor, 3)

##         milk_t fat_h
## milk_t  1.000 0.736
## fat_h   0.736 1.000
```

Os parâmetros são denominados `V1` a `V9` na ordem da tabela: covariâncias de rebanho (V1–V3), genéticas (V4–V6) e residuais (V7–V9). A correlação residual entre leite e gordura é alta e estimada com precisão; a correlação genética tem um erro padrão grande, como esperado com apenas 1314 vacas.

8.7 Efeitos genéticos diretos e maternos

Em caracteres expressos no início da vida (peso ao nascer e à desmama, tamanho da leitegada, sobrevivência) e em alguns caracteres da progênie da mãe, o fenótipo de um animal depende de seus próprios genes e da habilidade materna de sua mãe, que também é em parte genética. Os efeitos genéticos direto e materno costumam ser correlacionados negativamente (Willham, 1963; Meyer, 1992). O modelo (2.17) é ajustado com o termo `str(animal, dam, K = A)` (Seção 2.11), geralmente junto com um efeito materno de ambiente permanente `id(dam)`. Como os registros de leite da Seção 8.1 incluem poucas mães com registros próprios (ver o final desta seção), a estimação é ilustrada primeiro com uma população simulada cujos parâmetros verdadeiros são conhecidos.

A função a seguir simula seis gerações de 600 animais a partir de 300 fundadores, com 25 touros por geração escolhidos entre os machos das duas últimas gerações; valores genéticos diretos e maternos com $\sigma^2_d = 0.40$, $\sigma^2_m = 0.20$ e $\sigma_{dm} = -0.10$ (correlação genética −0,35); uma variância materna de ambiente permanente de 0,10; uma variância residual de 0,50; e 30 grupos contemporâneos, tratados como fixos. Todo animal nascido após os fundadores tem um registro, de modo que a maioria das mães tem registros próprios.

```
sim_maternal <- function(seed = 2026, nf = 300, ngen = 6, noff = 600,
                         S = matrix(c(0.40, -0.10, -0.10, 0.20), 2), s2pe =
0.10, s2e = 0.50) {
  set.seed(seed)
  ped <- data.frame(id = paste0("F", 1:nf), sire = NA, dam = NA,
                    sex = rep(c("M", "F"), length.out = nf), gen = 0)
  for (g in 1:ngen) {                                  # sires and dams from the
last two generations
```

```
    cand <- ped[ped$gen >= g - 2, ]
    sires <- sample(cand$id[cand$sex == "M"], 25)
    ped <- rbind(ped, data.frame(id = paste0("G", g, "_", 1:noff),
                                 sire = sample(sires, noff, TRUE),
                                 dam = sample(cand$id[cand$sex == "F"], noff,
TRUE),
                                 sex = sample(c("M", "F"), noff, TRUE), gen =
g))
  }
  A <- Amatrix(ped[, 1:3])
  U <- t(chol(A)) %*% matrix(rnorm(2 * nrow(A)), ncol = 2) %*% chol(S)   #
(direct, maternal)
  rownames(U) <- rownames(A)
  d <- ped[ped$gen > 0, ]
  d$cg <- factor(paste(d$gen, sample(1:5, nrow(d), TRUE), sep = "_"))   #
contemporary groups
  pe <- setNames(rnorm(nrow(A), sd = sqrt(s2pe)), rownames(A))
  d$y <- 10 + rnorm(nlevels(d$cg))[d$cg] + U[d$id, 1] + U[d$dam, 2] + pe[d$dam]
+
         rnorm(nrow(d), sd = sqrt(s2e))
  list(data = d, ped = ped, A = A, U = U)
}
sm <- sim_maternal()
dsm <- sm$data; Asim <- sm$A
c(records = nrow(dsm), animals = nrow(sm$A), dams = length(unique(dsm$dam)),
  dams_with_own_record = sum(unique(dsm$dam) %in% dsm$id))
```

```
##              records              animals                 dams
dams_with_own_record
##                 3600                 3900                 1407
1257
```

O modelo é ajustado com o motor denso, com a matriz **A**, e com o motor esparso, com sua inversa:

```
t_d <- system.time(
  dm_d <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "dense"))
Aisim <- Ainverse(sm$ped)
t_s <- system.time(
  dm_s <- mlreml(y ~ cg, random = ~ str(id, dam, K = Aisim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "sparse"))
c(dense = t_d[3], sparse = t_s[3], logLik_difference = dm_d$loglik -
dm_s$loglik)
```

```
##     dense.elapsed    sparse.elapsed logLik_difference
##         3.352e+01         1.592e+00        -5.401e-09
```

```
cbind(true = c(0.40, -0.10, 0.20, 0.10, 0.50), summary(dm_d)$varcomp[,
c("estimate", "std.error")])
```

```
##                                 true estimate std.error
## str(id, dam, K = Asim)!id:id     0.4  0.40027   0.06655
## str(id, dam, K = Asim)!dam:id   -0.1 -0.12404   0.04039
## str(id, dam, K = Asim)!dam:dam   0.2  0.27205   0.04846
```

```
## id(dam)!sigma2                  0.1  0.04608   0.02658
## R!units!sigma2                  0.5  0.51821   0.03935
```

Ambos os motores atingem o mesmo máximo, e o motor esparso é muito mais rápido mesmo com 3.600 registros. Todas as estimativas estão a cerca de dois erros padrão ou menos dos valores verdadeiros. Os efeitos maternos são estimados com menos precisão que os efeitos diretos, como é bem conhecido, e a variância materna de ambiente permanente é parcialmente confundida com a variância genética materna.

```
vc_matrix(dm_d, "str(id, dam, K = Asim)")$cor

##          id     dam
## id   1.0000 -0.3759
## dam -0.3759  1.0000

vpredict(dm_d, h2_direct ~ V1 / (V1 + V2 + V3 + V4 + V5))

##           estimate std.error
## h2_direct   0.3598   0.05505

vpredict(dm_d, h2_total ~ (V1 + 1.5 * V2 + 0.5 * V3) / (V1 + V2 + V3 + V4 +
V5))

##          estimate std.error
## h2_total   0.3148   0.03447
```

`vc_matrix()` retorna $\widehat{\boldsymbol{\Sigma}}$ e sua matriz de correlação. A herdabilidade total de Willham (1972), que governa a resposta à seleção fenotípica, é menor que a herdabilidade direta porque a covariância direta–materna é negativa.

```
ebv <- coef(dm_d, "random", "str(id, dam, K = Asim)")
ebv[ebv$level %in% c("G6_1", "G6_2"), ]

##      effect level    blup    pev     se reliability
## 3330     id  G6_1 -0.3304 0.2288 0.4784      0.4305
## 3681     id  G6_2 -0.1133 0.2282 0.4777      0.4381
## 7230    dam  G6_1  0.2735 0.2311 0.4807      0.1539
## 7581    dam  G6_2  0.1242 0.2306 0.4802      0.1646

c(r_direct = cor(ebv$blup[ebv$effect == "id"], sm$U[ebv$level[ebv$effect ==
"id"], 1]),
  r_maternal = cor(ebv$blup[ebv$effect == "dam"], sm$U[ebv$level[ebv$effect ==
"dam"], 2]))

##   r_direct r_maternal
##     0.6712     0.5159

dm_0 <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim, cov = "diag") +
id(dam), data = dsm,
               trace = FALSE)
lrt(dm_0, dm_d, boundary = FALSE)

##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        14.69 0.000127 10047 10034 10072 10065
```

Cada animal recebe um valor genético direto (`effect = "id"`) e um materno (`effect = "dam"`), incluindo os machos, cujo valor materno se expressa em suas filhas. As correlações com os valores verdadeiros simulados mostram que os valores maternos são preditos com

menor acurácia. O modelo com `cov = "diag"` tem a covariância direta–materna fixada em zero; como uma covariância de zero não está na fronteira do espaço paramétrico, o teste da razão de verossimilhanças usa `boundary = FALSE`, e rejeita $\sigma_{dm} = 0$.

Aplicação aos dados de leite. Dos 1.314 registros de primeira lactação da Seção 8.3, 907 têm mãe conhecida, mas apenas 14 mães têm um registro próprio de primeira lactação. Sem registros das mães, o efeito genético materno é identificado apenas por meio dos parentescos entre meias-irmãs maternas e seus parentes, e a covariância direta–materna não é estimável: com `str(animal, dam, K = A)` a correlação estimada tende a −1 e o algoritmo não converge em 50 iterações. O modelo sem a covariância converge em poucas iterações, com a variância genética materna na fronteira:

```
c(records = nrow(m1), with_dam = sum(!is.na(m1$dam)),
  dams_with_own_record = sum(unique(na.omit(m1$dam)) %in% m1$animal))
```

```
##              records             with_dam dams_with_own_record
##                 1314                  907                   14
```

```
a5 <- mlreml(milk_t ~ 1, random = ~ id(herd) + str(animal, dam, K = A, cov =
"diag"),
             data = m1, trace = FALSE)
summary(a5)$varcomp
```

```
##                                              estimate std.error   z.ratio
bound
## id(herd)!sigma2                             5.646e+00    1.3654 4.135e+00
P
## str(animal, dam, K = A, cov = "diag")!animal 6.751e-01    0.8682 7.775e-01
P
## str(animal, dam, K = A, cov = "diag")!dam    6.103e-08    0.9978 6.117e-08
B
## R!units!sigma2                              1.238e+01    1.1259 1.100e+01
P
```

```
lrt(a1, a5)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.944e-07     0.5 7217 7219 7233 7240
```

Assim como para o efeito ambiental materno da Seção 8.5, esses dados não contêm evidência de uma variância genética materna para a produção de leite na primeira lactação: a log-verossimilhança é a mesma do modelo da Seção 8.3.

8.8 Grandes conjuntos de dados: o motor esparso

Os exemplos deste capítulo têm alguns milhares de registros, um tamanho em que o motor denso ainda é prático. O motor esparso (Seção 2.12) resolve, em vez disso, as equações do modelo misto, e sua vantagem cresce rapidamente com o tamanho dos dados. O modelo de repetibilidade da Seção 8.4 é ajustado novamente com ambos os motores; o motor esparso recebe a inversa de $\mathbf{A}$ construída por `Ainverse()`, que tem o atributo `INVERSE = TRUE`:

```
Ai <- Ainverse(pp)
t_dense <- system.time(
  a2d <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, A) +
id(animal),
                data = milk, trace = FALSE, engine = "dense"))
t_sparse <- system.time(
```

```
  a2s <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, Ai) +
id(animal),
                data = milk, trace = FALSE, engine = "sparse"))
c(dense = t_dense[3], sparse = t_sparse[3])

##  dense.elapsed sparse.elapsed
##         25.720          0.526

summary(a2s)

## MLREML fit (AI-REML, sparse engine)
## Observations: 3397   Fixed-effect columns: 5   Iterations: 7 (converged)
## REML log-likelihood: -9400.8394   AIC: 18809.68   BIC: 18834.20
##
## Variance parameters:
##                          estimate std.error z.ratio bound
## id(herd)!sigma2            4.4460    1.0410    4.27     P
## vm(animal, Ai)!sigma2      0.7982    0.7329    1.09     P
## id(animal)!sigma2          4.7206    0.7970    5.92     P
## R!units!sigma2            10.3960    0.3236   32.12     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

A saída é a da Seção 8.4, e o motor esparso precisa de bem menos de um segundo em vez de cerca de meio minuto. As diferenças entre os dois motores são da ordem da tolerância de convergência, porque as iterações seguem trajetórias numéricas ligeiramente diferentes:

```
e_d <- coef(a2d, "random", "vm(animal, A)"); e_s <- coef(a2s, "random",
"vm(animal, Ai)")
c(logLik = a2d$loglik - a2s$loglik, theta = max(abs(a2d$theta - a2s$theta)),
  ebv = max(abs(e_d$blup - e_s$blup)), pev = max(abs(e_d$pev - e_s$pev)),
  se_beta = max(abs(sqrt(diag(vcov(a2d))) - sqrt(diag(vcov(a2s))))))

##    logLik     theta       ebv       pev   se_beta
## 2.910e-10 1.573e-08 1.576e-08 1.891e-08 3.790e-11

a2s$sparse[c("N", "nnzL")]

## $N
## [1] 7968
##
## $nnzL
## [1] 106055
```

As equações do modelo misto deste modelo têm 7.968 incógnitas: 5 efeitos fixos, 57 rebanhos, os valores genéticos dos 6.547 animais do pedigree e os efeitos de ambiente permanente das 1.359 vacas com registros. O modelo bivariado da Seção 8.6, com matrizes de covariância não estruturadas para os efeitos de rebanho, genéticos e residuais e registros com um único caráter observado, também é ajustado pelo motor esparso:

```
t_bs <- system.time(
  bvs <- mlreml(cbind(milk_t, fat_h) ~ trait,
                random = ~ us(trait):id(herd) + us(trait):vm(animal, Ai),
                residual = ~ us(trait):units, data = m1, trace = FALSE, engine
= "sparse"))
t_bs[3]
```

```
## elapsed
##   1.655

c(logLik_dense = bv$loglik, difference = bv$loglik - bvs$loglik)

## logLik_dense   difference
##   -5.445e+03    1.237e-08

vpredict(bvs, rg ~ V5 / sqrt(V4 * V6))

##    estimate std.error
## rg   0.2544     0.538
```

O ajuste esparso leva menos de dois segundos (37,6 s com o motor denso na Seção 8.6) e atinge a mesma log-verossimilhança. A Seção 15.2 mostra o tamanho dos conjuntos de dados que o motor esparso processa, com dados simulados de até dois milhões de registros.

9 Exemplo 6: GBLUP de passo único com indivíduos genotipados e não genotipados

9.1 Contexto

Na prática, nem todo indivíduo com pedigree e fenótipo é genotipado. O GBLUP de passo único (Legarra et al., 2009; Christensen e Lund, 2010) usa, em uma única análise, o pedigree de todos os indivíduos e os marcadores dos genotipados, por meio da matriz $\mathbf{H}$ da equação (2.16). Os indivíduos não genotipados se beneficiam dos marcadores de seus parentes genotipados.

Para ilustrá-lo com dados reais, usamos novamente as 599 linhagens de trigo de `BGLR`, que têm pedigree e marcadores, e simulamos a situação habitual: apenas 360 linhagens (60 %) são consideradas genotipadas. Isso permite comparar as predições por pedigree (A-BLUP) e as predições de passo único (H-BLUP) com os fenótipos reais das linhagens não genotipadas.

```
data(wheat)
ids <- paste0("L", 1:599)
Aw <- wheat.A; dimnames(Aw) <- list(ids, ids)
Mw <- 2 * wheat.X; rownames(Mw) <- ids
yw <- setNames(wheat.Y[, 1], ids)
set.seed(7)
genotyped <- sample(ids, 360)
Gg <- Gmatrix(Mw[genotyped, ], method = "VanRaden")
```

9.2 Alinhamento de G com o pedigree e construção de H

Antes de combinar as matrizes, $\mathbf{G}$ é alinhada com $\mathbf{A}_{22}$ (o bloco do pedigree das linhagens genotipadas) e combinada com 5 % de $\mathbf{A}_{22}$:

```
A22 <- Aw[genotyped, genotyped]
Gal <- align_G(Gg, A22)
c(alpha = attr(Gal, "alpha"), beta = attr(Gal, "beta"))

##  alpha   beta
## 0.3799 0.7989
```

```
rbind(G_original = c(diag = mean(diag(Gg)), all = mean(Gg)),
      G_aligned = c(mean(diag(Gal)), mean(Gal)),
      A22 = c(mean(diag(A22)), mean(A22)))
```

```
##             diag       all
## G_original 2.000 3.550e-17
## G_aligned  1.978 3.799e-01
## A22        1.978 3.799e-01
```

```
H <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE)
range(eigen(H, symmetric = TRUE, only.values = TRUE)$values)
```

```
## [1] 9.786e-04 2.197e+02
```

Após o alinhamento, as médias da diagonal e de todos os elementos de $\mathbf{G}$ coincidem com as de $\mathbf{A}_{22}$. A $\mathbf{H}$ resultante é positiva definida. Se não fosse, `bend()` a corrigiria.

A inversa $\mathbf{H}^{-1}$, usada por programas baseados nas equações do modelo misto, é obtida com `inverse = TRUE`; a verificação numérica mostra que ela é a inversa de $\mathbf{H}$:

```
Hi <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE, inverse = TRUE)
max(abs(Hi %*% H - diag(599)))
```

```
## [1] 1.669e-11
```

9.3 Comparação por validação

Os fenótipos de 120 linhagens escolhidas aleatoriamente (genotipadas e não genotipadas) são removidos, preditos com A-BLUP e com H-BLUP, e a correlação com os fenótipos reais é calculada separadamente para ambos os grupos. Isso é repetido cinco vezes.

```
Ar <- Aw + diag(1e-3, 599); Hr <- H + diag(1e-3, 599)
res <- do.call(rbind, lapply(1:5, function(r) {
  set.seed(100 + r)
  test <- sample(ids, 120)
  ytr <- yw; ytr[test] <- NA
  pA <- gblup(ytr, Ar)$gebv
  pH <- gblup(ytr, Hr)$gebv
  ng <- setdiff(test, genotyped); gg <- intersect(test, genotyped)
  data.frame(rep = r, A_non_genot = cor(pA[ng], yw[ng]), H_non_genot =
cor(pH[ng], yw[ng]),
             A_genot = cor(pA[gg], yw[gg]), H_genot = cor(pH[gg], yw[gg]))
}))
round(res, 3)
```

```
##   rep A_non_genot H_non_genot A_genot H_genot
## 1   1       0.424       0.402   0.349   0.412
## 2   2       0.362       0.315   0.226   0.465
## 3   3       0.341       0.353   0.488   0.500
## 4   4       0.344       0.283   0.461   0.415
## 5   5       0.584       0.636   0.387   0.537
```

```
round(colMeans(res[, -1]), 3)
```

```
## A_non_genot H_non_genot     A_genot     H_genot
##       0.411       0.398       0.382       0.466
```

O ganho do H-BLUP sobre o pedigree concentra-se nas linhagens genotipadas, cujos parentescos realizados são conhecidos. Para as linhagens não genotipadas, H-BLUP e A-BLUP têm, em média, capacidade preditiva semelhante, com diferenças em ambas as direções entre as repetições. Em teoria, os indivíduos não genotipados podem se beneficiar porque suas covariâncias com os genotipados são atualizadas por meio de $\mathbf{A}_{12}\mathbf{A}_{22}^{-1}$, mas esse benefício depende de quão aparentados eles são com os indivíduos genotipados; nessas linhagens de trigo, com um pedigree que já descreve bem os parentescos, ele não é observado. Esse é um resultado comum na literatura: a vantagem do passo único para indivíduos não genotipados costuma ser pequena.

9.4 Passo único com o motor geral

`gblup()` foi usado acima por rapidez, mas **H** pode ser usada em qualquer modelo `mlreml()` por meio de `vm()`, por exemplo, com variâncias residuais heterogêneas ou em um modelo multiambiente:

```
dss <- data.frame(gen = ids, y = yw)
fss <- mlreml(y ~ 1, random = ~ vm(gen, Hr), data = dss, trace = FALSE)
summary(fss)$varcomp

##                     estimate std.error z.ratio bound
## vm(gen, Hr)!sigma2   0.3648   0.06446   5.659     P
## R!units!sigma2       0.5186   0.05096  10.176     P

vpredict(fss, h2 ~ V1 / (V1 + V2))

##    estimate std.error
## h2   0.4129   0.05974
```

9.5 Recomendações

- Verifique se os identificadores de **G** estão no pedigree e se o pedigree está completo para trás.
- Alinhe **G** com $\mathbf{A}_{22}$ (`align = TRUE`) e combine-a com uma pequena fração de $\mathbf{A}_{22}$ (5–10 %).
- Em populações com pedigrees rasos, os parâmetros τ e ω de $\mathbf{H}^{-1}$ podem ser ajustados para reduzir o viés de predição (Misztal et al., 2014).

10 Exemplo 7: medidas repetidas e regressão aleatória

Muitos caracteres são medidos várias vezes no mesmo indivíduo: peso em diferentes idades, produção de leite ao longo da lactação, altura das plantas durante a estação, produtividade em cortes sucessivos. Tratar essas observações como independentes subestima os erros padrão e desperdiça a informação sobre a trajetória de cada indivíduo. O MLREML oferece duas abordagens complementares: modelar a correlação dos resíduos dentro dos indivíduos (estruturas residuais) e modelar as trajetórias individuais com coeficientes aleatórios (regressão aleatória).

10.1 Correlação residual: crescimento de crianças

Os dados `Orthodont` (Potthoff e Roy, 1964; em `nlme`) registram a distância da hipófise à fissura pterigomaxilar em 27 meninos e meninas aos 8, 10, 12 e 14 anos. Comparam-se várias estruturas para a matriz de covariância dos quatro registros de cada criança.

```
library(nlme)
ort <- as.data.frame(Orthodont)
ort$Subject <- factor(as.character(ort$Subject))
ort$agef <- ort$age
o1 <- mlreml(distance ~ age * Sex, random = ~ Subject, data = ort, trace =
FALSE)
o2 <- mlreml(distance ~ age * Sex, residual = ~ ar1(agef):id(Subject), data =
ort, trace = FALSE)
o3 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
ar1(agef):id(Subject),
             data = ort, trace = FALSE)
o4 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units,
             data = ort, trace = FALSE)
o5 <- mlreml(distance ~ age * Sex, residual = ~ us(agef):id(Subject), data =
ort, trace = FALSE)
data.frame(model = c("random intercept (compound symmetry)", "AR1",
                     "intercept + AR1", "intercept + residual by sex",
"unstructured"),
           parameters = sapply(list(o1, o2, o3, o4, o5), function(f)
sum(f$free)),
           logLik = sapply(list(o1, o2, o3, o4, o5), `[[`, "loglik"),
           AIC = sapply(list(o1, o2, o3, o4, o5), `[[`, "AIC"))
```

```
##                                  model parameters logLik   AIC
## 1 random intercept (compound symmetry)          2 -216.9 437.8
## 2                                  AR1          2 -222.3 448.6
## 3                       intercept + AR1          3 -216.9 439.8
## 4          intercept + residual by sex          3 -207.6 421.2
## 5                           unstructured         10 -212.3 444.5
```

Em `ar1(agef)`, as idades são numéricas, de modo que o MLREML usa as distâncias reais entre as medições: a correlação entre dois registros separados por Δ anos é ρ^{Δ}. Assim, a mesma estrutura funciona para medições em intervalos irregulares, como faria `expc()`. O modelo com uma variância residual específica para cada sexo é preferido pelo AIC: as meninas têm uma variabilidade intraindividual muito menor que os meninos.

```
summary(o4)
```

```
## MLREML fit (AI-REML)
## Observations: 108   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -207.6102   AIC: 421.22   BIC: 429.15
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## Subject!sigma2                  3.4135    1.0870    3.14     P
## R!diag(Sex):units!Sex_Male      2.7883    0.5711    4.88     P
## R!diag(Sex):units!Sex_Female    0.6104    0.1528    3.99     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

10.2 Regressão aleatória linear: privação de sono

Os dados `sleepstudy` (Belenky et al., 2003; em `lme4`) registram o tempo médio de reação de 18 pessoas durante 10 dias de privação de sono. Cada pessoa tem sua própria reta, com intercepto e inclinação aleatórios correlacionados:

$$y_{ij} = (\beta_0 + b_{0i}) + (\beta_1 + b_{1i})\, t_j + e_{ij}, \qquad \begin{pmatrix} b_{0i} \\ b_{1i} \end{pmatrix} \sim N\left(\mathbf{0}, \begin{pmatrix} \sigma_0^2 & \sigma_{01} \\ \sigma_{01} & \sigma_1^2 \end{pmatrix}\right).$$

No MLREML, a base `pol(Days, 1)` gera as colunas $(1, t)$ e a estrutura `us` sua matriz de covariância; o termo completo é `us(pol(Days, 1)):id(Subject)`.

```
data(sleepstudy, package = "lme4")
rr <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE)
summary(rr)

## MLREML fit (AI-REML)
## Observations: 180   Fixed-effect columns: 2   Iterations: 4 (converged)
## REML log-likelihood: -871.8141   AIC: 1751.63   BIC: 1764.36
##
## Variance parameters:
##                                                 estimate std.error z.ratio
bound
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_0:0  612.090    288.80    2.12
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:0    9.604     46.68    0.21
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:1   35.072     14.78    2.37
P
## R!units!sigma2                                 654.940     77.19    8.49
P
##
## Bound codes: P positive/free, B at boundary, F fixed.

vpredict(rr, cor_int_slope ~ V2 / sqrt(V1 * V3))

##                estimate std.error
## cor_int_slope  0.06555     0.325
```

Os BLUPs dos coeficientes de cada pessoa permitem reconstruir suas retas. Os níveis de um termo seguem a ordem de seus componentes, com o último variando mais rapidamente. Em `us(pol(Days, 1)):id(Subject)`, o primeiro componente é a base polinomial e o segundo, a pessoa, de modo que os primeiros 18 BLUPs são os interceptos das 18 pessoas e os 18 seguintes, suas inclinações. A matriz de coeficientes é formada por colunas:

```
b <- coef(rr, "random")[[1]]
head(b$level, 3)

## [1] "0:308" "0:309" "0:310"

subj <- levels(factor(sleepstudy$Subject))

B <- matrix(b$blup, ncol = 2, dimnames = list(subj, c("intercept", "slope")))
head(B)

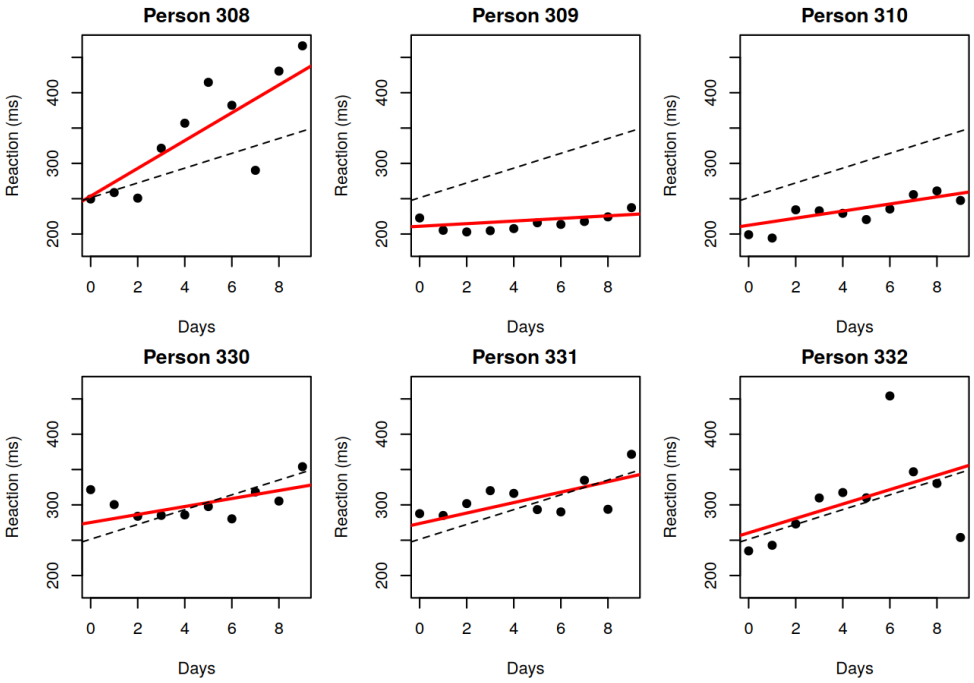
##     intercept   slope
## 308     2.259  9.1990
## 309   -40.399 -8.6197
## 310   -38.960 -5.4489
## 330    23.691 -4.8143
```

```
## 331    22.260 -3.0699
## 332     9.040 -0.2722
```

```
head(lme4::ranef(lme4::lmer(Reaction ~ Days + (Days | Subject),
sleepstudy))$Subject)
```

```
##     (Intercept)    Days
## 308       2.259  9.1990
## 309     -40.399 -8.6197
## 310     -38.960 -5.4489
## 330      23.691 -4.8144
## 331      22.260 -3.0699
## 332       9.040 -0.2722
```

```
op <- par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (s in subj[1:6]) {
  ds <- subset(sleepstudy, Subject == s)
  plot(ds$Days, ds$Reaction, pch = 19, ylim = c(180, 470), main =
paste("Person", s),
       xlab = "Days", ylab = "Reaction (ms)")
  abline(rr$beta[1] + B[s, 1], rr$beta[2] + B[s, 2], col = "red", lwd = 2)
  abline(rr$beta, lty = 2)
}
```



Tempo de reação observado (pontos), retas individuais preditas (vermelho) e reta média (tracejada).

```
par(op)
```

10.3 Regressão aleatória com polinômios de Legendre: curvas de crescimento

No melhoramento animal, as trajetórias são modeladas com polinômios de Legendre (Kirkpatrick et al., 1990; Schaeffer, 2004), que são ortogonais em $[-1,1]$ e fornecem matrizes de covariância mais bem condicionadas do que os polinômios brutos. Os dados `ChickWeight` (Crowder e Hand, 1990) contêm o peso de 50 pintinhos alimentados com quatro dietas, medido do nascimento ao dia 21.

```
cw <- as.data.frame(ChickWeight)
cw$Chick <- factor(as.character(cw$Chick))
ck1 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 1)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
ck2 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 2)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
lrt(ck1, ck2, boundary = FALSE)
```

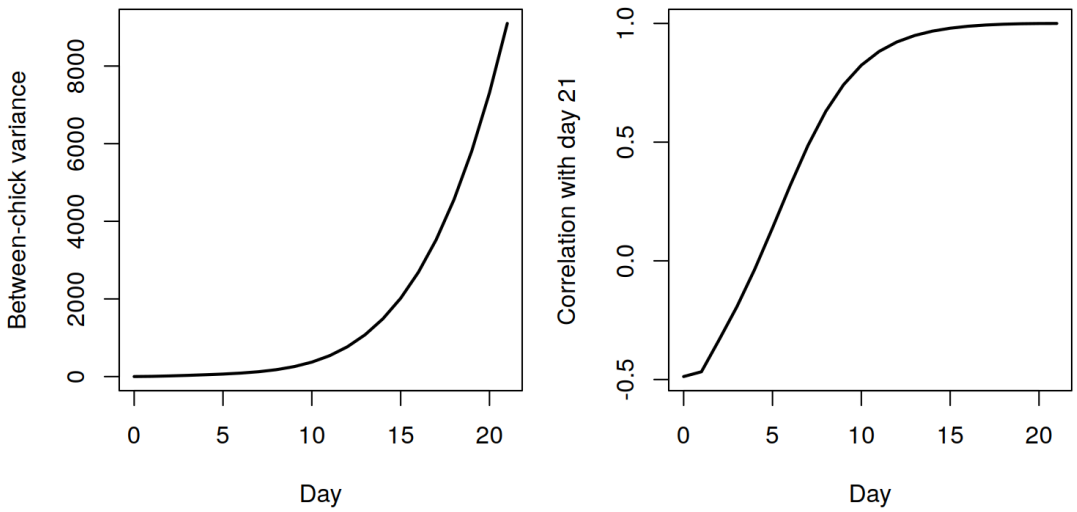
```
##   df LR.statistic   p.value AIC0 AIC1 BIC0 BIC1
## 1  3        445.3 3.436e-96 4538 4099 4603 4177
```

```
summary(ck2)$varcomp[1:6, ]
```

```
##                                                 estimate std.error z.ratio bound
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_0:0   1665.1    367.06   4.536     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:0   1542.1    340.91   4.523     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:1   1526.5    325.35   4.692     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:0    442.1    105.07   4.208     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:1    484.7    104.72   4.629     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:2    175.1     36.42   4.808     P
```

O modelo quadrático é claramente melhor. A função de covariância entre os pesos em duas idades t e t' é $\boldsymbol{\phi}(t)'\mathbf{K}\boldsymbol{\phi}(t')$, em que $\boldsymbol{\phi}(t)$ é o vetor de polinômios de Legendre avaliados em t e $\mathbf{K}$ é a matriz de covariância dos coeficientes. A partir dela, obtemos a variância entre pintinhos em cada idade e a correlação entre idades:

```
Kc <- vc_matrix(ck2, "us(leg(Time, 2)):id(Chick)")$cov
tt <- 0:21
Phi <- MLREML:::.legendre(c(tt, range(cw$Time)), 2)[seq_along(tt), ]
Cfun <- Phi %*% Kc %*% t(Phi)
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(tt, diag(Cfun), type = "l", lwd = 2, xlab = "Day", ylab = "Between-chick
variance")
plot(tt, cov2cor(Cfun)[, 22], type = "l", lwd = 2, xlab = "Day", ylab =
"Correlation with day 21")
```



Variância entre pintinhos estimada por regressão aleatória (esquerda) e correlação do peso em cada idade com o peso no dia 21 (direita).

```
par(op)
```

Com um pedigree, o mesmo termo com `vm(animal, A)` em vez de `id(Chick)` fornece a função de covariância genética aditiva, a partir da qual se obtêm as herdabilidades em cada idade e as correlações genéticas entre idades. Um modelo completo de regressão aleatória no melhoramento animal normalmente tem um termo genético e um termo de ambiente permanente:

```
mlreml(weight ~ group + leg(age, 3),
       random = ~ us(leg(age, 2)):vm(animal, A) + us(leg(age, 2)):id(animal),
       residual = ~ diag(age_class):units, data = records)
```

11 Exemplo 8: análise dialélica

11.1 Os dados

O conjunto de dados `hayman.tobacco` (Hayman, 1954; em `agridat`) é um dialelo completo de 8 variedades de tabaco (*Nicotiana rustica*): as 64 combinações de genitor feminino e masculino, incluindo autofecundações e cruzamentos recíprocos, avaliadas em 2 blocos em cada um de 2 anos. A resposta é o número de dias até o florescimento.

```
data(hayman.tobacco)
dt <- hayman.tobacco
dt$year_f <- factor(dt$year)
str(dt)
```

```
## 'data.frame':    256 obs. of  6 variables:
##  $ year  : int  1951 1951 1951 1951 1951 1951 1951 1951 1951 1951 ...
##  $ block : Factor w/ 2 levels "B1","B2": 1 1 1 1 1 1 1 1 2 2 ...
##  $ male  : Factor w/ 8 levels "G1","G2","G3",..: 1 2 3 4 5 6 7 8 1 2 ...
##  $ female: Factor w/ 8 levels "G1","G2","G3",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ day   : num  27.6 13.6 24.6 31.8 15 18.2 17.4 15.2 30.2 14.2 ...
##  $ year_f: Factor w/ 2 levels "1951","1952": 1 1 1 1 1 1 1 1 1 1 ...
```

```
with(dt, table(female, male))[1:4, 1:4]
```

```
##       male
## female G1 G2 G3 G4
##     G1  4  4  4  4
##     G2  4  4  4  4
##     G3  4  4  4  4
##     G4  4  4  4  4
```

11.2 O modelo

Em um dialelo, o valor do cruzamento entre a fêmea i e o macho j é decomposto como (Griffing, 1956; Cockerham e Weir, 1977):

$$y_{ijk} = \mu + \text{environment}_k + g_i + g_j + s_{ij} + m_i + r_{ij} + e_{ijk},$$

em que g é a capacidade geral de combinação (CGC) de cada genitor, a mesma quer ele atue como fêmea ou como macho; $s_{ij} = s_{ji}$ é a capacidade específica de combinação (CEC) do par, independente da direção do cruzamento; m_i é o efeito materno geral da fêmea; e r_{ij} é o efeito recíproco específico da combinação ordenada. Com genitores aleatórios, cada efeito tem sua própria variância.

A dificuldade técnica é a CGC: cada observação tem **dois** efeitos do mesmo fator. No MLREML isso é resolvido com a base de múltipla pertinência `mm(female, male)`, cuja matriz de delineamento tem um 1 na coluna de cada genitor (e um 2 para autofecundações). A CEC usa um fator com o cruzamento não ordenado e os efeitos recíprocos, um fator com o cruzamento ordenado.

```
dt$cross <- factor(with(dt, paste(pmin(as.character(female),
as.character(male)),
                                  pmax(as.character(female),
as.character(male)), sep = "x")))
dt$cross_ord <- factor(paste(dt$female, dt$male, sep = ">"))
c(unordered_crosses = nlevels(dt$cross), ordered_crosses =
nlevels(dt$cross_ord))
```

```
## unordered_crosses   ordered_crosses
##                36                64
```

```
di <- mlreml(day ~ year_f + year_f:block,
             random = ~ mm(female, male) + cross + female + cross_ord,
             data = dt, trace = FALSE)
summary(di)
```

```
## MLREML fit (AI-REML)
## Observations: 256   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -676.2058   AIC: 1362.41   BIC: 1380.06
##
## Variance parameters:
##                            estimate std.error z.ratio bound
## mm(female, male)!sigma2   9.0600    5.1950    1.74     P
## cross!sigma2              3.8401    1.6140    2.38     P
## female!sigma2             0.4936    0.6835    0.72     P
## cross_ord!sigma2          0.7941    0.9461    0.84     P
## R!units!sigma2            8.3574    0.8597    9.72     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

11.3 Interpretação

A variância da CGC é a maior das variâncias genéticas, indicando que a maior parte da variação genética nos dias até o florescimento é aditiva e é transmitida de forma previsível à progênie. A variância da CEC reflete efeitos não aditivos (dominância e epistasia) específicos de cada combinação. Os efeitos maternos e recíprocos são pequenos.

Sob as suposições usuais (genitores não aparentados, ausência de epistasia), $\sigma^2_A = 4\sigma^2_{GCA}$ em uma população diploide não endogâmica e $2\sigma^2_{GCA}$ para genitores totalmente endogâmicos, que é o caso das variedades de tabaco; a proporção da variância genética que é aditiva é estimada com `vpredict()`:

```
vpredict(di, prop_GCA ~ 2 * V1 / (2 * V1 + V2 + V3 + V4))
```

```
##          estimate std.error
## prop_GCA   0.7794    0.1137
```

```
vpredict(di, Baker ~ 2 * V1 / (2 * V1 + V2))
```

```
##       estimate std.error
## Baker   0.8251    0.1033
```

A razão de Baker (1978), $2\sigma^2_{GCA}/(2\sigma^2_{GCA} + \sigma^2_{SCA})$, próxima de um indica que predizer cruzamentos a partir da CGC dos genitores será preciso.

11.4 Capacidades de combinação preditas

```
gca <- coef(di, "random", "mm(female, male)")
gca[order(gca$blup), c("level", "blup", "se", "reliability")]
```

```
##   level    blup   se reliability
## 7    G7 -3.9663 1.29      0.8163
## 8    G8 -2.0061 1.29      0.8163
## 2    G2 -1.6694 1.29      0.8163
## 5    G5 -1.3773 1.29      0.8163
## 6    G6  0.2339 1.29      0.8163
## 4    G4  0.5217 1.29      0.8163
## 1    G1  3.7705 1.29      0.8163
## 3    G3  4.4929 1.29      0.8163
```

Como a resposta é o número de dias até o florescimento, genitores com CGC negativa transmitem precocidade. O valor predito de um cruzamento não avaliado $i \times j$ é obtido somando a média, as duas CGCs e, quando disponíveis, a CEC e os efeitos maternos:

```
sca <- coef(di, "random", "cross")
g <- setNames(gca$blup, gca$level); s <- setNames(sca$blup, sca$level)
mu <- mean(di$fixed_fitted)
pred_cross <- function(p1, p2) {
  k <- paste(min(p1, p2), max(p1, p2), sep = "x")
  unname(mu + g[p1] + g[p2] + ifelse(k %in% names(s), s[k], 0))
}
c(G1xG3 = pred_cross("G1", "G3"), G7xG8 = pred_cross("G7", "G8"))
```

```
## G1xG3 G7xG8
## 27.85 10.95
```

11.5 Comparação de modelos

Os efeitos maternos e recíprocos são necessários? Modelos aninhados são comparados com a razão de verossimilhanças:

```
di0 <- mlreml(day ~ year_f + year_f:block, random = ~ mm(female, male) + cross,
              data = dt, trace = FALSE)
lrt(di0, di, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        3.974  0.1371 1362 1362 1373 1380
```

Com duas variâncias testadas simultaneamente na fronteira, a distribuição nula é uma mistura de χ^2_0, χ^2_1 e χ^2_2; o teste χ^2_2 mostrado aqui é conservador.

A mesma sintaxe serve para delineamentos de cruzamento como North Carolina I e II, o dialelo sem recíprocos (método 4 de Griffing) ou top-crosses, alterando quais termos são incluídos. Se os genitores tiverem pedigree ou marcadores, `mm()` não é combinado diretamente com `vm()` nesta versão. A solução é construir a matriz de covariância da CGC no nível do cruzamento: se $\mathbf{Z}_{mm}$ é a matriz de múltipla pertinência dos cruzamentos ordenados e $\mathbf{K}$ a matriz de parentesco dos genitores, $\mathrm{Var}(g_i + g_j)$ entre cruzamentos é $\sigma^2_{GCA}\,\mathbf{Z}_{mm}\mathbf{K}\mathbf{Z}_{mm}'$, e o termo é escrito `vm(cross_ord, Kc)` com `Kc` igual a essa matriz:

```
cr <- levels(dt$cross_ord)
p1 <- sub(">.*", "", cr); p2 <- sub(".*>", "", cr)
Zmm <- outer(p1, rownames(K), "==") + outer(p2, rownames(K), "==")
Kc <- Zmm %*% K %*% t(Zmm); dimnames(Kc) <- list(cr, cr)
mlreml(day ~ year_f + year_f:block, random = ~ vm(cross_ord, Kc) + cross, data
= dt)
```

12 Exemplo 9: ensaio de geração precoce sem repetições

12.1 Os dados

Nas fases iniciais de um programa de melhoramento, centenas de novas linhagens são avaliadas com uma única parcela cada, intercaladas com variedades testemunhas repetidas que servem para estimar o erro e a variação espacial. Cullis, Smith e Coombes (2006) mostraram que a análise espacial é especialmente valiosa nesses ensaios, porque sem repetição não há outra maneira de separar o mérito genético da fertilidade da parcela. Os dados `cullis.earlygen` (em `agridat`) provêm desse trabalho: 670 parcelas de trigo em 67 linhas por 10 colunas, com 532 genótipos. A maioria aparece apenas uma vez; as testemunhas são repetidas. Também foi registrada uma nota de plantas daninhas para cada parcela.

```
data(cullis.earlygen)
eg <- cullis.earlygen
c(plots = nrow(eg), genotypes = nlevels(eg$gen), rows = max(eg$row), columns =
max(eg$col))
```

```
##     plots genotypes      rows   columns
##       670       532        67        10
```

```
rep_gen <- table(eg$gen)
table(replicates = rep_gen)
```

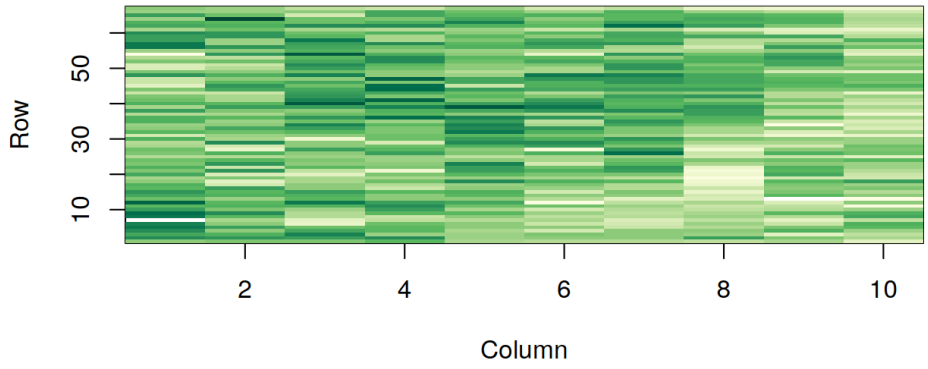
```
## replicates
##   1   3   4   5 121
## 525   1   4   1   1
```

```
head(sort(rep_gen, decreasing = TRUE), 6)
```

```
##
##    Kite   Banks  Hartog   K1056 Rosella  Vulcan
##     121       5       4       4       4       4
```

```
eg$check <- factor(ifelse(rep_gen[as.character(eg$gen)] > 1, "check", "line"))
```

```
Ym <- tapply(eg$yield, list(eg$col, eg$row), mean)
image(1:10, 1:67, Ym, col = hcl.colors(30, "YlGn", rev = TRUE), xlab =
"Column", ylab = "Row")
```



Produtividade observada nas 670 parcelas (67 linhas por 10 colunas).

12.2 Modelos

Comparam-se três modelos com os mesmos efeitos fixos (média, nota de plantas daninhas como covariável e diferença média entre testemunhas e novas linhagens) e efeitos genéticos aleatórios:

1. resíduos independentes;
2. resíduo AR1 × AR1 separável;
3. resíduo AR1 × AR1 mais um nugget independente.

```
e1 <- mlreml(yield ~ weed + check, random = ~ gen, data = eg, trace = FALSE)
e2 <- mlreml(yield ~ weed + check, random = ~ gen, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
e3 <- mlreml(yield ~ weed + check, random = ~ gen + units, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
data.frame(model = c("independent", "AR1 x AR1", "AR1 x AR1 + nugget"),
           parameters = sapply(list(e1, e2, e3), function(f) sum(f$free)),
           logLik = sapply(list(e1, e2, e3), `[[`, "loglik"),
           AIC = sapply(list(e1, e2, e3), `[[`, "AIC"))
```

```
##                 model parameters logLik  AIC
## 1         independent          2  -4897 9798
## 2           AR1 x AR1          4  -4840 9688
## 3 AR1 x AR1 + nugget          5  -4833 9676
```

```
summary(e3)
```

```
## MLREML fit (AI-REML)
## Observations: 668   Fixed-effect columns: 3   Iterations: 7 (converged)
## REML log-likelihood: -4832.9955   AIC: 9675.99   BIC: 9698.49
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                   7.237e+04 1.046e+04    6.92     P
## units!sigma2                 3.274e+04 7.974e+03    4.11     P
## R!ar1(row):ar1(col)!sigma2   5.704e+04 1.123e+04    5.08     P
## R!ar1(row):ar1(col)!row_cor 8.561e-01 3.998e-02   21.41     P
```

```
## R!ar1(row):ar1(col)!col_cor 4.027e-01 1.137e-01    3.54     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
wald(e3)
```

```
##          term df  wald.inc     F.inc ddf.inc     p.inc  wald.con     F.con
ddf.con
## 1 (Intercept)  1 3.521e+03 3.946e+03   10.93 2.480e-15 3.521e+03 3.946e+03
10.93
## 2        weed  1 8.288e+01 7.938e+01  496.09 9.735e-18 8.289e+01 7.940e+01
495.97
## 3       check  1 1.743e-02 1.737e-02  164.50 8.953e-01 1.743e-02 1.737e-02
164.50
##       p.con   p.chisq
## 1 2.480e-15 0.000e+00
## 2 9.693e-18 8.656e-20
## 3 8.953e-01 8.950e-01
```

Os modelos espaciais melhoram claramente a verossimilhança em relação ao modelo com resíduos independentes. O efeito das plantas daninhas é significativo e negativo, um exemplo de variação estranha que deve ser modelada com covariáveis quando tiver sido medida.

12.3 Consequências para a seleção

O objetivo de um ensaio de geração precoce é selecionar a fração superior das linhagens. A questão prática é se o modelo altera quais linhagens são selecionadas. Comparamos os BLUPs das novas linhagens sob o modelo independente e sob o modelo espacial preferido pelo AIC:

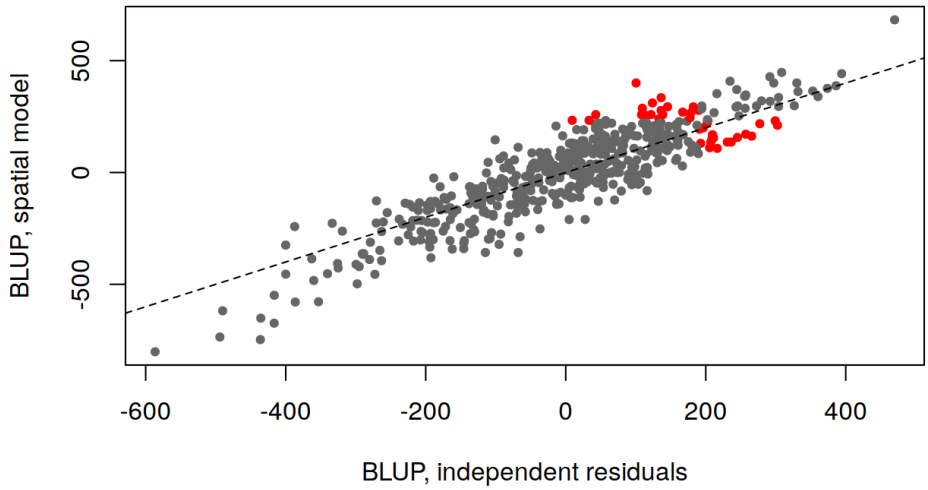
```
b1 <- coef(e1, "random", "gen"); bb <- coef(eb, "random", "gen")
new_lines <- names(rep_gen)[rep_gen == 1]
u1 <- setNames(b1$blup, b1$level)[new_lines]
ub <- setNames(bb$blup, bb$level)[new_lines]
cor(u1, ub, method = "spearman")
```

```
## [1] 0.8612
```

```
k <- round(0.1 * length(new_lines))
sel1 <- names(sort(u1, decreasing = TRUE))[1:k]
selb <- names(sort(ub, decreasing = TRUE))[1:k]
c(selected = k, in_common = length(intersect(sel1, selb)))
```

```
##  selected in_common
##        52        32
```

```
only_one <- setdiff(union(sel1, selb), intersect(sel1, selb))
plot(u1, ub, pch = 20, col = ifelse(new_lines %in% only_one, "red", "grey40"),
     xlab = "BLUP, independent residuals", ylab = "BLUP, spatial model")
abline(0, 1, lty = 2)
```



BLUPs das novas linhagens com resíduos independentes versus o modelo espacial. Os pontos vermelhos são linhagens selecionadas (10 % superiores) por apenas um dos dois modelos.

Uma parte importante das linhagens selecionadas muda com o modelo. Como o modelo espacial descreve melhor os dados, suas predições são mais confiáveis, o que também se reflete na confiabilidade média dos BLUPs e na herdabilidade generalizada:

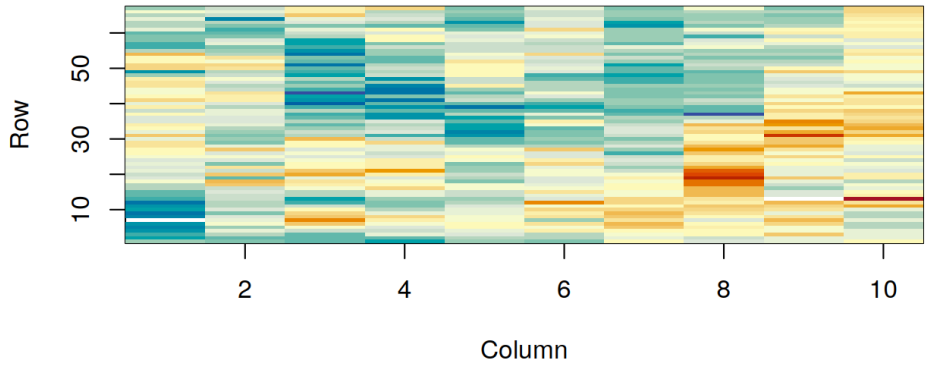
```
H2c <- function(f) {
  pr <- predict(f, "gen", sed = TRUE)
  1 - pr$avsed^2 / (2 * f$theta[1])
}
data.frame(model = c("independent", "spatial"),
           mean_reliability = c(mean(b1$reliability), mean(bb$reliability)),
           H2_Cullis = c(H2c(e1), H2c(eb)))

##         model mean_reliability H2_Cullis
## 1 independent           0.4025    0.4035
## 2     spatial           0.5767    0.5779
```

12.4 Tendência espacial e testemunhas

A variação espacial estimada pode ser visualizada com os resíduos do modelo após a remoção dos efeitos fixos e genéticos. As testemunhas repetidas, distribuídas por todo o ensaio, são o que a torna estimável.

```
gefe <- bb$blup[match(as.character(eb$data$gen), bb$level)]
esp <- eb$y - eb$fixed_fitted - gefe
Em <- tapply(esp, list(eb$data$col, eb$data$row), mean)
image(1:10, 1:67, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Column", ylab =
"Row")
```



Componente espacial estimado (produtividade menos os efeitos fixos e genéticos).

Este exemplo mostra o fluxo de trabalho recomendado por Cullis et al. (2006) para ensaios sem repetições: modelar a variação espacial e os fatores estranhos medidos, tratar os genótipos como aleatórios e basear a seleção nos BLUPs do modelo de melhor ajuste.

13 Uso avançado e solução de problemas

Este capítulo reúne técnicas que não pertencem a um tipo particular de experimento: como inspecionar a convergência, como usar valores iniciais e parâmetros fixos, como obter intervalos de confiança baseados na verossimilhança, o que fazer com estimativas na fronteira e matrizes mal condicionadas, e como reduzir o uso de memória. Termina com uma tabela de problemas comuns e suas soluções.

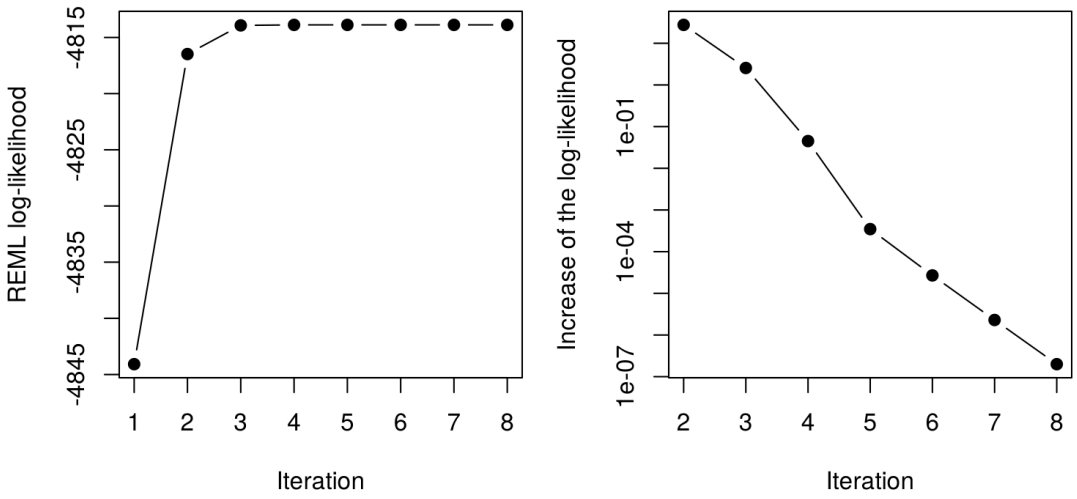
13.1 Histórico de iterações

O elemento `history` do objeto ajustado armazena a log-verossimilhança de cada iteração e o fator de amortecimento λ usado. Um λ igual a zero indica um passo AI puro; valores positivos indicam que o passo puro não era válido ou não melhorou a verossimilhança e teve de ser encurtado.

```
d <- agridat::besag.met
fh <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE, maxit =
100)
head(fh$history, 12)

##   iteration loglik lambda
## 1         1  -4844      0
## 2         2  -4816      0
## 3         3  -4814      0
## 4         4  -4814      0
## 5         5  -4814      0
## 6         6  -4814      0
## 7         7  -4814      0
## 8         8  -4814      0
```

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(fh$history$iteration, fh$history$loglik, type = "b", pch = 19,
     xlab = "Iteration", ylab = "REML log-likelihood")
inc <- diff(fh$history$loglik)
plot(seq_along(inc) + 1, pmax(inc, 1e-8), log = "y", type = "b", pch = 19,
     xlab = "Iteration", ylab = "Increase of the log-likelihood")
```



Histórico da log-verossimilhança REML (esquerda) e aumento da verossimilhança por iteração em escala logarítmica (direita) para o modelo FA(1).

```
par(op)
```

Perto do ótimo, o algoritmo AI converge de forma quase quadrática: o aumento da verossimilhança cai várias ordens de grandeza por iteração. Embora três variâncias específicas deste modelo estejam na fronteira, o tratamento da fronteira por conjunto ativo (Seção 2.4.1) impede que elas bloqueiem o passo conjunto, e o modelo converge em 8 iterações sem amortecimento. Uma diminuição lenta e constante dos aumentos indica uma verossimilhança plana em alguma direção.

13.2 Valores iniciais e reinícios

Quando um modelo complexo não converge, é aconselhável reiniciá-lo a partir dos valores finais da tentativa anterior ou das estimativas de um modelo mais simples. Os valores iniciais são fornecidos como um vetor nomeado:

```
ini <- fh$theta                                  # FA(1) estimates
fh2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
              residual = ~ diag(county):units, data = d, trace = FALSE, start =
ini)
c(iterations = fh2$iterations, logLik = fh2$loglik, converged = fh2$converged)

## iterations     logLik  converged
##          1      -4814          1
```

Começando no ótimo, o algoritmo confirma a convergência em uma ou duas iterações. Essa também é a forma de verificar se um ajuste realmente convergiu.

Para usar um modelo simples como ponto de partida de um complexo, os parâmetros comuns são copiados. Por exemplo, as variâncias de um modelo diagonal servem como variâncias específicas iniciais de um modelo FA:

```
fd <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE)
vg <- fd$theta[grep("diag\\(county\\):id\\(gen\\)", names(fd$theta))]
names(vg) <- sub("diag\\(county\\):id\\(gen\\)!county_", "fa(county,
1):id(gen)!county_var_", names(vg))
head(vg, 3)
```

```
## fa(county, 1):id(gen)!county_var_C1 fa(county, 1):id(gen)!county_var_C2
##                               57.41                               19.29
## fa(county, 1):id(gen)!county_var_C3
##                               94.35
```

Esse vetor pode ser passado a `start` (apenas para os parâmetros a serem inicializados; os demais assumem seus valores padrão).

13.3 Parâmetros fixos e verossimilhança perfilada

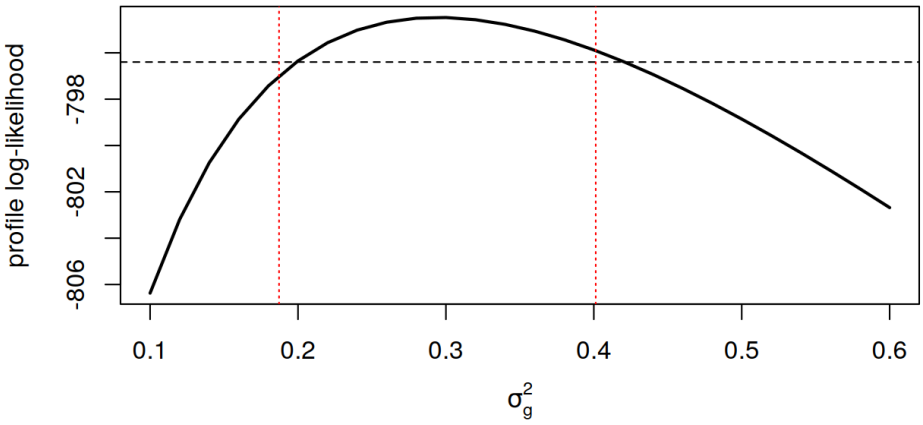
Com `fix`, os parâmetros são mantidos em seus valores iniciais. Uma aplicação importante é a **verossimilhança perfilada**: um parâmetro é fixado em uma grade de valores, os demais são estimados e a verossimilhança maximizada resultante é representada graficamente. O intervalo de confiança de 95 % baseado na verossimilhança contém os valores cuja log-verossimilhança perfilada está a menos de $\chi^2_{1,0.95}/2 = 1.92$ unidades do máximo. Ao contrário do intervalo de Wald (estimativa ± 1.96 erros padrão), ele não é necessariamente simétrico e nunca inclui valores negativos de uma variância.

Usamos o modelo GBLUP do Capítulo 7 para a variância genética do ambiente E1:

```
f0 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, trace = FALSE)
nm <- "vm(gen, G)!sigma2"
grid <- seq(0.10, 0.60, by = 0.02)
prof <- sapply(grid, function(s) {
  mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = setNames(s, nm), fix
= nm,
         trace = FALSE, pev = FALSE)$loglik
})
lim <- range(grid[prof >= max(prof) - qchisq(0.95, 1) / 2])
est <- f0$theta[nm]; se <- f0$varcomp[nm, "std.error"]
rbind(profile = lim, wald = est + c(-1, 1) * 1.96 * se)
```

```
##           [,1]   [,2]
## profile 0.2000 0.4200
## wald    0.1872 0.4013
```

```
plot(grid, prof, type = "l", lwd = 2, xlab = expression(sigma[g]^2), ylab =
"profile log-likelihood")
abline(h = max(prof) - qchisq(0.95, 1) / 2, lty = 2)
abline(v = est + c(-1, 1) * 1.96 * se, lty = 3, col = "red")
```



Log-verossimilhança perfilada da variância genética (GBLUP, ambiente E1). A linha horizontal marca o limiar do intervalo de 95 %; as linhas verticais, o intervalo de Wald.

O mesmo procedimento funciona para qualquer parâmetro, incluindo correlações genéticas e parâmetros autorregressivos, e é a forma recomendada de obter intervalos para parâmetros próximos da fronteira.

13.4 Componentes de variância conhecidos

Se os componentes de variância forem conhecidos (por exemplo, estimados em um grande conjunto de dados ou publicados), todos podem ser fixados para obter apenas os BLUEs e BLUPs. Novas gerações podem assim ser preditas com parâmetros estáveis sem reestimá-los a cada vez:

```
known_vc <- c("vm(gen, G)!sigma2" = 0.30, "R!units!sigma2" = 0.55)
fk <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = known_vc,
             fix = names(known_vc), trace = FALSE)
summary(fk)$varcomp
```

```
##                   estimate std.error z.ratio bound
## vm(gen, G)!sigma2     0.30        NA      NA     F
## R!units!sigma2        0.55        NA      NA     F
```

```
cor(fk$blups[[1]]$blup, f0$blups[[1]]$blup)
```

```
## [1] 1
```

13.5 Efeitos fixos aliased

Colunas da matriz de efeitos fixos que são combinações lineares de outras são removidas automaticamente, como os efeitos aliased do ASReml. Por exemplo, se o mesmo fator for incluído duas vezes com nomes diferentes, ou um fator que é função de outro (aqui, um agrupamento das repetições em dois campos), as colunas redundantes são descartadas:

```
ja <- agridat::john.alpha
ja$field <- factor(ifelse(ja$rep == "R1", "A", "B"))      # a function of rep
fa_alias <- mlreml(yield ~ rep + field, random = ~ gen, data = ja, trace =
FALSE)
```

```
c(possible_columns = ncol(model.matrix(~ rep + field, ja)), estimable_columns =
fa_alias$p)
```

```
##  possible_columns estimable_columns
##                 4                 3
```

```
colnames(fa_alias$X)
```

```
## [1] "(Intercept)" "repR2"       "repR3"
```

13.6 Parâmetros na fronteira

Uma variância estimada em seu limite inferior (código `B`) significa que a verossimilhança é maximizada na borda do espaço paramétrico. As causas habituais são que o efeito é realmente pequeno, que o delineamento não permite separá-lo de outros efeitos ou que o modelo está sobreparametrizado. Nesses casos:

1. o erro padrão relatado não é válido;
2. o teste da razão de verossimilhanças deve usar a mistura de distribuições χ^2 (`lrt(..., boundary = TRUE)`);
3. considere remover o termo ou substituir uma estrutura complexa (`us`) por uma mais parcimoniosa (`fa`, `diag`).

Uma variância em seu limite cujo escore é negativo é mantida no limite e removida do sistema AI (conjunto ativo), de modo que os demais parâmetros convergem para seu ótimo dada a fronteira; uma matriz não estruturada que converge para uma matriz singular (correlações de ± 1) interrompe o algoritmo com um aviso quando os passos se tornam desprezíveis.

Em modelos fator-analíticos, variâncias específicas iguais a zero (casos de Heywood) são frequentes e não impedem a interpretação do modelo.

13.7 Matrizes de parentesco mal condicionadas

O motor denso do MLREML não inverte as matrizes de parentesco, de modo que uma matriz singular é aceitável desde que $\mathbf{V}$ seja positiva definida; o motor esparso precisa da inversa e, portanto, de uma matriz positiva definida. No entanto, uma matriz com autovalores negativos (por exemplo, uma matriz $\mathbf{G}$ calculada com dados ausentes imputados, ou uma $\mathbf{H}$ mal construída) não é uma matriz de covariância válida. A função `bend()` a corrige substituindo os autovalores pequenos ou negativos:

```
set.seed(1)
Kbad <- cor(matrix(rnorm(30 * 20), 20))          # 30 x 30 with rank 20
Kbad[1, 2] <- Kbad[2, 1] <- 0.99                  # perturbation that makes it
indefinite
range(eigen(Kbad, only.values = TRUE)$values)
```

```
## [1] -0.7215  4.4667
```

```
Kok <- bend(Kbad, tol = 1e-4)
range(eigen(Kok, only.values = TRUE)$values)
```

```
## [1] 0.0004467 4.4667035
```

```
max(abs(Kok - Kbad))
```

```
## [1] 0.284
```

13.8 Escala das variáveis

O algoritmo é invariante à escala em teoria, mas na prática a convergência é mais robusta quando as respostas têm variâncias da ordem de um. Em modelos multivariados com caracteres em escalas muito diferentes (por exemplo, quilogramas de leite e porcentagem de proteína), é aconselhável reescalonar cada caráter, como foi feito no Capítulo 8 ao expressar o leite em toneladas e a gordura em centenas de quilogramas. As covariâncias na escala original são recuperadas multiplicando pelos fatores de escala.

13.9 Memória e tempo

- `engine = "sparse"` (Seção 2.12) evita completamente as matrizes $n \times n$; com mais de 5.000 registros ele é escolhido automaticamente quando o modelo é suportado.
- `pev = FALSE` omite o cálculo das variâncias do erro de predição, que é a parte mais custosa após o ajuste quando um termo tem muitos níveis (por exemplo, todos os animais do pedigree).
- `store = FALSE` não mantém as matrizes $n \times n$ necessárias para `predict()` e para o ajuste de Kenward–Roger de `wald()` (com o motor esparso, o fator de Cholesky); isso reduz muito o tamanho do objeto ajustado.
- Em uma análise longa (validação cruzada, simulações), remova os objetos ajustados que não são mais necessários e chame `gc()`.

13.10 Problemas comuns

Problemas comuns e soluções.

Mensagem ou sintoma	Causa provável	Solução
`levels of 'x' missing from the vm() matrix`	Alguns níveis do fator não estão nos nomes de linhas da matriz.	Verifique os identificadores (espaços, maiúsculas/minúsculas, prefixos); use `setdiff()` para encontrá-los.
`the matrix given to vm() must have rownames`	A matriz de parentesco não tem nomes.	Atribua `dimnames(K) <- list(ids, ids)`.
`term ... has more than one variance-type component`	Duas estruturas de variância em um termo, p. ex. `us(env):diag(gen)`.	Mantenha apenas uma estrutura de variância; a outra deve ser de correlação (`id`, `vm`, `ar1`...).
`AI-REML did not converge`	Verossimilhança plana, modelo sobreparametrizado ou valores iniciais ruins.	Aumente `maxit`, reinicie com `start = fit$theta`, simplifique o modelo.
`no improving step found`	O algoritmo não encontra um passo que aumente a verossimilhança, geralmente na fronteira.	Verifique os parâmetros com código `B`; tente outros valores iniciais.
`AI-REML stopped because the steps became negligible although the score is not zero`	Uma matriz não estruturada (`us`, `str()`) converge para uma matriz singular.	Verifique as correlações (± 1); use `fa`, `diag` ou `str(..., cov = "diag")`.
Variância com código `B` e erro padrão enorme	Parâmetro na fronteira.	Interprete como zero; use o LRT com mistura; considere remover o termo.

Mensagem ou sintoma	Causa provável	Solução
Correlação genética igual a ± 1	A matriz `us` está na fronteira do cone semidefinido.	Normal com poucos dados; considere um modelo `fa`.
Erro em `chol()` ou memória insuficiente	n grande demais para matrizes densas.	Use `engine = "sparse"` (Seção 2.12) ou veja as estratégias no Capítulo 15.
`the sparse engine does not support this model (...)`	O modelo tem uma estrutura sem implementação esparsa (`cs`, `ar1`, `expc`, `sexp`, `fa`) ou um resíduo não suportado.	Use `engine = "dense"`, ou uma estrutura suportada.
`the relationship matrix is not positive definite; use bend() or give its inverse`	Motor esparso com uma matriz de parentesco singular.	Corrija (bending) ou combine a matriz (Seção 13.7), forneça sua inversa ou use o motor denso.
`REML likelihoods are only comparable with identical fixed effects`	Modelos com efeitos fixos diferentes foram comparados com `lrt()`.	Compare apenas modelos com a mesma parte fixa; use testes de Wald para os efeitos fixos.

14 Estudo de simulação: propriedades dos estimadores

A comparação com outros softwares (Capítulo 15) verifica que o MLREML calcula corretamente a verossimilhança e seu máximo. Uma questão diferente é quão bons são os estimadores REML, seus erros padrão e os testes em amostras do tamanho usado na prática. Este capítulo responde com duas pequenas simulações que o leitor pode repetir e modificar.

14.1 Delineamento em blocos completos com genótipos aleatórios

Simulamos 200 experimentos com 40 genótipos em 3 blocos completos, com $\sigma^2_g = 1$, $\sigma^2_e = 2$ e efeitos fixos de bloco. A herdabilidade em base de média de genótipos é $H^2 = \sigma^2_g / \left(\sigma^2_g + \sigma^2_e/3\right) = 0.6$.

```
sim_rcbd <- function(seed, ngen = 40, nblk = 3, sg2 = 1, se2 = 2) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:ngen), blk = factor(1:nblk))
  dd$y <- c(0, 0.5, -0.5)[dd$blk] + rnorm(ngen, sd = sqrt(sg2))[dd$gen] +
    rnorm(nrow(dd), sd = sqrt(se2))
  f <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  h <- vpredict(f, H2 ~ V1 / (V1 + V2 / 3))
  c(sg2 = f$theta[[1]], se2 = f$theta[[2]], se_sg2 = f$varcomp$std.error[1],
    H2 = h$estimate, se_H2 = h$std.error, bound = f$varcomp$bound[1] == "B",
    conv = f$converged)
}
res1 <- as.data.frame(t(sapply(1:200, sim_rcbd)))
c(converged = mean(res1$conv), on_boundary = mean(res1$bound))

##   converged on_boundary
##           1           0
```

```
truth <- c(sg2 = 1, se2 = 2, H2 = 0.6)
tab1 <- data.frame(parameter = names(truth), true = truth,
                   mean = colMeans(res1[names(truth)]),
                   relative_bias_pct = 100 * (colMeans(res1[names(truth)]) -
truth) / truth,
                   empirical_sd = sapply(res1[names(truth)], sd))
tab1$mean_se <- c(mean(res1$se_sg2), NA, mean(res1$se_H2))
knitr::kable(tab1, digits = 3, row.names = FALSE,
             caption = "Simulation of 200 complete block experiments: mean,
bias and variability of the REML estimators.")
```

Simulação de 200 experimentos em blocos completos: média, viés e variabilidade dos estimadores REML.

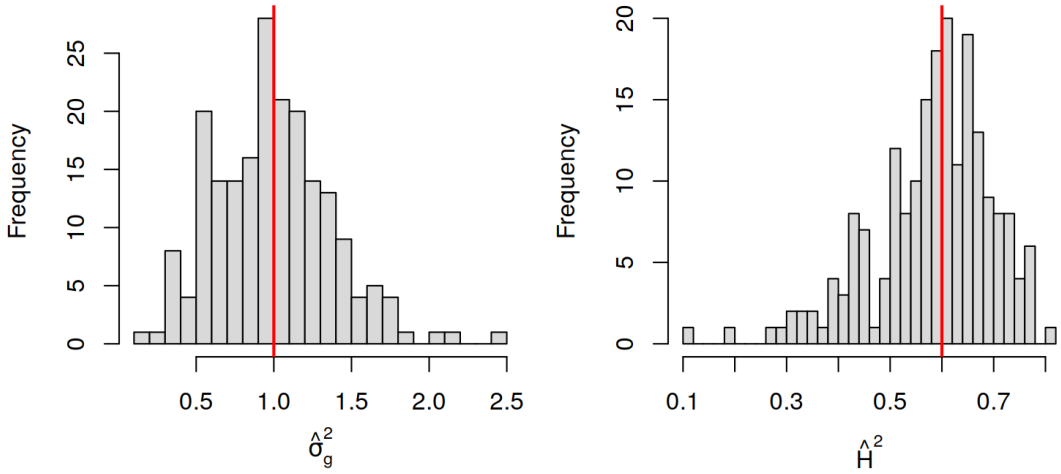
parameter	true	mean	relative_bias_pct	empirical_sd	mean_se
sg2	1.0	0.997	-0.292	0.378	0.392
se2	2.0	1.990	-0.511	0.326	NA
H2	0.6	0.582	-3.061	0.116	0.116

```
cov95 <- function(est, se, v) mean(est - 1.96 * se <= v & v <= est + 1.96 * se)
c(coverage_sg2 = cov95(res1$sg2, res1$se_sg2, 1), coverage_H2 = cov95(res1$H2,
res1$se_H2, 0.6))

## coverage_sg2  coverage_H2
##         0.95         0.94
```

Os estimadores REML têm um viés pequeno, e o erro padrão médio obtido da matriz de informação média é próximo do desvio padrão empírico das estimativas, indicando que a aproximação assintótica é razoável com 40 genótipos. A cobertura dos intervalos de Wald é próxima dos 95 % nominais. A distribuição de $\hat{\sigma}^2_g$ é assimétrica à direita, como esperado para um estimador de variância:

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
hist(res1$sg2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(sigma)[g]^2))
abline(v = 1, col = "red", lwd = 2)
hist(res1$H2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(H)^2))
abline(v = 0.6, col = "red", lwd = 2)
```



Distribuição das estimativas REML da variância genética (esquerda) e da herdabilidade (direita) em 200 experimentos simulados. As linhas verticais marcam os valores verdadeiros.

```
par(op)
```

14.2 Tamanho do teste da razão de verossimilhanças na fronteira

Para verificar a correção de fronteira de `lrt()`, simulamos 200 experimentos **sem** variância genética ($\sigma^2_g = 0$) e registramos a proporção de rejeições de $H_0: \sigma^2_g = 0$ a 5 %, com a χ^2_1 ingênua e com a mistura 50:50.

```
sim_null <- function(seed) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:40), blk = factor(1:3))
  dd$y <- rnorm(nrow(dd), sd = sqrt(2))
  f1 <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  f0 <- mlreml(y ~ blk, data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(mixture = lrt(f0, f1, boundary = TRUE)$p.value, naive = lrt(f0, f1,
boundary = FALSE)$p.value,
    boundary = f1$varcomp$bound[1] == "B")
}
pv <- as.data.frame(t(sapply(1:200, sim_null)))
colMeans(pv[c("mixture", "naive")] < 0.05)

## mixture   naive
##   0.020   0.015

mean(pv$boundary)

## [1] 0.49
```

Em 49 % das simulações, a variância genética é estimada na fronteira, perto da metade prevista pela teoria assintótica. Nessas simulações, a estatística é zero, de modo que ambas as versões do teste são conservadoras com esse tamanho de amostra: a χ^2_1 ingênua rejeitou em 1,5 % dos casos e a mistura 50:50 em 2 %, mais próxima dos 5 % nominais. A mistura é a aproximação assintótica correta; com poucos níveis do fator aleatório, o teste permanece um tanto conservador, de modo que um valor p significativo é evidência confiável de variância genética.

14.3 Modelo espacial

A segunda simulação avalia o modelo AR1 × AR1 em uma grade de 16 linhas por 12 colunas com 48 genótipos em 4 repetições, com $\rho_r = 0.7$, $\rho_c = 0.4$, variância espacial 1, variância genética 0,5 e nugget 0,2.

```
sim_sp <- function(seed, rr = 0.7, rc = 0.4) {
  set.seed(seed)
  dd <- expand.grid(row = 1:16, col = 1:12)
  dd$rep <- factor((dd$col - 1) %/% 3 + 1)
  dd$gen <- factor(unlist(lapply(1:4, function(r) sample(1:48))))
  Rr <- rr^abs(outer(1:16, 1:16, "-")); Rc <- rc^abs(outer(1:12, 1:12, "-"))
  xi <- drop(t(chol(kronecker(Rc, Rr))) %*% rnorm(192))       # rows vary
fastest
  dd$y <- rnorm(48, sd = sqrt(0.5))[dd$gen] + xi + rnorm(192, sd = sqrt(0.2))
  f <- mlreml(y ~ rep, random = ~ gen + units, residual = ~ ar1(row):ar1(col),
              data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(f$theta, conv = f$converged)
}
res3 <- as.data.frame(t(sapply(1:40, sim_sp)))
names(res3) <- c("sg2", "nugget", "s2_spatial", "rho_row", "rho_col", "conv")
truth3 <- c(sg2 = 0.5, nugget = 0.2, s2_spatial = 1, rho_row = 0.7, rho_col =
0.4)
data.frame(true = truth3, median = sapply(res3[names(truth3)], median),
           mean = colMeans(res3[names(truth3)]), sd =
sapply(res3[names(truth3)], sd))
```

```
##            true median   mean      sd
## sg2         0.5 0.5232 0.5091 0.12644
## nugget      0.2 0.1568 0.1723 0.11259
## s2_spatial  1.0 1.0209 1.0445 0.28329
## rho_row     0.7 0.6573 0.6636 0.09491
## rho_col     0.4 0.3723 0.4015 0.14507
## [1] 0.95
```

```
mean(res3$conv)
```

```
## [1] 0.95
```

95 % dos ajustes convergiram. Os parâmetros autorregressivos e a variância genética são bem recuperados, com viés pequeno. Separar o nugget da variância espacial é mais difícil, como é bem conhecido em geoestatística: com uma grade desse tamanho, ambos os componentes têm considerável variabilidade amostral, e sua soma é estimada com muito mais precisão do que cada um separadamente.

```
c(mean_sum = mean(res3$nugget + res3$s2_spatial), sd_sum = sd(res3$nugget +
res3$s2_spatial))
```

```
## mean_sum   sd_sum
##    1.217    0.290
```

Essas simulações são deliberadamente pequenas para que sejam executadas em poucos minutos. Para um estudo formal, aumente o número de réplicas para 1.000 ou mais e explore os tamanhos de experimento e os valores de parâmetros de interesse.

15 Validação numérica, desempenho e limitações

15.1 Comparação com outros softwares

Um programa de estimação deve fornecer os mesmos resultados que softwares consolidados quando ambos ajustam o mesmo modelo. A tabela a seguir compara a log-verossimilhança REML do MLREML com a de `lme4` e `nlme` para todos os modelos deste guia que esses pacotes conseguem ajustar, e a função `gblup()` com o motor geral. Os testes automáticos do pacote (`tests/testthat`) repetem essas comparações toda vez que o pacote é verificado.

```
library(lme4); library(nlme)
v <- list()
d <- agridat::john.alpha
f <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace = FALSE)
v[[1]] <- c("Alpha lattice, two variances", "lme4", f$loglik,
            logLik(lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)))
f <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy, trace = FALSE)
v[[2]] <- c("Random regression (us)", "lme4", f$loglik,
            logLik(lmer(Reaction ~ Days + (Days | Subject), sleepstudy)))
f <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units, data = ort,
            trace = FALSE)
v[[3]] <- c("Heterogeneous residual (diag)", "nlme::lme", f$loglik,
            logLik(lme(distance ~ age * Sex, random = ~ 1 | Subject,
                       weights = varIdent(form = ~ 1 | Sex), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ ar1(age):id(Subject), data =
ort, trace = FALSE)
v[[4]] <- c("Autoregressive residual (ar1)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corExp(form = ~ age
| Subject), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ us(age):id(Subject), data = ort,
trace = FALSE)
v[[5]] <- c("Unstructured residual (us)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corSymm(form = ~ 1 |
Subject),
                       weights = varIdent(form = ~ 1 | age), data = ort)))
v[[6]] <- c("GBLUP (vm)", "spectral gblup()", f1$loglik, g1$loglik)
tab <- as.data.frame(do.call(rbind, v))
names(tab) <- c("model", "reference", "logLik_MLREML", "logLik_reference")
tab$logLik_MLREML <- as.numeric(tab$logLik_MLREML)
tab$logLik_reference <- as.numeric(tab$logLik_reference)
tab$difference <- signif(tab$logLik_MLREML - tab$logLik_reference, 2)
knitr::kable(tab, digits = 4, caption = "REML log-likelihood of MLREML and of
other software.")
```

Log-verossimilhança REML do MLREML e de outros softwares.

model	reference	logLik_MLREML	logLik_reference	difference
Alfa-látice, duas variâncias	lme4	-46.6	-46.6	0
Regressão aleatória (us)	lme4	-871.8	-871.8	0
Resíduo heterogêneo (diag)	nlme::lme	-207.6	-207.6	0
Resíduo autorregressivo	nlme::gls	-222.3	-222.3	0

model	reference	logLik_MLREML	logLik_reference	difference
(ar1)				
Resíduo não estruturado (us)	nlme::gls	-212.3	-212.3	0
GBLUP (vm)	gblup() espectral	-794.5	-794.5	0

As diferenças são da ordem da tolerância de convergência. Componentes de variância, efeitos fixos, BLUPs e SEDs também concordam (Capítulos 4 e 10). Para estruturas que nenhum pacote R de uso geral ajusta (AR1 × AR1 com nugget, fator-analítico, termos genéticos com estrutura não estruturada entre ambientes), o teste automático verifica que o ponto encontrado é um máximo: perturbar qualquer parâmetro reduz a verossimilhança.

As ferramentas de pedigree são comparadas com `pedigreemm`:

```
Fp <- pedigreemm::inbreeding(pedCowsR)
c(max_diff_inbreeding = max(abs(Fp - Fi[p@label])))

## max_diff_inbreeding
##                   0

s <- sample(p@label, 300)
A <- Amatrix(pp)
Ap <- as.matrix(pedigreemm::getA(pedCowsR))[s, s]
c(max_diff_A = max(abs(Ap - A[s, s])))

## max_diff_A
##   1.11e-16

rm(A); invisible(gc())
```

Comparações adicionais. O código a seguir verifica outras capacidades: (1) a correlação exponencial com distâncias menores que um frente a `nlme::gls()`, com a qual o MLREML atinge o mesmo máximo; (2) as estatísticas F de Kenward–Roger e os graus de liberdade do denominador das hipóteses testadas por `wald()` frente às do `pbkrtest`; e (3) o motor esparso frente ao motor denso.

```
## 1. exponential correlation with distances smaller than one
set.seed(7)
dx <- data.frame(t = rep(c(0, 0.3, 0.7, 1.2, 2), 20), id = factor(rep(1:20,
each = 5)))
dx$y <- 0.5 * dx$t + unlist(lapply(1:20, function(i) cumsum(rnorm(5)) / 2))
fx <- mlreml(y ~ t, residual = ~ expc(t):id(id), data = dx, trace = FALSE)
gx <- gls(y ~ t, correlation = corExp(form = ~ t | id), data = dx)
c(MLREML = fx$loglik, nlme = as.numeric(logLik(gx)))

## MLREML   nlme
## -84.21 -84.21

c(rho_MLREML = unname(fx$theta[2]), rho_nlme = unname(exp(-1 /
coef(gx$modelStruct$corStruct, FALSE))))

## rho_MLREML   rho_nlme
##     0.7511     0.7511

## 2. Kenward-Roger F tests against pbkrtest
kr_row <- function(label, f, l, term) {
```

```
  j <- which(f$assign == match(term, attr(f$terms_fixed, "term.labels")))
  L <- chol(solve(vcov(f)))[j, , drop = FALSE]        # incremental hypothesis
of wald()
  w <- wald(f, ddf = "KR"); w <- w[w$term == term, ]
  pk <- KRmodcomp(l, L)$stats
  data.frame(model = label, term = term, F_MLREML = w$F.inc, F_pbkrtest =
pk$Fstat,
             ddf_MLREML = w$ddf.inc, ddf_pbkrtest = pk$ddf)
}
ja <- agridat::john.alpha
oats <- as.data.frame(nlme::Oats)
oats$nitro <- factor(oats$nitro)
oats$Block <- factor(as.character(oats$Block)); oats$Variety <-
factor(as.character(oats$Variety))
f1 <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = ja, trace = FALSE,
tol = 1e-10)
l1 <- lmer(yield ~ rep + gen + (1 | rep:block), data = ja)
f2 <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE, tol = 1e-10)
l2 <- lmer(Reaction ~ Days + (Days | Subject), sleepstudy)
f3 <- mlreml(yield ~ Variety * nitro, random = ~ Block + Block:Variety, data =
oats,
             trace = FALSE, tol = 1e-10)
l3 <- lmer(yield ~ Variety * nitro + (1 | Block) + (1 | Block:Variety), oats)
kr <- rbind(kr_row("Alpha lattice", f1, l1, "rep"), kr_row("Alpha lattice", f1,
l1, "gen"),
            kr_row("Random regression", f2, l2, "Days"),
            kr_row("Split plot (Oats)", f3, l3, "Variety"), kr_row("Split plot
(Oats)", f3, l3, "nitro"),
            kr_row("Split plot (Oats)", f3, l3, "Variety:nitro"))
knitr::kable(kr, digits = 4, row.names = FALSE,
             caption = "Kenward-Roger F statistics and denominator degrees of
freedom of MLREML and pbkrtest.")
```

Estatísticas F de Kenward-Roger e graus de liberdade do denominador do MLREML e do pbkrtest.

Modelo	term	F_MLREML	F_pbkrtest	ddf_MLREML	ddf_pbkrtest
Alfa-látice	rep	9.2124	9.2124	11.52	11.52
Alfa-látice	gen	5.3628	5.3628	35.50	35.50
Regressão aleatória	Days	45.8530	45.8530	17.00	17.00
Parcelas subdivididas (Oats)	Variety	1.4853	1.4853	10.00	10.00
Parcelas subdivididas (Oats)	nitro	37.6856	37.6857	45.00	45.00
Parcelas subdivididas (Oats)	Variety:nitro	0.3028	0.3028	45.00	45.00

As estatísticas F e os graus de liberdade do denominador coincidem com os do `pbkrtest` em cinco ou mais dígitos significativos. No delineamento de parcelas subdivididas o método reproduz os graus de liberdade clássicos dos erros da parcela principal (10) e da subparcela (45), e $\lambda = 1$.

```
## 3. sparse engine against the dense engine
both <- function(label, ...) {
  fd <- mlreml(..., trace = FALSE, engine = "dense")
  fs <- mlreml(..., trace = FALSE, engine = "sparse")
  bd <- do.call(rbind, fd$blups); bs <- do.call(rbind, fs$blups)
  f <- function(x) sprintf("%.1e", x)
  data.frame(model = label, logLik = round(fd$loglik, 2), logLik_diff =
f(fd$loglik - fs$loglik),
             theta_reldiff = f(max(abs(fd$theta - fs$theta) /
pmax(abs(fd$theta), 1e-8))),
             blup_diff = f(max(abs(bd$blup - bs$blup))), pev_diff =
f(max(abs(bd$pev - bs$pev))))
}
sp <- rbind(
  both("Alpha lattice (two variances)", yield ~ rep, random = ~ gen +
rep:block, data = ja),
  both("Random regression us(leg)", Reaction ~ Days, random = ~ us(leg(Days,
1)):id(Subject),
       data = sleepstudy),
  both("Heterogeneous residual diag(Sex):units", distance ~ age * Sex, random =
~ Subject,
       residual = ~ diag(Sex):units, data = ort),
  both("Split plot (two variances)", yield ~ Variety * nitro, random = ~ Block
+ Block:Variety,
       data = oats))
knitr::kable(sp, row.names = FALSE, align = "lrrrrr",
             caption = "Differences between the dense and the sparse engines
(log-likelihood, maximum relative difference of the variance parameters,
maximum absolute differences of BLUPs and PEVs).")
```

Diferenças entre os motores denso e esparso (log-verossimilhança, máxima diferença relativa dos parâmetros de variância, máximas diferenças absolutas dos BLUPs e das PEVs).

Modelo	logLik	logLik_diff	theta_reldiff	blup_diff	pev_diff
Alfa-látice (duas variâncias)	-46.6	-1.8e-12	2.8e-15	6.7e-15	8.3e-17
Regressão aleatória us(leg)	-871.8	3.0e-12	1.3e-14	4.0e-13	3.7e-12
Resíduo heterogêneo diag(Sex):units	-207.6	6.2e-12	6.8e-15	1.8e-13	5.9e-15
Parcelas subdivididas (duas variâncias)	-264.5	-1.3e-12	1.2e-14	6.3e-13	7.1e-13

Os dois motores fornecem os mesmos resultados a menos de erros de arredondamento. Além disso, o termo direto–materno foi verificado frente a uma maximização direta da verossimilhança (2.6), com $\mathbf{V}$ construída a partir de (2.18) (as estimativas coincidem até a precisão do otimizador e os BLUPs e PEV até 10^{-13}), e os testes automáticos do pacote verificam que, com a covariância fixada em zero, ele reproduz o modelo com dois termos independentes `vm(animal, A) + vm(dam, A)`.

15.2 Tempo de computação e memória

O custo do motor denso do MLREML é dominado por operações com a matriz densa $n \times n$ $\mathbf{V}$. Para medi-lo, um modelo GBLUP foi ajustado a dados simulados de diferentes tamanhos no mesmo computador (um núcleo, OpenBLAS):

```r
times <- do.call(rbind, lapply(c(500, 1000, 2000, 3000), function(n) {
  set.seed(n)
  Msim <- matrix(rbinom(n * 1000, 2, 0.3), n); rownames(Msim) <- paste0("i",
1:n)
  Ks <- Gmatrix(Msim) + diag(1e-3, n)
  ds <- data.frame(id = rownames(Msim), y = drop(Msim %*% rnorm(1000, 0, 0.05))
+ rnorm(n))
  t1 <- system.time(fs <- mlreml(y ~ 1, random = ~ vm(id, Ks), data = ds, trace
= FALSE))[3]
  t2 <- system.time(gblup(setNames(ds$y, ds$id), Ks))[3]
  data.frame(n = n, iterations = fs$iterations, sec_mlreml = t1,
             sec_per_iteration = t1 / (fs$iterations + 2), sec_gblup = t2,
             memory_V_GB = round(3 * 8 * n^2 / 1e9, 2))
}))
knitr::kable(times, digits = 2, row.names = FALSE,
             caption = "Fitting time of a GBLUP model by number of
observations.")
```

Tempo de ajuste de um modelo GBLUP por número de observações.

n	iterations	sec_mlreml	sec_per_iteration	sec_gblup	memory_V_GB
500	4	0.12	0.02	0.04	0.01
1000	3	0.53	0.11	0.24	0.02
2000	4	4.05	0.68	1.47	0.10
3000	3	9.25	1.85	17.63	0.22

O tempo por iteração cresce aproximadamente com n^3 e a memória com n^2 (a coluna de memória estima três matrizes $n \times n$ de precisão dupla: $\mathbf{V}$, $\mathbf{P}$ e uma matriz de trabalho). Com 8 GB de memória, o MLREML ajusta confortavelmente modelos com cerca de 6.000 registros; com 16 GB, cerca de 9.000. Em um modelo com um único termo aleatório, o algoritmo AI converge em pouquíssimas iterações, de modo que `mlreml()` é tão rápido quanto `gblup()` ou mais rápido. O custo de `gblup()` é dominado pela decomposição espectral, que é várias vezes mais cara que uma fatoração de Cholesky; sua vantagem aparece quando o modelo geral precisa de muitas iterações ou quando a decomposição pode ser reutilizada.

O número de parâmetros tem influência linear: cada parâmetro requer formar sua matriz derivada e um traço, a um custo de $O(n^2)$. Modelos com muitos parâmetros (`us` com muitos ambientes, `fa` com vários fatores) também precisam de mais iterações.

Motor esparso. Para medir o motor esparso, foram simuladas populações do tipo leiteiro com dez gerações sobrepostas, touros de inseminação artificial usados em vários rebanhos, mães dentro de rebanhos, registros em todos os animais com pais conhecidos e grupos contemporâneos definidos por rebanho e ano (efeitos fixos, cerca de 20 registros cada). O modelo animal aditivo `y ~ hy`, `random = ~ vm(id, Ai)`, e o modelo direto–materno com `random = ~ str(id, dam, K = Ai)` foram ajustados no mesmo computador acima (um núcleo, OpenBLAS, 8 GB de memória), junto com um modelo sem pedigree para registros repetidos de vacas em rebanhos, `y ~ year`, `random = ~ herd + cow`, com quatro registros por vaca. O código é instalado com o pacote: `system.file("scale", "sparse_scale.R", package = "MLREML")`.

Motor esparso: tamanho do problema, tempo de computação e memória (elementos do fator em seu armazenamento supernodal, que inclui os zeros explícitos dos supernós).

Modelo	Registros	Animais ou vacas	Equações	Elementos do fator (milhões)	Iterações	Tempo (s)	Memória máxima (GB)
Modelo animal	23,750	25,000	26,234	1.5	4	3.6	0.34
Modelo animal	47,500	50,000	52,468	3.8	4	12.3	0.65
Modelo animal	95,000	100,000	104,908	10.7	4	20.1	0.77
Modelo animal	237,500	250,000	262,287	49.4	5	125.1	2.36
Direto–materno	23,750	25,000	51,234	4.0	8	20.0	0.54
Direto–materno	47,500	50,000	102,468	11.1	5	48.6	1.19
Direto–materno	95,000	100,000	204,908	34.4	8	134.5	2.15
Registros repetidos	1,000,000	250,000	250,358	3.0	5	26.5	1.43
Registros repetidos	2,000,000	500,000	500,691	6.0	7	63.2	2.56

As estimativas foram próximas dos valores simulados em todos os casos (por exemplo, 0,304 e 0,644 para as variâncias aditiva e residual do modelo animal com 100.000 registros, simuladas com 0,30 e 0,60 mais um componente materno que o modelo aditivo não inclui; e 2,00, 1,00 e 2,99 para as variâncias de rebanho, vaca e residual do modelo com um milhão de registros, simuladas com 2, 1 e 3). O tempo e a memória são governados pelo número de elementos não nulos do fator de Cholesky das equações do modelo misto, e não pelo número de registros. Nos modelos animais, o pedigree e os grupos contemporâneos ligam muitos animais e o fator se preenche: de 100.000 para 250.000 registros, seu tamanho foi multiplicado por 4,6 e o tempo por 6. Com 8 GB de memória, os maiores modelos animais que puderam ser ajustados com essa estrutura populacional foram o modelo aditivo com 250.000 registros e o modelo direto–materno com 100.000 registros; os tamanhos seguintes testados (500.000 e 250.000 registros) excederam a memória. Um modelo cujos efeitos aleatórios são aninhados, como registros repetidos de vacas dentro de rebanhos, tem um fator com poucos elementos não nulos, e o motor esparso o ajustou com um e dois milhões de registros em cerca de um minuto. Portanto, modelos animais com milhões de registros precisam de um computador com várias vezes mais memória do que o usado aqui ou, para avaliações nacionais de rotina, dos programas especializados com métodos de solução iterativos (Seção 15.4).

15.3 Estratégias para grandes conjuntos de dados

- **Motor esparso.** Com as estruturas que suporta, `engine = "sparse"` analisa todos os registros em uma única etapa (Seção 2.12); use `Ainverse()` para parentescos por pedigree, e lembre-se de que seu custo depende do preenchimento do fator de Cholesky, que cresce com o número de efeitos fixos e aleatórios conectados pelos registros.

- **Análise em dois estágios.** Primeiro, ajuste cada ensaio ou ambiente separadamente e depois analise as médias ajustadas (BLUEs) com seus pesos (Smith et al., 2001; Piepho et al., 2012). As médias reduzem o número de observações a uma por genótipo e ambiente.
- **Compressão de dados.** Resuma os indivíduos em grupos (por exemplo, por quantis do fenótipo) e ajuste um modelo ponderado às médias dos grupos.
- **Subamostras para estimar variâncias.** Estime os componentes em uma subamostra representativa e fixe-os (`start` e `fix`) para predizer o conjunto de dados completo com `gblup()`.
- **Mais memória e uma BLAS paralela.** Com OpenBLAS ou MKL multithread, o tempo por iteração diminui quase em proporção ao número de núcleos.

15.4 Limitações da versão 1.0.0

1. **Escala.** O motor denso está limitado a alguns milhares de registros. Em um computador com 8 GB de memória, o motor esparso ajusta modelos animais com algumas centenas de milhares de registros e modelos com efeitos aleatórios aninhados com milhões de registros (Seção 15.2), porque sua memória cresce com o preenchimento do fator de Cholesky; ele não pretende substituir os programas especializados (ASReml, BLUPF90, DMU, WOMBAT) em avaliações nacionais com dezenas de milhões de registros, métodos de solução por iteração sobre os dados ou modelos genômicos de passo único com centenas de milhares de animais genotipados. As estruturas com parâmetros de correlação (`cs`, `ar1`, `expc`, `sexp`) e as estruturas fator-analíticas estão disponíveis apenas no motor denso.
2. **Covariância entre termos.** Dois termos aleatórios só podem ser correlacionados por meio de `str()`, o que exige que seus fatores compartilhem os níveis da mesma matriz **K** (por exemplo, efeitos genéticos direto e materno); correlações entre termos com matrizes de parentesco diferentes ou com bases diferentes não estão disponíveis. Um intercepto e uma inclinação são correlacionados dentro de um único termo de regressão aleatória.
3. **Graus de liberdade.** Os graus de liberdade de Kenward–Roger estão disponíveis para o motor denso; o motor esparso usa $n - p$, e a aproximação de Satterthwaite não está implementada.
4. **Distribuições.** Apenas respostas normais são ajustadas; não há modelos lineares mistos generalizados.
5. **Estruturas.** Algumas estruturas específicas do ASReml não estão disponíveis, como `corgh` com variâncias separadas, ARMA, splines cúbicos aleatórios ou `at()` para termos condicionais, embora `diag()` e `us()` cubram a maioria dos usos de `at()`. A correlação dos modelos espaciais é comum a todos os ambientes quando se usa `diag(env):ar1(row):ar1(col)`.
6. **Erros padrão na fronteira.** Como no ASReml, os erros padrão dos parâmetros em seu limite não são válidos.

Essas limitações são as prioridades de desenvolvimento para versões futuras, em particular a extensão do motor esparso às estruturas com parâmetros de correlação e aos modelos fator-analíticos, e o ajuste de Kenward–Roger para o motor esparso.

16 Resumo das funções

A tabela a seguir resume todas as funções exportadas. A documentação completa de cada uma, com seus argumentos, valores e exemplos, está no manual de referência incluído como apêndice ao final deste documento, e também está disponível no R com `?function_name`.

Funções do MLREML.

Função	Finalidade
`mlreml()`	Ajusta um modelo linear misto por AI-REML (motor denso ou esparso).
`summary()`, `print()`	Componentes de variância, verossimilhança, AIC e BIC; efeitos fixos com erros padrão.
`coef()`	Efeitos fixos (`type = "fixed"`) ou BLUPs com PEV, erro padrão e confiabilidade (`type = "random"`).
`fitted()`, `residuals()`, `vcov()`, `logLik()`	Métodos padrão do R.
`wald()`	Testes de Wald incrementais e condicionais para termos fixos, com graus de liberdade do denominador de Kenward–Roger ou residuais.
`predict()`	Médias preditas, erros padrão e matriz de SED para uma classificação.
`vpredict()`	Funções dos parâmetros de variância com erros padrão pelo método delta.
`lrt()`	Teste da razão de verossimilhanças, com correção de fronteira.
`vc_matrix()`	Matriz de covariância e de correlação de um componente `diag`, `us` ou `fa` ou de um termo `str()`; cargas e variâncias específicas.
`variogram()`	Variograma amostral dos resíduos de um ensaio de campo.
`plot()`	Gráficos de diagnóstico e de convergência.
`pedigree_prepare()`	Limpa, completa e ordena um pedigree.
`inbreeding_coef()`	Coeficientes de endogamia (Meuwissen e Luo, 1992).
`Amatrix()`	Matriz densa de parentesco aditivo (método tabular).
`Ainverse()`	Inversa esparsa de **A** (regras de Henderson com endogamia), utilizável diretamente em `vm()` e `str()`.
`snp_qc()`	Controle de qualidade de marcadores (MAF, taxa de chamada por marcador e por indivíduo).
`Gmatrix()`	Matriz de parentesco genômico (VanRaden, Yang ou escalonada).
`align_G()`	Alinha **G** com $\mathbf{A}_{22}$.
`Hmatrix()`	Matriz de passo único **H** ou sua inversa.
`bend()`	Transforma uma matriz simétrica em uma positiva definida.
`gblup()`	GBLUP por rotação espectral.
`cv_gblup()`	Validação cruzada de k grupos (folds) do GBLUP.

16.1 Equivalências de sintaxe com o ASReml-R

Para usuários do ASReml-R, a tabela a seguir traduz as construções mais comuns.

Tradução da sintaxe do ASReml-R.

ASReml-R (versão 4)	MLREML
`asreml(fixed = y ~ rep, random = ~ gen, data = d)`	`mlreml(y ~ rep, random = ~ gen, data = d)`
`random = ~ vm(animal, ainv)`	`random = ~ vm(animal, A)` (matriz **A**) ou `random = ~ vm(animal, Ainverse(ped))`
`random = ~ fa(env, 2):id(gen)`	igual
`random = ~ us(env):vm(gen, Ginv)`	`random = ~ us(env):vm(gen, G)`
`residual = ~ dsum(~ units | env)`	`residual = ~ diag(env):units`
`residual = ~ ar1(row):ar1(col)`	igual
`random = ~ str(~ animal + dam, ~ us(2):vm(animal, ainv))`	`random = ~ str(animal, dam, K = A)`
`random = ~ leg(age, 2):id(animal)` com `us()`	`random = ~ us(leg(age, 2)):id(animal)`
`random = ~ and(male)` junto com `female`	`random = ~ mm(female, male)`
`fixed = cbind(y1, y2) ~ trait, residual = ~ id(units):us(trait)`	`cbind(y1, y2) ~ trait, residual = ~ us(trait):units`
`summary(fit)$varcomp`	`summary(fit)$varcomp`
`wald(fit)`	`wald(fit)` (graus de liberdade do denominador de Kenward–Roger por padrão)
`predict(fit, classify = "gen")`	`predict(fit, classify = "gen")`
`vpredict(fit, h2 ~ V1/(V1+V2))`	igual
`lrt(fit0, fit1)`	igual

A diferença mais importante é que `vm()` recebe a própria matriz de parentesco, porque o motor denso do MLREML trabalha com a forma marginal do modelo; sua inversa também pode ser fornecida quando tem o atributo `INVERSE = TRUE`, como o resultado de `Ainverse()`. Com o motor denso as matrizes podem ser singulares; o motor esparso precisa que sejam positivas definidas.

16.2 Guia rápido de modelos

Os modelos a seguir cobrem as análises mais comuns. Os nomes das variáveis são ilustrativos.

```
## Complete blocks, random genotypes
mlreml(y ~ rep, random = ~ gen, data = d)

## Alpha lattice or incomplete blocks, fixed genotypes
mlreml(y ~ rep + gen, random = ~ rep:block, data = d)

## Row x column spatial analysis with nugget and linear trend
mlreml(y ~ rep + row_num, random = ~ gen + units,
       residual = ~ ar1(row):ar1(column), data = d)

## Irregular spatial data (coordinates)
mlreml(y ~ 1, random = ~ gen + units, residual = ~ sexp(x, y), data = d)
```

```r
## MET: genetic and residual variances by environment
mlreml(y ~ env, random = ~ env:rep + diag(env):id(gen),
       residual = ~ diag(env):units, data = d)

## MET: two-factor factor analytic model
mlreml(y ~ env, random = ~ env:rep + fa(env, 2):id(gen),
       residual = ~ diag(env):units, data = d, maxit = 200)

## GBLUP and multi-environment GBLUP
mlreml(y ~ 1, random = ~ vm(gen, G), data = d)
mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~ diag(env):units,
data = d)

## Pedigree + markers (two kernels)
mlreml(y ~ 1, random = ~ vm(gen, A) + vm(gen, G), data = d)

## Single-step
H <- Hmatrix(A, G); mlreml(y ~ 1, random = ~ vm(id, H), data = d)

## Animal model, repeatability and maternal effect
mlreml(y ~ herd + sex, random = ~ vm(animal, A), data = d)
mlreml(y ~ herd + parity, random = ~ vm(animal, A) + id(animal), data = d)
mlreml(y ~ herd, random = ~ vm(animal, A) + id(dam), data = d)

## Direct-maternal animal model with maternal permanent environment
mlreml(y ~ herd + sex, random = ~ str(animal, dam, K = A) + id(dam), data = d)

## Bivariate with genetic and residual correlations
mlreml(cbind(y1, y2) ~ trait + trait:herd, random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = d)

## Repeated measures with autoregressive residual
mlreml(y ~ trt * time, random = ~ subject, residual = ~ ar1(time):id(subject),
data = d)

## Random regression
mlreml(y ~ group + age, random = ~ us(leg(age, 2)):vm(animal, A) +
       us(leg(age, 2)):id(animal), residual = ~ diag(age_class):units, data =
d)

## Diallel (GCA, SCA, maternal, reciprocal)
mlreml(y ~ block, random = ~ mm(female, male) + cross + female + cross_ord,
data = d)

## Known variance components (BLUE and BLUP only)
mlreml(y ~ 1, random = ~ vm(gen, G), data = d,
       start = c("vm(gen, G)!sigma2" = 0.3, "R!units!sigma2" = 0.6),
       fix = c("vm(gen, G)!sigma2", "R!units!sigma2"))

## Large data set: sparse engine with the inverse of A
Ai <- Ainverse(ped)
mlreml(y ~ hys + sex, random = ~ vm(animal, Ai) + id(animal), data = d, engine
= "sparse")
```

16.3 Funções pós-ajuste

```r
summary(fit)                          # variance components
coef(fit, "random", "vm(gen, G)")     # BLUP, PEV, reliability
wald(fit, ddf = "KR")                 # tests of fixed effects (Kenward-Roger)
predict(fit, classify = "env:gen")    # predicted means and SED
```

```
vpredict(fit, h2 ~ V1 / (V1 + V2))     # functions of components
vc_matrix(fit, "us(env):vm(gen, G)")   # covariance and correlation matrices
lrt(fit0, fit1)                        # likelihood ratio
variogram(fit, "row", "column")        # sample variogram
plot(fit)                              # diagnostics
```

Referências

Baker, R.J. (1978). Issues in diallel analysis. *Crop Science* 18, 533–536.

Bates, D., Mächler, M., Bolker, B., Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software* 67(1), 1–48.

Belenky, G. et al. (2003). Patterns of performance degradation and restoration during sleep restriction and subsequent recovery. *Journal of Sleep Research* 12, 1–12.

Besag, J., Higdon, D. (1999). Bayesian analysis of agricultural field experiments. *Journal of the Royal Statistical Society B* 61, 691–746.

Burgueño, J., de los Campos, G., Weigel, K., Crossa, J. (2012). Genomic prediction of breeding values when modeling genotype × environment interaction using pedigree and dense molecular markers. *Crop Science* 52, 707–719.

Butler, D.G., Cullis, B.R., Gilmour, A.R., Gogel, B.J., Thompson, R. (2017). *ASReml-R Reference Manual Version 4*. VSN International, Hemel Hempstead.

Chen, Y., Davis, T.A., Hager, W.W., Rajamanickam, S. (2008). Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software* 35(3), 22.

Christensen, O.F., Lund, M.S. (2010). Genomic prediction when some animals are not genotyped. *Genetics Selection Evolution* 42, 2.

Christensen, O.F., Madsen, P., Nielsen, B., Ostersen, T., Su, G. (2012). Single-step methods for genomic evaluation in pigs. *Animal* 6, 1565–1571.

Cockerham, C.C., Weir, B.S. (1977). Quadratic analyses of reciprocal crosses. *Biometrics* 33, 187–203.

Crossa, J. et al. (2010). Prediction of genetic values of quantitative traits in plant breeding using pedigree and molecular markers. *Genetics* 186, 713–724.

Crowder, M.J., Hand, D.J. (1990). *Analysis of Repeated Measures*. Chapman and Hall, London.

Cullis, B.R., Smith, A.B., Coombes, N.E. (2006). On the design of early generation variety trials with correlated data. *Journal of Agricultural, Biological and Environmental Statistics* 11, 381–393.

Erisman, A.M., Tinney, W.F. (1975). On computing certain elements of the inverse of a sparse matrix. *Communications of the ACM* 18, 177–179.

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *Journal of Agricultural, Biological and Environmental Statistics* 2, 269–293.

Gilmour, A.R., Gogel, B.J., Cullis, B.R., Welham, S.J., Thompson, R. (2015). *ASReml User Guide Release 4.1*. VSN International, Hemel Hempstead.

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440–1450.

Griffing, B. (1956). Concept of general and specific combining ability in relation to diallel crossing systems. *Australian Journal of Biological Sciences* 9, 463–493.

Halekoh, U., Højsgaard, S. (2014). A Kenward–Roger approximation and parametric bootstrap methods for tests in linear mixed models — the R package pbkrtest. *Journal of Statistical Software* 59(9).

Hayman, B.I. (1954). The analysis of variance of diallel tables. *Biometrics* 10, 235–244.

Henderson, C.R. (1975). Best linear unbiased estimation and prediction under a selection model. *Biometrics* 31, 423–447.

Henderson, C.R. (1976). A simple method for computing the inverse of a numerator relationship matrix used in prediction of breeding values. *Biometrics* 32, 69–83.

Henderson, C.R. (1984). *Applications of Linear Models in Animal Breeding*. University of Guelph.

John, J.A., Williams, E.R. (1995). *Cyclic and Computer Generated Designs*. Chapman and Hall, London.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *Journal of Dairy Science* 78, 449–456.

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709–1723.

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983–997.

Kirkpatrick, M., Lofsvold, D., Bulmer, M. (1990). Analysis of the inheritance, selection and evolution of growth trajectories. *Genetics* 124, 979–993.

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *Journal of Dairy Science* 92, 4656–4663.

Lin, L., Yang, C., Meza, J.C., Lu, J., Ying, L., E, W. (2011). SelInv — an algorithm for selected inversion of a sparse symmetric matrix. *ACM Transactions on Mathematical Software* 37(4), 40.

Meuwissen, T.H.E., Hayes, B.J., Goddard, M.E. (2001). Prediction of total genetic value using genome-wide dense marker maps. *Genetics* 157, 1819–1829.

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genetics Selection Evolution* 24, 305–313.

Meyer, K. (1992). Variance components due to direct and maternal effects for growth traits of Australian beef cattle. *Livestock Production Science* 31, 179–204.

Misztal, I., Legarra, A., Aguilar, I. (2014). Using recursion to compute the inverse of the genomic relationship matrix. *Journal of Dairy Science* 97, 3943–3952.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *Journal of Dairy Science* 76, 1479–1483.

Patterson, H.D., Thompson, R. (1971). Recovery of inter-block information when block sizes are unequal. *Biometrika* 58, 545–554.

Pérez, P., de los Campos, G. (2014). Genome-wide regression and prediction with the BGLR statistical package. *Genetics* 198, 483–495.

Piepho, H.P., Möhring, J., Schulz-Streeck, T., Ogutu, J.O. (2012). A stage-wise approach for the analysis of multi-environment trials. *Biometrical Journal* 54, 844–860.

Pinheiro, J.C., Bates, D.M. (2000). *Mixed-Effects Models in S and S-PLUS*. Springer, New York.

Potthoff, R.F., Roy, S.N. (1964). A generalized multivariate analysis of variance model useful especially for growth curve problems. *Biometrika* 51, 313–326.

Quaas, R.L. (1976). Computing the diagonal elements and inverse of a large numerator relationship matrix. *Biometrics* 32, 949–953.

Schaeffer, L.R. (2004). Application of random regression models in animal breeding. *Livestock Production Science* 86, 35–45.

Self, S.G., Liang, K.Y. (1987). Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *Journal of the American Statistical Association* 82, 605–610.

Smith, A.B., Cullis, B.R. (2018). Plant breeding selection tools built on factor analytic mixed models for multi-environment trial data. *Euphytica* 214, 143.

Smith, A.B., Cullis, B.R., Thompson, R. (2001). Analyzing variety by environment data using multiplicative mixed models and adjustments for spatial field trend. *Biometrics* 57, 1138–1147.

Smith, A.B., Cullis, B.R., Thompson, R. (2005). The analysis of crop cultivar breeding and evaluation trials: an overview of current mixed model approaches. *Journal of Agricultural Science* 143, 449–462.

Stram, D.O., Lee, J.W. (1994). Variance components testing in the longitudinal mixed effects model. *Biometrics* 50, 1171–1177.

Takahashi, K., Fagan, J., Chen, M.S. (1973). Formation of a sparse bus impedance matrix and its application to short circuit study. *Proceedings of the 8th Power Industry Computer Applications (PICA) Conference*, Minneapolis, 63–69.

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *Journal of Dairy Science* 91, 4414–4423.

Vazquez, A.I., Bates, D.M., Rosa, G.J.M., Gianola, D., Weigel, K.A. (2010). Technical note: an R package for fitting generalized linear mixed models in animal breeding. *Journal of Animal Science* 88, 497–504.

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Australian and New Zealand Journal of Statistics* 46, 325–347.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18–27.

Willham, R.L. (1972). The role of maternal effects in animal breeding: III. Biometrical aspects of maternal effects in animals. *Journal of Animal Science* 35, 1288–1293.

Wright, K. (2024). *agridat: Agricultural Datasets*. R package, CRAN.

Apêndice: manual de referência das funções

Este apêndice reproduz a página de ajuda de cada função do pacote. No R, ela está disponível com `?function_name`.

16.4 `MLREML-package`: MLREML: modelos lineares mistos por REML de Informação Média

16.4.0.1 Descrição

O MLREML ajusta modelos lineares mistos para o melhoramento de plantas e animais por máxima verossimilhança restrita com o algoritmo de Informação Média. A função principal é `mlreml`. Há dois motores de cálculo disponíveis: um motor denso baseado na matriz de variância das observações, que suporta todas as estruturas de variância, e um motor esparso baseado nas equações do modelo misto, na fatoração de Cholesky esparsa e na inversa esparsa de Takahashi, para conjuntos de dados com centenas de milhares a milhões de registros. Guias do usuário em inglês e em espanhol, com a teoria e exemplos resolvidos com dados reais, são instalados com o pacote: `system.file("doc", "MLREML-guide-en.pdf", package = "MLREML")` e `system.file("doc", "MLREML-guide-es.pdf", package = "MLREML")`.

16.4.0.2 Detalhes

Funções principais:

> `mlreml`: ajuste do modelo (incluindo modelos direto-materno com o termo `str()` e o motor esparso).
>
> `summary.mlreml`, `wald` (com graus de liberdade do denominador de Kenward-Roger), `predict.mlreml`, `vpredict`, `lrt`, `vc_matrix`, `variogram`: inferência.
>
> `pedigree_prepare`, `Amatrix`, `Ainverse`, `inbreeding_coef`, `snp_qc`, `Gmatrix`, `align_G`, `Hmatrix`, `bend`: matrizes de parentesco.
>
> `gblup`, `cv_gblup`: GBLUP rápido.

16.4.0.3 Autor(es)

Mantenedor: Osval A. Montesinos-Lopez maintainer@example.org

16.5 `mlreml`: Ajusta um modelo linear misto por REML de Informação Média

16.5.0.1 Descrição

`mlreml` ajusta o modelo linear misto

$$y = X\beta + \sum_k Z_k\, u_k + e, \quad u_k \sim N(0, G_k), \quad e \sim N(0, R)$$

por máxima verossimilhança restrita (REML) com o algoritmo de Informação Média (AI). As matrizes de covariância G_k e R são especificadas com uma sintaxe semelhante à do ASReml, como produtos diretos (de Kronecker) de estruturas de variância.

16.5.0.2 Uso

```
mlreml(
  fixed,
  random = NULL,
  residual = NULL,
  data,
  start = NULL,
  fix = NULL,
  maxit = 50,
  tol = 1e-06,
  pev = TRUE,
  trace = TRUE,
  store = TRUE,
  engine = c("auto", "dense", "sparse")
)
```

16.5.0.3 Argumentos

Argumento	Descrição
`fixed`	fórmula bilateral para os efeitos fixos.
`random`	fórmula unilateral com os termos aleatórios, ou `NULL`.
`residual`	fórmula unilateral para a estrutura residual.
`data`	data frame.
`start`	vetor nomeado opcional de valores iniciais (nomes como em `summary(fit)$varcomp`).
`fix`	vetor de caracteres com os nomes dos parâmetros de variância a serem mantidos fixos em seus valores iniciais.
`maxit`	número máximo de iterações AI.
`tol`	tolerância de convergência para a mudança da log-verossimilhança REML.
`pev`	lógico; calcular as variâncias do erro de predição dos BLUPs.
`trace`	lógico; imprimir o histórico de iterações.
`store`	lógico; manter as matrizes necessárias para `predict.mlreml` e para o ajuste de Kenward-Roger de `wald`.
`engine`	`"auto"`, `"dense"` ou `"sparse"` (ver a seção sobre os motores de cálculo).

16.5.0.4 Valor

Um objeto da classe `"mlreml"`.

16.5.0.5 Especificação dos termos aleatórios e dos resíduos

Cada termo aleatório é um produto de componentes separados por `:`. Cada componente é uma função de estrutura aplicada a um fator, a uma base polinomial de uma covariável ou a um fator de múltipla pertinência:

`id(f)` ou `f`
efeitos independentes, matriz identidade.

`vm(f, K)`
efeitos correlacionados segundo uma matriz conhecida `K` (pedigree A, genômica G, passo único H, parentesco). `K` deve ter rownames que contenham todos os níveis de `f`; pode conter níveis adicionais (indivíduos sem registros), cujos BLUPs também são retornados. `K` também pode ser fornecida por meio de sua inversa, marcada com o atributo `INVERSE = TRUE` (como a retornada por `Ainverse`); o motor esparso então a usa diretamente.

`diag(f)`
efeitos independentes com uma variância diferente para cada nível de `f`.

`us(f)`
matriz de covariância não estruturada (totalmente parametrizada).

`fa(f, k)`
estrutura fator-analítica com `k` fatores, $\Lambda\Lambda' + \Psi$.

`cs(f)`
correlação de simetria composta.

`ar1(f)`
correlação autorregressiva de primeira ordem; se os níveis de `f` forem numéricos, as distâncias reais são usadas, de modo que lacunas são permitidas.

`expc(x)`
correlação exponencial $\rho^{|x_i - x_j|}$ para uma covariável numérica com espaçamento irregular.

`sexp(x, y)`
correlação exponencial isotrópica $\rho^{d_{ij}}$ em coordenadas bidimensionais (dados espaciais irregulares).

`pol(x, k)`, `leg(x, k)`
bases de polinômios brutos e de Legendre normalizados de ordem `k` (regressão aleatória); geralmente combinadas com `us()` ou `diag()`.

`mm(f1, f2)`
fator de múltipla pertinência (por exemplo, capacidade geral de combinação dos dois genitores em um dialelo).

`units`
as unidades de observação (um nível por registro).

Um termo cujos componentes são todos estruturas de correlação (`id`, `vm`, `cs`, `ar1`, `expc`, `sexp`) recebe uma variância de escala. No máximo uma estrutura de variância (`diag`, `us`, `fa`) é permitida por termo, e ela carrega a escala.

A fórmula `residual` usa a mesma sintaxe; o padrão é `~ units` (resíduos independentes com uma variância comum) e, para modelos multivariados, `~ us(trait):units`.

16.5.0.6 Efeitos correlacionados de vários fatores (modelos direto-materno)

O termo `str(f1, f2, ..., K = A, cov = "us")` define $m \geq 2$ efeitos aleatórios que compartilham os níveis da matriz `K` e são correlacionados entre si: com $u = ({u_1}', \ldots, {u_m}')'$ e $Z = [Z_1, \ldots, Z_m]$,

$$Var(u) = \Sigma \otimes K,$$

em que Σ é uma matriz $m \times m$ não estruturada (`cov = "us"`) ou diagonal (`cov = "diag"`). A aplicação usual é o modelo animal direto-materno, `str(animal, dam, K = A)`, no qual Z_1 liga cada registro ao animal e Z_2 à sua mãe, e Σ contém as variâncias genéticas direta e materna e sua covariância. Um valor ausente de um fator de `str()` (por exemplo, uma mãe desconhecida) significa que o registro não recebe efeito dele; esses registros são mantidos. Se `K` for omitido, os níveis são independentes ($K = I$).

16.5.0.7 Motores de cálculo

O motor `"dense"` trabalha com a matriz $n \times n$ $V = Var(y)$ e suporta todas as estruturas; seu custo cresce com n^3 e sua memória com n^2, de modo que se destina a até alguns milhares de registros. O motor `"sparse"` trabalha com as equações do modelo misto de Henderson, uma fatoração de Cholesky esparsa (CHOLMOD, por meio do pacote Matrix) e a inversa esparsa (de Takahashi), de modo que seu custo depende do número de elementos não nulos e não de n^2; ele processa de centenas de milhares a milhões de registros. Suporta termos aleatórios construídos com componentes `id`, `vm`, `diag` e `us` (com fatores ou bases `pol`, `leg` ou `mm`) e termos `str()`, e as estruturas residuais `units`, `diag(f):units` e, em modelos multivariados, `us(trait):units`. `engine = "auto"` (padrão) usa o motor esparso quando há mais de 5000 registros e o modelo é suportado, e o motor denso caso contrário. Ambos os motores fornecem as mesmas estimativas, BLUPs e variâncias do erro de predição até a precisão numérica.

16.5.0.8 Modelos multivariados

Se o lado esquerdo de `fixed` for `cbind(y1, y2, ...)`, os dados são empilhados internamente, um fator `trait` é criado e `units` identifica os registros originais, de modo que `us(trait):units` é a matriz de covariância residual entre caracteres medidos no mesmo registro. Registros com alguns caracteres ausentes são mantidos para os caracteres observados.

16.5.0.9 Referências

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440-1450.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *J. Dairy Sci.* 78, 449-456.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *J. Dairy Sci.* 76, 1479-1483.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18-27.

16.5.0.10 Exemplos

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)),
                gen = factor(rep(1:30, 4)))
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d)
summary(fit)
```

16.6 summary.mlreml: Métodos print e summary para ajustes mlreml

16.6.0.1 Descrição

Métodos print e summary para ajustes mlreml

16.6.0.2 Uso

```
## S3 method for class 'mlreml'
summary(object, ...)

## S3 method for class 'summary.mlreml'
print(x, ...)

## S3 method for class 'mlreml'
print(x, ...)

## S3 method for class 'mlreml'
fitted(object, ...)

## S3 method for class 'mlreml'
residuals(object, ...)

## S3 method for class 'mlreml'
vcov(object, ...)

## S3 method for class 'mlreml'
logLik(object, ...)
```

16.6.0.3 Argumentos

Argumento	Descrição
object, x	um objeto da classe "mlreml".
...	argumentos adicionais (ignorados).

16.6.0.4 Valor

summary retorna uma lista com os componentes de variância (varcomp), os efeitos fixos com erros padrão (coef.fixed), a log-verossimilhança e os critérios de informação.

16.7 coef.mlreml: Extrai efeitos fixos e aleatórios

16.7.0.1 Descrição

Extrai efeitos fixos e aleatórios

16.7.0.2 Uso

```
## S3 method for class 'mlreml'
coef(object, type = c("fixed", "random"), term = NULL, ...)

ranef_mlreml(object, term = NULL)
```

```
fixef_mlreml(object)
```

16.7.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"`.
`type`	`"fixed"` ou `"random"`.
`term`	para efeitos aleatórios, o rótulo de um termo aleatório (como em `names(object$blups)`); `NULL` retorna todos os termos.
`...`	ignorado.

16.7.0.4 Valor

Um vetor nomeado (efeitos fixos) ou um data frame (BLUPs com variância do erro de predição, erro padrão e confiabilidade).

16.8 `wald`: Testes de Wald para efeitos fixos

16.8.0.1 Descrição

Calcula estatísticas de Wald incrementais e condicionais para cada termo da fórmula fixa, com testes F cujos graus de liberdade do denominador são obtidos pelo método de Kenward e Roger (1997) ou, como aproximação para grandes amostras, pelos graus de liberdade residuais $n - rank(X)$.

16.8.0.2 Uso

```
wald(object, ddf = c("auto", "KR", "residual"))
```

16.8.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"`.
`ddf`	`"auto"` (Kenward-Roger para ajustes do motor denso armazenados com `store = TRUE`, graus de liberdade residuais caso contrário), `"KR"` ou `"residual"`.

16.8.0.4 Detalhes

A estatística incremental de um termo o testa ajustado para os termos que o precedem na fórmula fixa (testes sequenciais, do "tipo I"). A estatística condicional testa um termo ajustado para todos os demais termos que não o contêm, respeitando o princípio da marginalidade como faz o ASReml (equivalente aos testes do "tipo II"): o efeito principal `A` em `y ~ A * B` é ajustado para `B`, mas não para `A:B`. Ambas as estatísticas são calculadas a partir do fator de Cholesky da inversa da matriz de covariância dos efeitos fixos, com as colunas reordenadas para a estatística condicional (o termo depois dos termos que não o contêm e antes dos termos que o contêm).

Com o método de Kenward-Roger, a estatística de cada hipótese $L\beta = 0$ é $F = \lambda(L\hat{\beta})'(L\Phi_A L')^{-1}(L\hat{\beta})/q$, em que Φ_A é a matriz de covariância dos efeitos fixos ajustada para a incerteza dos parâmetros de variância estimados, e os graus de liberdade do denominador m e a escala λ são obtidos igualando os momentos de F aos de uma distribuição $F(q, m)$. É usada a matriz de informação esperada; as segundas derivadas de V são incluídas para as estruturas que não são lineares em seus parâmetros (`ar1`, `expc`, `sexp`, `fa`, produtos de componentes

parametrizados). Os parâmetros de variância fixos ou na fronteira do espaço paramétrico são tratados como conhecidos.

16.8.0.5 Valor

Um data frame com uma linha por termo fixo: graus de liberdade do numerador (`df`); estatística de Wald incremental, estatística F, graus de liberdade do denominador e valor-p (`wald.inc`, `F.inc`, `ddf.inc`, `p.inc`); o mesmo para o teste condicional (`wald.con`, `F.con`, `ddf.con`, `p.con`); e o valor-p da estatística de Wald condicional referida a uma distribuição qui-quadrado (`p.chisq`). O atributo `"ddf"` indica o método usado.

16.8.0.6 Referências

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983-997.

Halekoh, U., Hojsgaard, S. (2014). A Kenward-Roger approximation and parametric bootstrap methods for tests in linear mixed models - the R package pbkrtest. *J. Stat. Softw.* 59(9).

16.8.0.7 Exemplos

```r
set.seed(2)
d <- data.frame(trt = gl(3, 20), blk = gl(4, 5, 60))
d$y <- c(0, 1, 2)[d$trt] + rnorm(4)[d$blk] + rnorm(60)
fit <- mlreml(y ~ trt, random = ~ blk, data = d, trace = FALSE)
wald(fit)
wald(fit, ddf = "residual")
```

16.9 `predict.mlreml`: Médias preditas com erros padrão e erros padrão das diferenças

16.9.0.1 Descrição

Forma predições para os níveis de um ou mais fatores (`classify`), calculando a média dos efeitos fixos sobre os níveis dos outros fatores fixos (média simples) e avaliando as covariáveis em sua média. Os termos aleatórios cujos fatores estão todos contidos em `classify` são incluídos como BLUPs; os demais termos aleatórios são ignorados (sua esperança é zero). Os erros padrão e a matriz de erros padrão das diferenças (SED) são calculados a partir da variância conjunta do erro de predição dos efeitos fixos e aleatórios, seguindo Welham et al. (2004).

16.9.0.2 Uso

```r
## S3 method for class 'mlreml'
predict(object, classify, sed = TRUE, ...)
```

16.9.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"` (ajustado com `store = TRUE`).
`classify`	caractere, por exemplo `"gen"` ou `"env:gen"`.
`sed`	lógico; retornar a matriz de SED.
`...`	ignorado.

16.9.0.4 Valor

Uma lista com `pvals` (valores preditos e erros padrão), `sed` (matriz) e `avsed` (SED médio).

16.9.0.5 Referências

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Aust. N. Z. J. Stat.* 46, 325-347.

16.10 `vpredict`: Funções dos parâmetros de variância com erros padrão pelo método delta

16.10.0.1 Descrição

Avalia uma função dos parâmetros de variância (por exemplo, uma herdabilidade ou uma correlação genética) e seu erro padrão aproximado a partir da inversa da matriz de Informação Média, como a função `vpredict` do ASReml-R. Os parâmetros são referidos como `V1`, `V2`, ... na ordem de `summary(fit)$varcomp`.

16.10.0.2 Uso

```
vpredict(object, formula)
```

16.10.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"`.
`formula`	uma fórmula como `h2 ~ V2 / (V2 + V3)`.

16.10.0.4 Valor

Um data frame com a estimativa e seu erro padrão.

16.10.0.5 Exemplos

```
set.seed(3)
d <- data.frame(fam = gl(40, 5))
d$y <- rnorm(40, sd = 1)[d$fam] + rnorm(200, sd = 2)
fit <- mlreml(y ~ 1, random = ~ fam, data = d, trace = FALSE)
vpredict(fit, h2 ~ 4 * V1 / (V1 + V2))
```

16.11 `lrt`: Teste da razão de verossimilhanças entre modelos aninhados

16.11.0.1 Descrição

Compara dois modelos com os mesmos efeitos fixos que diferem em sua estrutura de variância. Com `boundary = TRUE`, o valor-p usa uma mistura 50:50 de distribuições qui-quadrado, apropriada quando um único parâmetro de variância é testado na fronteira do espaço paramétrico.

16.11.0.2 Uso

```
lrt(fit0, fit1, boundary = TRUE)
```

16.11.0.3 Argumentos

Argumento	Descrição
`fit0`	modelo reduzido.
`fit1`	modelo completo.
`boundary`	lógico; usar a distribuição de mistura.

16.11.0.4 Valor

Um data frame com a estatística, os graus de liberdade e o valor-p.

16.12 `vc_matrix`: Matriz de covariância e de correlação estimada de um componente estruturado

16.12.0.1 Descrição

Retorna a matriz de covariância estimada de um componente `us`, `diag` ou `fa` (por exemplo, a matriz de covariância genética entre ambientes ou caracteres), sua matriz de correlação e, para `fa`, as cargas e as variâncias específicas.

16.12.0.2 Uso

```
vc_matrix(object, term)
```

16.12.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"`.
`term`	rótulo do termo (como em `object$varcomp$term`), ou `"residual"`.

16.12.0.4 Valor

Uma lista com `cov` e `cor` (e `loadings`, `psi` para estruturas fator-analíticas).

16.13 `variogram`: Variograma amostral dos resíduos de um ensaio de campo

16.13.0.1 Descrição

Calcula o semivariograma empírico dos resíduos por deslocamento absoluto de linha e de coluna, o diagnóstico padrão das análises espaciais (Gilmour, Cullis e Verbyla, 1997).

16.13.0.2 Uso

```
variogram(object, row, col, plot = TRUE, resid = NULL)
```

16.13.0.3 Argumentos

Argumento	Descrição
`object`	um ajuste `"mlreml"`.
`row`, `col`	nomes das variáveis numéricas de linha e de coluna nos dados.
`plot`	lógico; desenhar um gráfico em perspectiva.
`resid`	vetor opcional de resíduos (padrão: resíduos condicionais).

16.13.0.4 Valor

Um data frame com a defasagem de linha, a defasagem de coluna, a semivariância e o número de pares.

16.13.0.5 Referências

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *JABES* 2, 269-293.

16.14 `plot.mlreml`: Gráficos de diagnóstico

16.14.0.1 Descrição

Resíduos versus valores ajustados, gráfico de quantis normais e histograma dos resíduos, e o histórico da log-verossimilhança REML.

16.14.0.2 Uso

```
## S3 method for class 'mlreml'
plot(x, ...)
```

16.14.0.3 Argumentos

Argumento	Descrição
`x`	um ajuste `"mlreml"`.
`...`	passados para `plot`.

16.15 `pedigree_prepare`: Prepara um pedigree

16.15.0.1 Descrição

Recodifica pais desconhecidos, adiciona os pais que não aparecem como indivíduos (como fundadores), verifica erros e ordena o pedigree de modo que os pais precedam sua progênie.

16.15.0.2 Uso

```
pedigree_prepare(ped)
```

16.15.0.3 Argumentos

Argumento	Descrição
`ped`	data frame cujas três primeiras colunas são indivíduo, pai (sire) e mãe (dam). Pais desconhecidos podem ser codificados como `NA`, `0`, `"0"`, `"-"` ou `""`.

16.15.0.4 Valor

Um data frame com as colunas `id`, `sire`, `dam` (caractere, `NA` para desconhecido), `sire_i`, `dam_i` (posições inteiras, 0 para desconhecido) e `gen` (número da geração).

16.15.0.5 Exemplos

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
pedigree_prepare(ped)
```

16.16 `inbreeding_coef`: Coeficientes de endogamia (Meuwissen e Luo, 1992)

16.16.0.1 Descrição

Coeficientes de endogamia (Meuwissen e Luo, 1992)

16.16.0.2 Uso

```
inbreeding_coef(ped)
```

16.16.0.3 Argumentos

Argumento	Descrição
`ped`	um pedigree como o retornado por `pedigree_prepare`, ou qualquer data frame aceito por ela.

16.16.0.4 Valor

Um vetor nomeado de coeficientes de endogamia.

16.16.0.5 Referências

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genet. Sel. Evol.* 24, 305-313.

16.17 `Amatrix`: Matriz de parentesco do numerador (aditiva)

16.17.0.1 Descrição

Constrói a matriz densa de parentesco aditivo A com o método tabular. Adequada para até aproximadamente 15.000 indivíduos; para pedigrees maiores, use a inversa esparsa `Ainverse`.

16.17.0.2 Uso

```
Amatrix(ped)
```

16.17.0.3 Argumentos

Argumento	Descrição
`ped`	pedigree (ver `pedigree_prepare`).

16.17.0.4 Valor

Uma matriz simétrica com os nomes dos indivíduos como dimnames.

16.17.0.5 Exemplos

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
Amatrix(ped)
```

16.18 `Ainverse`: Inversa esparsa da matriz de parentesco aditivo

16.18.0.1 Descrição

Regras de Henderson com endogamia (Quaas, 1976), usando os coeficientes de endogamia de Meuwissen e Luo (1992).

16.18.0.2 Uso

```
Ainverse(ped)
```

16.18.0.3 Argumentos

Argumento	Descrição
`ped`	pedigree (ver `pedigree_prepare`).

16.18.0.4 Valor

Uma matriz simétrica esparsa (pacote `Matrix`) com o atributo `INVERSE = TRUE`, de modo que pode ser fornecida diretamente a `vm()` ou `str()` em `mlreml` (o motor esparso a usa como está; o motor denso a inverte).

16.19 `snp_qc`: Controle de qualidade de uma matriz de SNPs

16.19.0.1 Descrição

Controle de qualidade de uma matriz de SNPs

16.19.0.2 Uso

```
snp_qc(M, maf = 0.05, callrate = 0.9, ind_callrate = 0.9)
```

16.19.0.3 Argumentos

Argumento	Descrição
`M`	matriz de genótipos (indivíduos nas linhas), codificada como 0/1/2 ou -1/0/1; valores ausentes como `NA`.
`maf`	frequência mínima do alelo menos frequente.
`callrate`	taxa de chamada mínima por marcador.
`ind_callrate`	taxa de chamada mínima por indivíduo.

16.19.0.4 Valor

Uma lista com a matriz filtrada (`M`, codificada como 0/1/2), as frequências alélicas e um relatório dos marcadores e indivíduos removidos.

16.20 `Gmatrix`: Matriz de parentesco genômico

16.20.0.1 Descrição

Matriz de parentesco genômico

16.20.0.2 Uso

```
Gmatrix(M, method = c("VanRaden", "Yang", "scaled"), p = NULL)
```

16.20.0.3 Argumentos

Argumento	Descrição
`M`	matriz de genótipos (indivíduos nas linhas) codificada como 0/1/2 ou -1/0/1. Valores ausentes são substituídos pela média do marcador.
`method`	`"VanRaden"` (VanRaden 2008, método 1), `"Yang"` (marcadores padronizados, método 2 de VanRaden) ou `"scaled"` (marcadores centrados e escalonados divididos pelo número de marcadores, ZZ'/p, o kernel do GBLUP como regressão ridge).
`p`	vetor opcional de frequências alélicas de base.

16.20.0.4 Valor

Uma matriz simétrica com os rownames de `M`.

16.20.0.5 Referências

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *J. Dairy Sci.* 91, 4414-4423.

16.21 `align_G`: Alinha uma matriz genômica com a matriz de pedigree

16.21.0.1 Descrição

Reescalona G como $\alpha + \beta G$ de modo que as médias de sua diagonal e de todos os seus elementos sejam iguais às de A22 (Christensen et al., 2012).

16.21.0.2 Uso

```
align_G(G, A22)
```

16.21.0.3 Argumentos

Argumento	Descrição
`G`	matriz de parentesco genômico.
`A22`	parentescos por pedigree dos indivíduos genotipados.

16.21.0.4 Valor

A matriz alinhada, com os atributos `alpha` e `beta`.

16.22 `Hmatrix`: Matriz de parentesco de passo único H e sua inversa

16.22.0.1 Descrição

Combina a matriz de pedigree A de todos os indivíduos com a matriz genômica G dos indivíduos genotipados (Legarra et al., 2009; Aguilar et al., 2010). Opcionalmente, G é combinada com A22 e alinhada a ela antes da combinação.

16.22.0.2 Uso

```
Hmatrix(A, G, blend = 0.05, align = TRUE, tau = 1, omega = 1, inverse = FALSE)
```

16.22.0.3 Argumentos

Argumento	Descrição
`A`	matriz de parentesco por pedigree de todos os indivíduos (densa).
`G`	matriz de parentesco genômico dos indivíduos genotipados; seus rownames devem estar em `rownames(A)`.
`blend`	peso de A22 em $(1 - w)G + wA_{22}$.
`align`	lógico; alinhar G a A22 com `align_G`.
`tau, omega`	fatores de escala de G^{-1} e A_{22}^{-1} na inversa.
`inverse`	lógico; retornar H^{-1} em vez de H.

16.22.0.4 Valor

H (ou sua inversa) com os dimnames de A.

16.22.0.5 Referências

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *J. Dairy Sci.* 92, 4656-4663.

16.23 `bend`: Torna uma matriz simétrica positiva definida (bending)

16.23.0.1 Descrição

Torna uma matriz simétrica positiva definida (bending)

16.23.0.2 Uso

```
bend(K, tol = 1e-04)
```

16.23.0.3 Argumentos

Argumento	Descrição
`K`	matriz simétrica.
`tol`	autovalor mínimo após o bending (relativo ao maior).

16.23.0.4 Valor

A matriz corrigida (bent).

16.24 `gblup`: GBLUP rápido por decomposição espectral

16.24.0.1 Descrição

Ajusta $y = X\beta + g + e$, $g \sim N(0, K\sigma_g^2)$, $e \sim N(0, I\sigma_e^2)$ por REML exato usando a autodecomposição da matriz de parentesco dos indivíduos fenotipados (Kang et al., 2008), o que reduz o REML a uma otimização unidimensional. As predições (GEBVs) são retornadas para todos os indivíduos em `K`, fenotipados ou não. Os resultados são idênticos aos de `mlreml(y ~ ..., random = ~ vm(id, K))`, mas muito mais rápidos.

16.24.0.2 Uso

```
gblup(y, K, X = NULL, id = names(y))
```

16.24.0.3 Argumentos

Argumento	Descrição
`y`	vetor numérico de fenótipos, nomeado por indivíduo, ou com os identificadores dos indivíduos fornecidos em `id`.
`K`	matriz de parentesco com rownames.
`X`	matriz de delineamento opcional dos efeitos fixos (padrão: intercepto).
`id`	vetor de caracteres opcional de identificadores para `y`.

16.24.0.4 Valor

Uma lista com `varcomp` (σ_g^2, σ_e^2, h^2), `beta`, `gebv` (vetor nomeado para todos os indivíduos em `K`) e `loglik`.

16.24.0.5 Referências

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709-1723.

16.25 `cv_gblup`: Validação cruzada K-fold do GBLUP

16.25.0.1 Descrição

Particiona aleatoriamente os indivíduos fenotipados em `folds` grupos, prediz cada grupo a partir dos demais com `gblup` e relata a correlação de Pearson entre os valores preditos e os observados.

16.25.0.2 Uso

```
cv_gblup(y, K, folds = 5, reps = 1, seed = 1)
```

16.25.0.3 Argumentos

Argumento	Descrição
`y`	vetor nomeado de fenótipos.
`K`	matriz de parentesco.
`folds`	número de grupos (folds).
`reps`	número de repetições da partição aleatória.
`seed`	semente aleatória.

16.25.0.4 Valor

Um data frame com uma linha por repetição e grupo.